DTU Compute
Department of Applied Mathematics and Computer Science

Agent-System Interface Designs for Enterprise Platforms

A Comparative Empirical Study

Master Thesis



Agent-System Interface Designs for Enterprise Platforms
A Comparative Empirical Study

Master Thesis
26th of June 2026

By
Alexander Nygård

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: Vibeke Hempler, 2012

Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Building 324, DK-2800 Kgs. Lyngby Denmark

www.compute.dtu.dk

Approval

This thesis has been prepared at DTU Compute, Department of Applied Mathematics and Computer Science, at the Technical University of Denmark (DTU), in partial fulfilment of the requirements for the degree Master of Science.

The thesis investigates how harness architecture influences the capability, reliability, and efficiency of LLM agents operating on enterprise architecture tasks in Ardoq.

Alexander Nygård - s253921



Signature

26th of June 2026

Date

Abstract

Although large language model (LLM) agents reason effectively, their practical value in enterprise settings is often limited by how they interface with target software. This thesis studies the *agent harness* (the non-model infrastructure handling actions, documentation, state placement, observations, and error feedback) as a performance-bearing architectural variable. We evaluate three harness designs using a shared, seeded local deployment of Ardoq, an enterprise architecture platform: traditional tool calling (H1), programmatic tool calling (H2) via sandboxed Python execution, and an agent-native domain module (H3) that raises the abstraction level over the same execution runtime.

The evaluation benchmark consists of 24 read-only analytical tasks based on recurring enterprise architecture workflows. Each task defines a typed output contract verified objectively in real time against the deployment's live Ardoq API. Seven repeated runs per task-harness combination yield 504 evaluation runs.

Shifting from traditional tool calling to programmatic execution (H1$\rightarrow$H2) removes a major state-coordination bottleneck. H2 increases the mean task success rate from 39.29% to 91.67% ($+52.4$ pp). It also reduces mean total tokens by 66%, the final context window by 45%, and estimated model cost per run by 48%. The largest gains concentrate on graph traversal, metamodel analysis, set construction, and cross-source composition; on these tasks, H1 typically fails either by exhausting its turn or time budget as context accumulates or by returning over-inclusive answers.

Raising the abstraction level (H2$\rightarrow$H3) largely preserves correctness while further improving operational efficiency. H3's exact pass rate is 91.07%, only 0.60 percentage points below H2, with no median per-task difference. It reduces mean total tokens by 18.9%, the final context window by 10.3%, per-task model cost by 15.2%, and wall-clock latency is 11.5% lower. One possible reason for H3's slightly lower success rate is that its domain-tailored interface exposes less exploratory data, which saves tokens but may also limit the agent's ability and inclination to explore when needed.

The findings support programmatic tool calling as the default for compositional enterprise workflows when code execution is operationally viable. Traditional tool calling remains sufficient for narrow report-based tasks. High-level domain modules are a selective trade-off, justified when stable, recurring workflows warrant the governance and maintenance costs of the abstraction. These conclusions are limited to one frontier model, one seeded Ardoq deployment, and read-only analytical tasks.

Keywords: LLM agents, agent harnesses, Enterprise Architecture, Ardoq, tool calling, code execution, programmatic tool use, enterprise AI

Acknowledgements

I would like to thank my main supervisor, Alberto Lafuente, for his guidance throughout the project and for helping me keep the thesis focused as the work developed.

I am also grateful to my secondary supervisor, Alex Jung, for his supervision and for useful perspectives on the methodological and technical parts of the study.

I would also like to thank Ardoq for making this project possible as an industrial thesis project. In particular, I thank Falco Winkler for his support as company supervisor, for helping frame the project in relation to Ardoq's work on agent systems, and for providing practical feedback when needed.

I also thank my friends for making the process more enjoyable, and for tolerating more conversations about agents and enterprise architecture than anyone should reasonably have to.

Finally, I thank my parents and my brother for their support during the thesis period, and for taking the time to read and comment on drafts. I also thank my other siblings for their lack of support and welcomed distractions during this time.





Co-funded by
the European Union



Declaration on the Use of Generative AI

Generative artificial intelligence (AI) tools were used during the work on this thesis. All usage complied with DTU's regulations on the use of generative AI, and I, as the author, take full responsibility for the content, claims, and references of this document. Confidential and Ardoq-internal material was only processed with tools covered by appropriate enterprise data-processing agreements, and no personal or sensitive data was entered into any consumer-grade tool. The tools used and the purposes for which they were used are listed below.

Enterprise tools (Ardoq)

The following tools were provided under Ardoq's enterprise plans and were used exclusively when working with Ardoq's code, internal documentation, and systems. Their use complied with enterprise data-processing requirements in line with Ardoq's internal policies.

- **Gemini (Google)** — discussing and working through Ardoq-specific material.
- **Claude Code (Anthropic)** — writing and debugging the experiment code and other code that interacts with Ardoq's systems.

Personal tools

The following tools were used under private subscriptions for general thesis work that did not involve confidential Ardoq material.

- **Gemini (Google)** and **Claude (Anthropic)** — brainstorming, idea generation, and iterating on concepts.
- **Perplexity** — web and academic search.
- **Claude Code (Anthropic)** and **OpenAI Codex (OpenAI)** — writing and debugging the data-analysis code used to process and interpret the results, as well as spell checking and proofreading the thesis text.

Every AI-assisted change was reviewed by me before being accepted, I ran every experiment myself and checked the outputs against the protocols described in this thesis, and the arguments presented here are my own.

Contents

Preface . . . ii
Abstract . . . iii
Acknowledgements . . . iv
Declaration on the Use of Generative AI . . . v

1 Introduction 1
1.1 Motivation . . . 1
1.2 Research Questions . . . 2
1.3 Scope and Contributions . . . 3
1.4 Paper Structure . . . 4

2 Background 5
2.1 Language Models and Tool Use . . . 5
2.2 From Language Models to Agents . . . 6
2.3 Ardoq and the Enterprise Architecture Domain . . . 7
2.4 Compositional Demands of Enterprise Analysis . . . 13
2.5 Interaction Scale and Code-Execution Agents . . . 13
2.6 Three Agent Harness Architectures . . . 14

3 Related Work 19
3.1 LLM Agents and Tool Use . . . 19
3.2 Code Generation for System Interaction . . . 20
3.3 Agent Architectures, State, and Domain Abstraction . . . 21
3.4 Enterprise AI and EA Automation . . . 22
3.5 Benchmark Design for Agent Evaluation . . . 23
3.6 Summary and Gap . . . 24

4 Methodology 25
4.1 Research Design and Comparisons . . . 25
4.2 The Ardoq Benchmark Suite . . . 27
4.3 Architecture Configurations . . . 31
4.4 Harness-Configuration Design-Validation Studies . . . 33
4.5 Experimental Procedure . . . 34
4.6 Evaluation Metrics . . . 35
4.7 Analysis Plan . . . 35
4.8 Validity and Limitations . . . 36

5 Implementation 39
5.1 Implementation goals and architecture . . . 39
5.2 Common experimental foundation . . . 40
5.3 From implementation artifacts to model-visible interfaces . . . 41
5.4 H1: Traditional tool-calling harness . . . 43
5.5 Shared Python execution service . . . 44
5.6 H2: Programmatic harness . . . 46
5.7 H3: Agent-native domain module harness . . . 47
5.8 Example workflow across the three harnesses . . . 51
5.9 Telemetry, validation, and measurement boundaries . . . 51

6 Evaluation **53**
6.1 Overview . . . 53
6.2 Harness-Configuration Design-Validation Results . . . 54
6.3 Main Harness Comparison . . . 54
6.4 Where the Differences Occur . . . 56
6.5 Operational Behaviour . . . 61
6.6 Failure Behaviour . . . 64
6.7 Discussion . . . 67

7 Conclusion **73**
7.1 Answer to the Research Question . . . 73
7.2 Contributions . . . 74
7.3 Practical Implications . . . 74
7.4 Scope of the Conclusions . . . 75
7.5 Future Work . . . 75
7.6 Closing Perspective . . . 76

Bibliography **77**

A Appendix **83**
A.1 Ardoq Definitions . . . 83
A.2 Full Statistical Diagnostics . . . 83
A.3 Benchmark Task Catalogue . . . 85
A.4 Supplementary Evaluation Tables . . . 95
A.5 Harness System Prompts . . . 96
A.6 H3 Tool-Description Patterns and SDK Examples . . . 98
A.7 Post-Run Reflection Schema and Prompt . . . 100
A.8 Representative Trial Trace . . . 102
A.9 Task-suite Validation Survey Instrument . . . 105

1 Introduction

1.1 Motivation

Large language models (LLMs) have moved rapidly into workplace use. McKinsey reports a sharp increase in regular organisational use between 2023 and 2024, while Microsoft and LinkedIn found that 75% of surveyed knowledge workers used generative AI at work in 2024 [1, 2, 3]. McKinsey estimates that generative AI could add $2.6–$4.4 trillion in annual value across analysed business use cases, provided that organisations can adopt the technology and redeploy work effectively [4].

Realised value remains uneven. Few organisations attribute a meaningful share of earnings to generative AI, and many remain between experimentation and scaled use [2, 5]. Productivity depends not only on the model, but also on task context, expertise, workflow design, data, governance, and human–AI collaboration [6, 5]. For enterprise software, the LLM must be connected to the applications, data models, permissions, validation rules, and operating procedures through which work is performed.

The agent–system interface is therefore a central design problem. An enterprise agent must discover and parameterise actions, preserve intermediate state, and recover from errors. The *agent harness*, the non-model infrastructure between the LLM and target system, handles these concerns. Recent work supports treating this harness as an architectural variable: tool representation and composition can affect reliability without changing the LLM [7, 8]. Tool-use research also increasingly studies system-level reliability across multi-tool trajectories [9].

Efficiency is part of the same problem: token-heavy traces increase operational cost and can increase latency as observations accumulate. Model computation also has an environmental cost. MIT's review of generative AI's environmental impact notes that generative-AI workloads are contributing to rising data-centre power demand, creating pressure to improve the computational efficiency of training and inference [10]. Harness design affects both operating economics and the model computation needed for a workflow.

One response is to let the LLM write code that composes operations and retains intermediate state. Anthropic reports large token savings for this pattern, and Cloudflare uses typed code interfaces and lightweight sandboxes to expose large API surfaces [11, 12, 13]. These vendor reports establish practical interest rather than independent evidence.

Independent evidence also finds that executable Python can improve success and reduce action steps on API tasks [14]. Whether this generalises to enterprise platforms, which workflows benefit, and when domain abstractions are needed remain open questions.

Most code-execution approaches make an existing tool or API surface callable without changing its developer-oriented abstractions. This thesis also asks what operations, values, and domain concepts an interface would expose if designed for the agent. That question motivates the third harness in the comparison, which provides an agent-native domain module rather than a thin programmatic wrapper. The idea that an agent's interface should be designed for the agent rather than inherited has begun to attract attention [15, 16]. It has not, however, been brought into a code-execution harness over a real enterprise platform, nor compared in controlled fashion against thinner code wrap-

pers and the traditional tool-calling harness (H1) it is meant to improve. This controlled comparison is the central concern of the thesis.

Enterprise Architecture (EA) platforms such as Ardoq provide a demanding context for studying this question. EA platforms manage relational data through typed components, references, workspaces, reports, viewpoints, and metamodels. An agent must compose operations, interpret platform-specific schemas, preserve identifiers, and ground decisions in the Ardoq model governing the relevant workspace. Prior work on enterprise knowledge graphs similarly argues that LLMs become more useful in enterprise settings when grounded in structured relational representations rather than unstructured text alone [17, 18].

This thesis compares three agent harnesses: the Traditional tool-calling harness (H1), the Programmatic harness (H2), and the Agent-native domain module harness (H3). The model, tasks, and platform remain fixed. The controlled design stages two architectural contrasts: H1–H2 changes invocation style while preserving platform coverage, and H2–H3 changes abstraction level while retaining code generation as the execution mechanism. The study measures how these changes affect task completion, resource use, and failure behaviour.

1.2 Research Questions

The research questions follow the two architectural transitions. Correctness, failures, tokens, and latency provide evidence for each.

RQ: *How does agent harness architecture affect the performance and behaviour of LLM agents on enterprise architecture workflows in Ardoq?*

This overarching question is addressed through three sub-questions.

SQ1: Invocation style. *How does replacing the Traditional tool-calling harness (H1) with the Programmatic harness (H2) affect task success, failure modes, token usage, and latency?*

SQ1 compares structured calls with generated Python over the same low-level Ardoq operations. It tests whether Python can retain intermediate state and compose dependent operations more effectively than turn-by-turn JSON calls. It is motivated by evidence that multi-step, sequentially dependent tasks stress the LLM's ability to preserve intermediate state and compose operations across turns [19, 9]. If a programmatic harness helps, the effect should be clearest in tasks requiring iteration, aggregation, graph traversal, or conditional execution.

SQ2: Abstraction level. *How does replacing a low-level programmatic API with an agent-native domain module affect task success, failure modes, token usage, and latency?*

SQ2 compares low-level Python functions with an agent-native domain module while holding code execution fixed. H3's operations encode recurring Ardoq and EA workflow patterns. Prior work on tool interface design suggests that the representation and composition of available actions can materially affect agent reliability without changing the underlying model [7, 8]. The question is whether domain-shaped operations reduce semantic misuse and state-management failures, or introduce new failures through abstraction mismatch, hidden assumptions, and maintenance burden.

SQ3: Task sensitivity. *Which categories of enterprise architecture tasks benefit most or least from each harness architecture?*

SQ3 examines whether architectural effects vary by task category and difficulty. Aggregate success rates can conceal deployment-relevant variation: an architecture that works well for report aggregation may fail on typed graph traversal or multi-source joins, and one that saves tokens on longer tasks may add overhead on shorter ones [20, 21]. The benchmark therefore records task tags and practitioner-rated difficulty scores to test whether architectural differences widen as compositional demands increase.

SQ1 and SQ2 test the two transitions; SQ3 locates their task sensitivity. Engineering cost informs practical interpretation rather than a separate research question.

1.3 Scope and Contributions

This thesis is a controlled, single-platform case study. Ardoq's typed components, graph traversals, and analytical assets make it a demanding testbed for agent–system interfaces. These characteristics reflect the complex information structures and analytical problems encountered in enterprise architecture. Ardoq's active investment in AI-assisted features for enterprise architecture also makes the platform a practically relevant setting in which to study how LLM agents should interact with enterprise data. The author's part-time employment at Ardoq provided domain insight into the platform and the challenges faced by enterprise architects, as well as access to a local development deployment containing a realistic organisational dataset.

The benchmark created as part of this thesis consists of 24 automatically verifiable, read-only analytical tasks. It targets compositional work that requires multi-step retrieval, traversal, filtering, joining, or aggregation while retaining an objectively verifiable answer. The tasks can provide evidence for a business decision, but do not ask the agent to make that decision. Each task defines a typed output model and a deterministic oracle that independently recomputes the expected answer against the live Ardoq API. This follows the objective-verification principle established by Agent-Diff for enterprise API tasks [20]. The suite excludes subjective assessment, generated-text quality, and LLM-as-a-judge scoring, making success traceable to platform state. The suite covers report discovery and aggregation, component queries, typed graph traversal, viewpoint execution, dashboard drill-down, metamodel reachability, parent resolution, recursive hierarchy aggregation, set operations, multi-source joins, and date-based partitioning. This follows the principle, established in prior benchmark design work, that aggregate performance scores conceal important variation across task families and that per-category analysis is necessary for understanding how harness design interacts with task structure [19, 21].

The thesis makes four contributions. First, it provides a **controlled architectural framing of the agent harness as the unit of comparison**. Prior work has mapped agent-system design dimensions and shown that tool interface representation affects reliability [22, 7, 8]. It has not, however, systematically compared different harness architectures across invocation style and abstraction level within a single enterprise platform. The staged controlled design provides a methodological template for comparing these realised architectural transitions while keeping the model, platform, tasks, and verification fixed.

Second, it provides an **Ardoq-centred benchmark with objectively verifiable tasks**. The benchmark's task design is grounded in recurring EA workflows, including risk analysis, lifecycle management, process quality, capability management, infrastructure cost, data governance, portfolio aggregation, and vulnerability management. Each task is operationalised as a typed evaluation contract with a live-API oracle. The benchmark is architecture-agnostic: the same task definitions, verification conditions, and procedures apply to all three harnesses. Benchmark tasks are reviewed by human experts for their

difficulty, usefulness, and frequency of use in typical EA workflows.

Third, it provides a **comparative evaluation methodology** for correctness, resource efficiency, and error behaviour. The evaluation uses repeated trials per harness-task combination to characterise both mean performance and variability. This follows the methodological principle that single-run task success rates systematically overestimate production reliability [21, 23]. It reports token usage, wall-clock latency, task success rate, and categorised error rates under the same LLM, task suite, and platform conditions.

Fourth, it derives **practical guidance** for enterprise agent integration. The guidance is scoped explicitly to the conditions of the experiment: a single frontier LLM, a single platform, and the final fixed task suite. The conclusions recognise the limits on generalisability. The guidance is intended to characterise trade-offs rather than provide universal prescriptions: which architectures fit which task conditions, where failures concentrate, and what engineering commitments each architecture entails.

The study does not attempt broad claims about all enterprise platforms, evaluate human usability, measure production reliability under sustained multi-user load, or compare multi-agent architectures.

1.4 Paper Structure

Chapter 2 establishes the technical and domain context for comparing agent–system interfaces in enterprise architecture. Chapter 3 positions the study within existing research and identifies the gap addressed by the comparison. Chapter 4 explains how the comparison was designed and controlled, including the design-validation studies used to select the final H1 and H2 variants. Chapter 5 describes how the three harnesses realise this design and how they divide responsibility between the model and the interface. Chapter 6 presents the design-validation findings and the main harness comparison, then relates the observed differences to the interface designs and research questions. Chapter 7 answers the main research question, states the contributions and limits of the study, and outlines directions for future work.

2 Background

This chapter develops the concepts needed for the comparison: tool-using language-model agents, locations of agent state, Ardoq's graph model, and the compositional demands of enterprise analysis. It then defines the three harness architectures.

2.1 Language Models and Tool Use

A large language model (LLM) is a neural network, typically based on the transformer architecture [24], trained to predict successive tokens and subsequently adapted to follow instructions [25, 26]. Instructions, interface descriptions, prior interactions, retrieved data, generated code, and final answers all enter the systems studied here as token sequences. The model consumes a bounded sequence, the *context window*, and generates a continuation one token at a time.

Token generation alone affords no direct access to external state: generating the name of a database operation does not execute it or reveal its result. Connecting an LLM to a database, service, or current platform value therefore requires a surrounding component that interprets a generated request, performs the external action, and returns an observation.

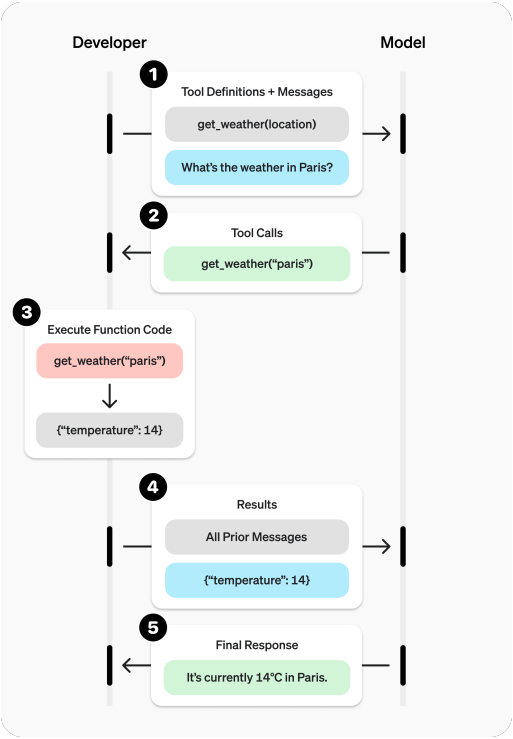


Figure 2.1: Example of a tool calling sequence, adapted from OpenAI's Function Calling API documentation [27].

A common mechanism is *tool calling*, also termed function calling. The model receives specifications of the available tools and emits a structured object, typically JSON, that identifies a tool and supplies its arguments. An orchestrator validates the object, executes the corresponding operation, serialises the result, and appends it to the context as a new observation, as illustrated in Figure 2.1. Tool calling is related to but distinct from *structured output*: structured output constrains a model response to a declared schema, whereas a tool call is a structured request that the surrounding system interprets as an action. This thesis uses both mechanisms: structured calls mediate platform interaction, and task-specific structured outputs constrain final answers so that they can be verified. Schick et al. [28] showed that such behaviour can be acquired by the model itself: through self-supervised training, a model can learn when to invoke a tool, which tool to select, and how to populate its arguments, rather than relying on external logic to govern every decision.

Two properties recur throughout the thesis. The model acts only through an execution layer, and it can use only the operations and observations exposed by that layer. Its behaviour therefore depends on the interface's names, descriptions, types, call representation, result format, and context cost as well as on the model itself.

2.2 From Language Models to Agents

A single LLM invocation cannot adapt its actions to intermediate feedback. An *agent* embeds the model in an iterative control loop: the surrounding system assembles the task, interface description, and retained history; requests an action or final answer; dispatches the action; returns the observation; and repeats until completion or a stopping condition [26]. An agent is, in this operational sense, a model coupled to tools, state, and orchestration rather than the model alone.

In the classical taxonomy of Russell and Norvig [29], the evaluated systems in this thesis are *goal-directed LLM agents* (Figure 2.2): each receives a natural-language objective and structured target schema, then revises its actions in response to observations. They do not learn during evaluation or maintain an explicit transition model.

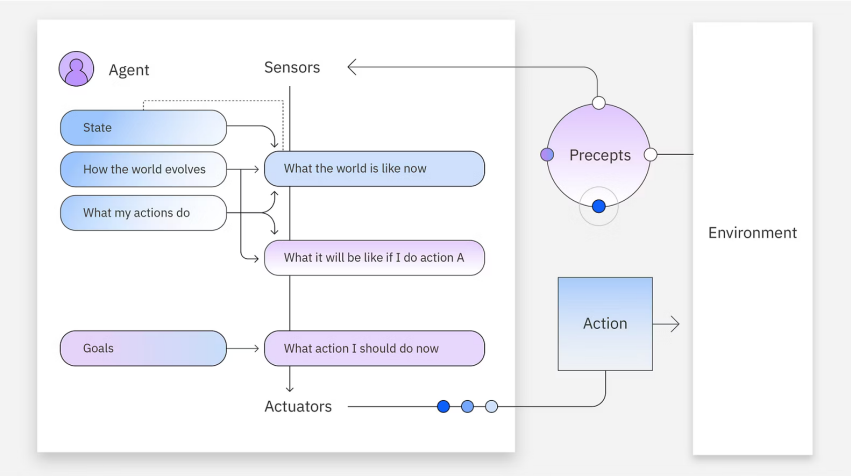


Figure 2.2: Conceptual structure of a goal-directed agent. The model selects actions in relation to an objective and the observations retained by the surrounding system [30].

The idea of coupling model reasoning with external action has direct predecessors. Yao et al. [31] introduced ReAct, which prompts a model to interleave free-text reasoning steps with environment actions and observations; related work treats tool invocation as a learned behaviour [28] and equips agents to reflect on and retry unsuccessful attempts [32]. The harnesses studied in this thesis inherit the observe–act loop these works established, but realise it through tool-calling and code-execution interfaces rather than through ReAct's textual prompting scheme.

The same construction supports coding agents that edit and test repositories [33], web agents that complete multi-step tasks [34], and API agents that select operations from large catalogues [35, 26]. Their performance depends jointly on the model and its interface. Wei [22] survey seventy public agent projects and identify the tool system, context management, and orchestration as recurring architectural dimensions whose design shapes behaviour independently of the model. The layer that mediates between the model and a target system is thus a subject of study in its own right; this thesis terms it the *agent harness* and examines it in Section 2.6.

2.2.1 Locations of Agent State

An iterative agent needs information from earlier steps, but that information can reside in four locations (Table 2.1). *Conversation state* consists of task instructions, prior model messages, tool calls, and observations represented in the model's context window. It is

directly available to the model on a later turn, but consumes finite context and must be sent again as part of that turn's input. *Execution state* consists of intermediate values retained by an external runtime, such as identifiers, report rows, or traversal frontiers stored in Python variables. It can remain outside the context window, but becomes visible to the model only when generated code reads it and prints a summary or result. *Environment state* is the authoritative platform data (in this case, Ardoq's components, references, models, and analytical assets) which the agent can observe only through permitted operations. Finally, *orchestration state* records the progress of the control loop, including the current run, request count, retry status, and termination condition; it is maintained by the surrounding framework rather than used as task data by the model.

Table 2.1: State locations in a tool-using LLM agent.

Location	**Representative contents**	**Model visibility**	**Main constraint**
Conversation context	Prior calls, observations, task instructions	Directly visible on the next model turn	Finite context; repeated input and growing histories
Execution environment	Variables, intermediate records, traversal frontiers	Indirect; code must inspect or print it	Runtime lifetime, isolation, and execution failures
External environment	Authoritative Ardoq graph and analytical assets	Only through exposed operations	API semantics, permissions, and current platform data
Orchestrator	Run identity, request count, retries, termination status	Normally not task-visible	Framework policy and experiment limits

This distinction is central to the comparison. H1 carries intermediate results primarily in conversation observations, whereas H2 and H3 can retain them in execution state and compose dependent operations without returning every record to the model. All three observe the same Ardoq state under the same orchestration limits.

2.3 Ardoq and the Enterprise Architecture Domain

2.3.1 Enterprise Architecture and Ardoq's Users

Enterprise Architecture (EA) describes and governs how an organisation's business, data, applications, and technology fit together. The Open Group Architecture Framework (TOGAF) treats EA as both architectural knowledge and a method for developing and governing it [36]. EA representations support decisions about dependencies, change, cost, risk, and organisational transformation.

Ardoq is a commercial EA platform used by large organisations to document, analyse, and transform their technology landscapes. Enterprise architects are its primary users, while application owners, portfolio managers, risk and compliance teams, and transformation leaders consume or contribute to the resulting analyses. Rather than storing architecture as isolated records or static diagrams, Ardoq represents it as a schema-governed graph for querying dependencies, tracing impacts, and automating workflows [37]. Questions about change impact, incomplete ownership data, or aggregate cost and risk require traversal and computation across thousands of connected elements. This scale and con-

nectivity challenge both human analysts and LLM agents. Figure 2.3 shows an example view of the Ardoq platform interface.

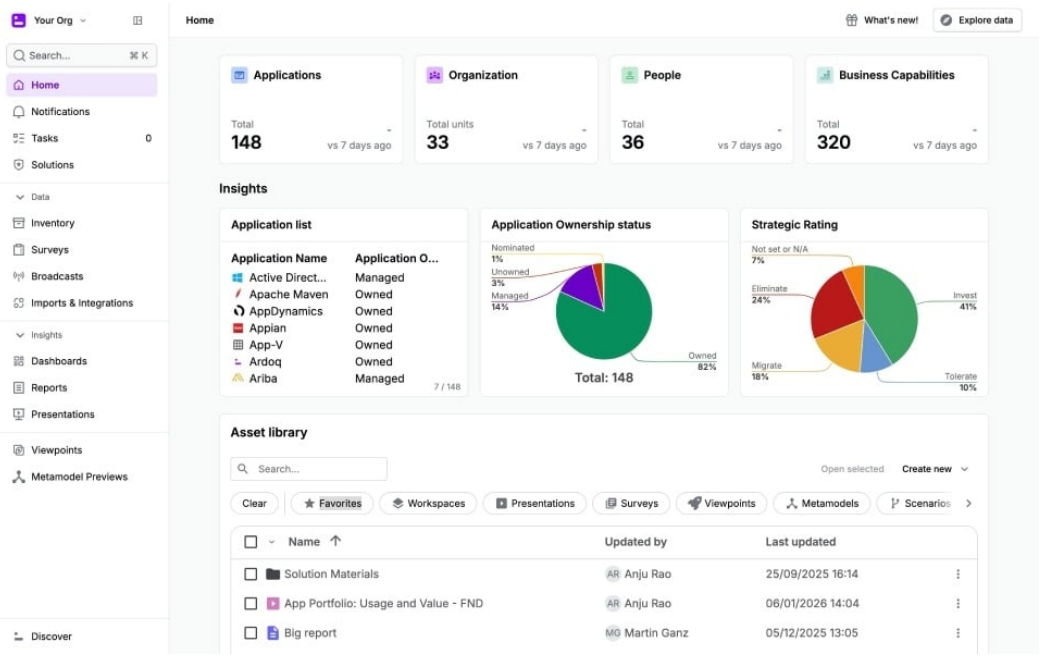


Figure 2.3: Example Ardoq platform interface, illustrating the visual environment in which users inspect architecture data and analytical views [38].

2.3.2 The Ardoq Data Model

Ardoq can be understood conceptually as a directed, typed graph built from two element types:

- **Components** are the nodes of the graph. A component represents any architectural element, such as an application, business process, technology product, data object, team, risk, or capability. Each carries system fields, including an identifier, a name, a type, and a `parent` field that places it in a tree hierarchy, together with model-defined custom fields such as owner, lifecycle phase, risk score, cost, or end-of-life date.
- **References** are the edges of the graph. A reference is a first-class, directed, typed relationship from a source component to a target component, for example an application that *is supported by* a server. References carry their own identifiers and fields and may cross workspace boundaries, linking elements that belong to different domains.

Connecting components through references yields the *instance graph*: the concrete applications, servers, risks, controls, capabilities, and other components recorded for an organisation. Questions about dependencies, change impact, or ownership are answered by traversing this graph. Two further constructs organise and govern it:

- **Workspaces** partition the graph into named groupings of related components, such as Applications, Infrastructure, or Business Capabilities. Because references frequently cross workspace boundaries, the overall graph remains connected across these domains.

- **Models** form the *metamodel* or schema layer that governs each workspace. A model defines which component types, reference types, hierarchy rules, and custom fields are valid, so the vocabulary of types and fields is defined per model rather than globally. The metamodel can itself be viewed as a schema graph: its nodes are types rather than concrete components, and its edges describe the relationship patterns permitted between them.

On top of this graph, Ardoq provides higher-level analytical assets that package common query and presentation patterns, so that users need not traverse the graph by hand. Three are central to this thesis:

- **Reports** are filterable tabular views over the graph. A report defines typed columns, server-side filters, and aggregations, and returns the rows that match its query.
- **Dashboards** are monitoring views that compose several report-backed widgets into a single summary, aggregating data for an at-a-glance portfolio or status overview.
- **Viewpoints** are reusable graph-traversal templates. A viewpoint specifies a starting component type and a path of typed references and, when executed from a seed component, returns the components, references, and workspaces it reaches.

Figure 2.4 summarises these layers: analytical assets query or traverse the instance graph, while the instance graph conforms to a metamodel that defines valid component types, reference types, and fields.

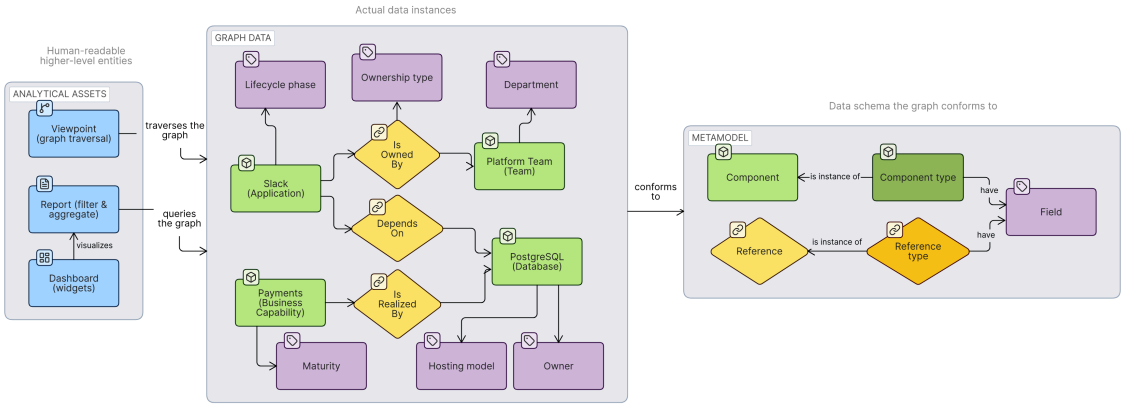


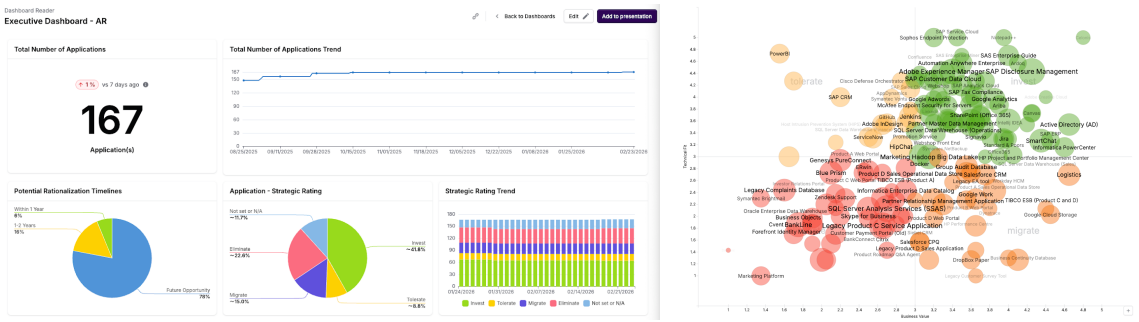
Figure 2.4: Conceptual Ardoq data model. Analytical assets query and visualise graph data; graph instances consist of components, references, and fields; and those instances conform to a metamodel defining component types, reference types, and field definitions.

As Figure 2.4 indicates, these assets provide two levels of access: low-level traversal over components and references, and higher-level queries through reports, dashboards, and viewpoints. The harness determines how the model reaches each level.

Consider a recurring lifecycle-impact question: which servers support applications scheduled to retire within the next twelve months, where no successor application is planned, and what are those servers' hosting types and lifecycle phases? Answering it requires more than finding one record. The application and server types and their relevant lifecycle fields must be identified in the governing metamodel; candidate applications must be filtered by dates and successor status; directed support relationships must be followed from the matching applications to infrastructure components, potentially across

workspace boundaries; and the returned server identifiers must be enriched with fields needed for the answer. The final result may be compact even though producing it requires several dependent retrieval and computation stages. This example recurs below because it makes the interface consequences of state placement, programmatic composition, and domain abstraction concrete.

Ardoq also packages recurring customer objectives as Solution Guides: curated, preconfigured assets for specific business outcomes [39]. These guides are curated by Ardoq's internal enterprise architecture experts. Examples include application rationalisation, application risk management, capability modelling, application hosting, and regulatory or operational compliance workflows. These solutions introduce and configure the practical artifacts through which users work with a use case: metamodels, reports, dashboards, viewpoints, custom fields, and other analytical assets. Figure 2.5 shows an example from the Application Rationalization Solution. Its executive dashboard turns portfolio data into a decision-oriented view of opportunity size and financial return, while its bubble-chart viewpoint visualises applications in the TIME framework: Tolerate, Invest, Migrate, or Eliminate, according to business value and technical fit [40].



(a) Executive dashboard for portfolio decision support.

(b) TIME bubble-chart view for application rationalisation.

Figure 2.5: Example assets from the Ardoq Application Rationalization Solution: a dashboard summarising portfolio opportunities and a diagram classifying applications by business value and technical fit [40].

The local development dataset combines these solution-oriented assets in one seed environment, approximating an Ardoq organisation in which several use cases coexist. Table 2.2 reports the scale of the local development instance used to ground the platform context in this thesis. The instance holds thousands of components and references across dozens of component and reference types, and most references connect components in different domains, so that cross-domain traversal is the norm rather than the exception.

Table 2.2: Summary statistics for the local Ardoq development dataset used to ground the platform context in this thesis. Counts are taken from the latest seed database snapshot available to Ardoq developers, pulled on June 3rd, 2026.

Concept	Count
Components	6,797
References	7,527
Component types	52
Reference types	36
Fields	546
Reports	400
Viewpoints	104
Dashboards	83

Three properties make Ardoq demanding for LLM agents. First, graph analysis is *sequentially dependent*: each traversal result can determine the next operation, so difficulty depends on path depth and branching. Second, the platform offers *layered abstractions*, from primitive graph access to reports and viewpoints, and the agent must choose the appropriate layer. Third, reliable action requires *metamodel grounding*: model-defined types, relationships, and fields must replace generic assumptions about labels such as "application" or "business capability" [17, 18, 41]. These properties distinguish Ardoq from standardised, single-domain benchmarks. At the time of the study, there was no standardised benchmark suite for evaluating LLM agents on Ardoq or on enterprise-architecture platform workflows more broadly. The task suite developed in this thesis therefore has a dual role: it is the experimental instrument used to compare the three harnesses, and it is itself a deliverable that operationalises complex but objectively verifiable Ardoq analytical work.

2.3.3 Agents in Ardoq

Ardoq already operates production LLM agents for an embedded chatbot, insight panels, and natural-language search. A dedicated AI service between the backend and the model implements the tool-calling pattern from Section 2.1. The service runs the agent loop through an agent framework called Pydantic AI, routes model inference through an internal gateway, and exposes Ardoq operations to the model as typed tool functions whose JSON parameter schemas are generated from their signatures. The model therefore sees ordinary tool definitions, while each function's implementation calls back into Ardoq's REST API to retrieve enterprise architecture data on demand. Agents have no durable memory between independent user requests. Within a request, prior messages remain available through the conversation context and the agent can query current platform data again; information used to continue the interaction must be present in or derivable from those sources. Operations execute under the requesting user's credentials, restricting them to data that user is authorised to access.

In addition to one monolithic agent, the platform runs a catalogue of task-scoped agents that share this tool surface. A general-purpose assistant has access to the full set of operations, while narrower agents are restricted to the tools relevant to a single analytical asset. For example, a report-scoped agent is limited to report operations, with dashboard,

viewpoint, and metamodel-scoped agents defined the same way. Scoping an agent to a task is therefore largely a matter of restricting its tool set. Every agent declares a structured output type that the model must populate, so responses reach the calling code as validated objects rather than free text. The same tool functions are also exposed through a Model Context Protocol (MCP) server. MCP is an open client–server protocol through which compatible AI applications can discover and use server-provided tools, resources, and prompts through a standard message interface [42]. Ardoq's server thereby lets external assistants query Ardoq data without going through the Ardoq frontend [43].

This infrastructure supplies the H1 baseline and the common API foundation for all three harnesses. The open question is whether turn-by-turn tool calling remains suitable for larger graph analyses, or whether code execution and domain-shaped abstractions work better.

Pre-Existing Agent Tools

The production agents draw on a shared library of typed tool functions, organised into domain modules that mirror Ardoq's analytical domains. Table 2.3 lists this pre-existing tool set and the purpose of each domain. Each function exposes only its domain parameters to the model, while authentication, the deployment URL, and logging are injected by the framework and never appear in the tool schema. The same functions underlie all of the task-scoped agents described above, which differ only in the subset they are permitted to call.

Table 2.3: The 17 pre-existing Ardoq agent tool surface, grouped by domain module. Each tool is a typed Python function whose JSON schema is presented to the model and whose implementation calls back into Ardoq's REST API.

Domain	Tools	Purpose
Components	`search_components` `get_component_fields`	Locate components by name or type, and retrieve field values for one or more components.

Reports	`list_reports`
	`get_report_definition`
	`get_report_data`
	`get_report_column_suggestions`
	`get_report_aggregations`
		List reports and query their tabular data with optional filtering and server-side aggregation.

Dashboards	`list_dashboards`
	`get_dashboard_definition`
	`get_dashboard_data`
		Inspect dashboard widget configurations and retrieve widget values.

Viewpoints	`list_viewpoints`
	`get_viewpoint_definition`
	`get_viewpoint_data`
		Discover and execute saved graph-traversal viewpoints from a starting component.

Metamodel	`get_metamodel` `get_metamodel_fields`	Inspect component types, reference types, and their field definitions.
Traversal	`traverse_one_hop`	Perform a one-hop graph traversal when no viewpoint applies.
Entity resolution	`resolve_entity_by_name`	Resolve a human-readable name to a component, report, viewpoint, workspace, or metamodel type.

The catalogue in Table 2.3 contains 17 pre-existing production tools. For the experiment, this surface was expanded to 20 low-level analytical operations by adding broader component-query capabilities and the metamodel operations needed by the benchmark.

The final experimental catalogue is documented in Chapter 4; H1 presents those operations as JSON tool schemas, while H2 exposes the same low-level coverage through a programmatic interface.

2.4 Compositional Demands of Enterprise Analysis

Output size does not determine the difficulty of an enterprise analytical task. A short answer may require a long chain of discovery, retrieval, traversal, and computation, while one request may return a large report. Table 2.4 defines five dimensions of interaction burden. They overlap and are not independent experimental variables, but serve as a useful conceptual model of typical workflows.

Table 2.4: Dimensions of compositional burden in Ardoq analytical workflows.

Dimension	Definition	Ardoq pattern	Interface burden
Dependency depth	Sequential stages whose parameters depend on earlier results	Discover a report, retrieve rows, then traverse from returned identifiers	Additional model round trips or explicit program sequencing
Fan-out	Number of entities or branches over which work must be repeated	Enrich every server, visit child units recursively, or traverse from many seeds	Iteration, batching, pagination, and N+1 requests
Intermediate-data volume	Data retained between retrieval and final answer construction	Join two report result sets although only their difference is returned	Context consumption or external execution state
Local computation	Deterministic transformation after retrieval	Filtering, joins, ranking, aggregation, date partitioning, and null handling	Need for variables, control flow, and consistent data representation
Schema discovery	Resolution of user terms to valid platform constructs	Find component types, field identifiers, relationship directions, reports, or viewpoints	Preliminary calls that constrain all subsequent operations

Dependency depth concerns parameter flow, while fan-out captures repeated work over collections or graph branches. Intermediate-data volume separates processed data from answer size. Local computation covers deterministic transformations such as grouping, ranking, aggregation, and set operations. Schema discovery precedes these operations when the agent must resolve the platform constructs that express the user's terms.

The lifecycle-impact example exhibits all five dimensions despite its small final answer. It requires schema discovery, dependent retrieval and traversal, fan-out across applications and servers, retained identifiers, and local date filtering. The dimensions suggest where harness effects may differ: H1 faces sequential and observation pressure, H2 adds programmatic iteration and computation, and H3 can encode recurring coordination patterns. These are design expectations, not findings; Chapter 6 tests whether the measured behaviour follows them.

2.5 Interaction Scale and Code-Execution Agents

Tool-calling interfaces face four scaling pressures on such workflows: operation descriptions consume context, observations enlarge the history, dependencies require model round trips, and joins or aggregations require computational infrastructure. Longer histories can increase token cost and latency and can make relevant information harder for the model to use reliably [44, 45].

In the basic tool-calling pattern, a result is serialised into the context before the model selects its next dependent action. This does not require returning an unbounded raw response: pagination, server-side filtering and aggregation, truncation, or compact serialisation can reduce observation size. Those mitigations do not remove the underlying dependency, however, and they require the interface designer or the model to decide which information can safely be omitted. The lifecycle-impact example still requires the model to carry the application or server identifiers needed by later stages, even if each individual response is compact.

One response to this issue is to let the model write code that invokes operations, retains intermediate data, performs local computation, and prints only what the model or final answer needs. For the lifecycle example, one program can filter applications, traverse to servers, enrich them, and assemble the answer without returning each intermediate collection to the conversation. This can reduce model round trips and observation volume, but the benefit is not automatic: programmatic interfaces also require API documentation in context and can incur code-generation, execution, debugging, and error-reporting overhead.

Industry systems have adopted this pattern to manage large tool catalogues and intermediate data. Anthropic reports a substantial token reduction in a multi-system workflow, while Cloudflare's Code Mode exposes an API surface through a typed programming interface [11, 12]. These are vendor reports rather than controlled studies. Independent work nevertheless shows that executable actions can improve success and reduce action steps in API-centric settings, and that programs provide variables, loops, conditionals, and multi-call composition [14, 46, 47].

Most code-execution approaches wrap an existing tool or API surface without changing its operation boundaries. This raises two questions: whether programmatic composition over low-level operations helps, and whether abstractions designed around recurring agent burdens help further. These questions distinguish the harnesses introduced next.

Note that executing model-generated code creates a security boundary involving credentials, process isolation, filesystem and network access, resource limits, session separation, and audit logging. This paper does not address these properties. Its execution service is experimental infrastructure, not a production-grade sandbox; deployment would require separate security assessment and hardening.

2.6 Three Agent Harness Architectures

In this thesis, the *agent harness* is the execution layer between a language-model reasoning core and an operational environment. It determines the available actions, their description and invocation format, state handling, observations, and failure feedback. This usage follows emerging agent-systems terminology, in which an "agent harness" denotes the infrastructure for tool mediation, context handling, and execution boundaries, distinct from both the base model and any application-level framework [22]. The harness is an architectural variable: holding the model fixed and changing the harness changes what the agent can perceive, compose, validate, and recover from. Figure 2.6 locates six responsibilities between the reasoning core and the Ardoq platform: action space, interface description and discovery, invocation format, state handling, observation format, and failure handling, which represent different design dimensions of the boundary.

The three harnesses are interface designs over the same Ardoq analytical API, positioned in the two-dimensional design space shown in Figure 2.7. The horizontal axis is *invocation style*, ranging from traditional tool calling, in which the model emits structured calls

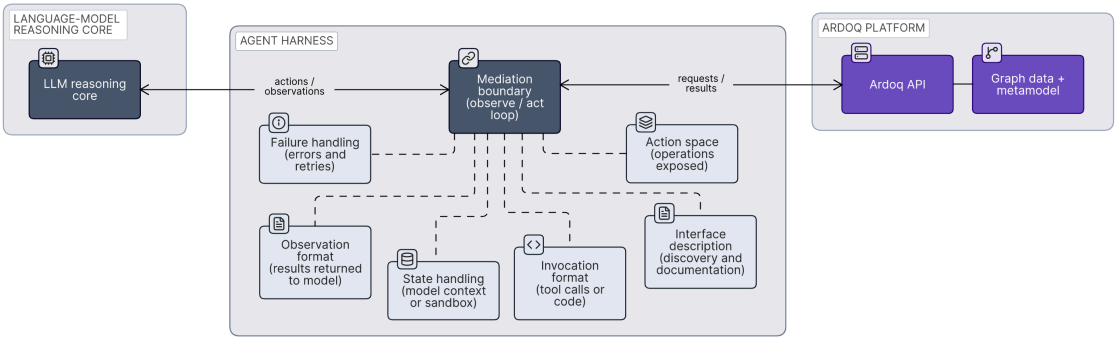


Figure 2.6: Conceptual boundary of an agent harness. The language-model reasoning core and the Ardoq platform remain outside the harness, while the harness determines the action space, interface description and discovery, invocation format, state handling, observation format, and failure handling.

whose results it can act on only after they return, to code execution, in which it writes programs that the harness runs. The vertical axis is *abstraction level*. A low-level operation corresponds closely to one platform capability and leaves sequencing, batching, identifier management, traversal, and post-processing to the caller. A higher-level domain operation packages recurring sequences, names concepts in domain vocabulary, normalises representations, or enforces invariants. Such an abstraction can reduce the number of coordination decisions required of generated code, but it can also conceal lower-level work, reduce observability, encode assumptions that fit some tasks poorly, and create maintenance obligations as the underlying platform changes.

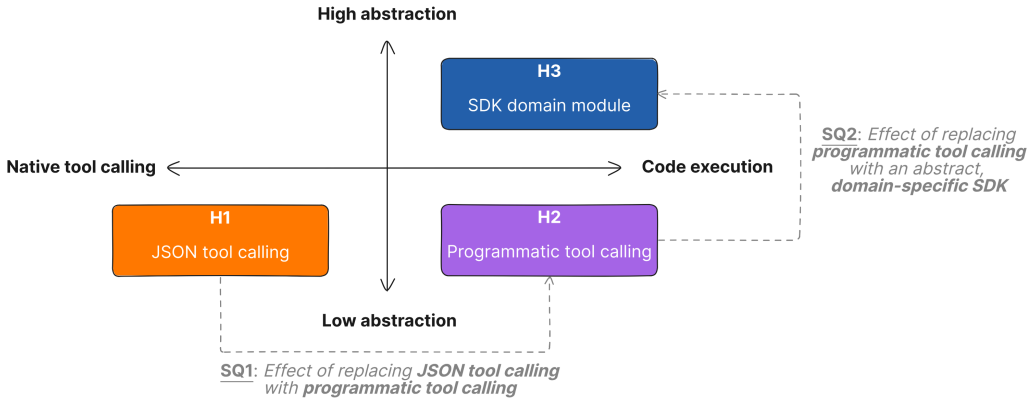


Figure 2.7: The three evaluated harnesses positioned by invocation style and abstraction level. SQ1 examines the realised H1–H2 transition and SQ2 the H2–H3 transition; the high-abstraction/traditional-tool-calling corner is outside the study.

H1 uses traditional tool calling over low-level operations; H2 exposes the same coverage through code; and H3 retains code execution but raises the abstraction to an agent-native domain module. The H1–H2 contrast addresses SQ1, and H2–H3 addresses SQ2. State placement, documentation, validation, error feedback, and observation handling vary with these realised architectures. The fourth corner of the design space (a high-level domain

interface invoked through traditional tool calls) is not evaluated. The study therefore compares two neighbouring transitions, H1 to H2 and H2 to H3. It does not test every possible combination of invocation style and abstraction level, so its conclusions only concern these two realised transitions.

For clarity, *Agent-native* is an operational design term, not a claim of superiority. An agent-native interface selects operation boundaries, return structures, and composition mechanisms around recurring agent reasoning and coordination burdens instead of inheriting them from a human- or developer-facing API. The term is consistent with emerging work on agent-first tool interfaces [15]; the specific Ardoq domain module evaluated here is one implementation of that broader design stance. In this paper, *domain-module harness* refers to this concrete agent-native H3 implementation. *SDK* (Software Development Kit) is also used to refer to the domain-module.

Table 2.5 shows how the architectures distribute work among the conversation, execution environment, and reusable interface layer. Their schemas, references, names, types, and examples are part of the agent–system interface because they affect what the model can discover and use.

Table 2.5: Conceptual comparison of the three harness architectures.

Property	**H1**	**H2**	**H3**
Model-visible action	JSON tool call	Python snippet	Python snippet
Exposed abstraction	Low-level analytical operations	Same low-level coverage	Domain-shaped module
Interface description	JSON schemas	Python function reference	SDK reference and usage patterns
Intermediate task state	Only conversation observations	Conversation and Python namespace	Conversation and Python namespace
Composition location	Across model turns	Generated program	Generated program and reusable module operations
Observation	Individual tool results	Printed output or traceback	Printed output or traceback
Characteristic risk	Sequential/context burden	Code and runtime failures	Abstraction mismatch and hidden assumptions

H1: traditional tool-calling harness. H1 exposes Ardoq operations as JSON-schema tools and returns each validated call as a conversation observation. Schema validation can catch missing fields, invalid enumerations, and type errors before they reach the platform, while explicit calls provide action-level traceability [7]. Its principal limitation for this study is composition cost. The model can batch several independent calls in a single turn (called *parallel tool calling*), but it cannot use one call's result within the same turn: any operation that depends on an earlier identifier, value, or schema must wait for a separate turn. In the lifecycle example, application discovery and filtering must return before their identifiers can parameterise traversal, and server identifiers must return before enrichment. Each dependent stage therefore adds another model round trip and carries relevant state forward in the conversation.

H2: programmatic tool calling. H2 keeps the same operations but lets the model write Python that chains calls, retains results, performs local computation, and prints selected

output from a sandbox. This is the code-execution paradigm of Section 2.5 applied to Ardoq. A single H2 program can filter the lifecycle applications, preserve their identifiers, traverse to servers, enrich the server records, and assemble an answer while retaining intermediate collections in execution state. This can reduce model round trips and keep large intermediate data out of the context. The cost is a broader failure surface and more execution infrastructure: generated code can fail at parsing, import, runtime, or semantic levels, so the harness must provide a sandbox, error feedback, and retry handling. Whether the compositional capability outweighs its documentation, execution, and repair overhead is an empirical question.

H3: agent-native domain module. H3 keeps H2's code-execution mechanism but replaces its low-level primitives with declarative queries, filtered reports, metamodel-aware operations, normalised subgraphs, and chainable traversal. For the lifecycle example, a query abstraction can combine type resolution, filtering, batching, and record hydration, while a traversal abstraction can preserve and advance the frontier across typed relationships. This is a form of *semantic compression*: recurring platform coordination is represented by fewer, domain-named operations in generated code. The compression does not remove the work; it moves it into reusable implementation and can make lower-level calls less visible. Its operation boundaries, names, and return formats encode assumptions that may fit some tasks poorly, hide platform detail, or overfit the benchmark and current dataset. The domain module also creates a larger maintenance surface than a thin wrapper. H3 therefore tests one concrete agent-native design, not the proposition that higher abstraction is universally beneficial.

3 Related Work

This chapter reviews five strands of work relevant to agent–system interfaces: tool use, code generation, state and domain abstraction, enterprise automation, and executable-agent benchmarks. Together they expose the gap addressed here: a controlled comparison of harness architectures within one schema-governed EA platform.

3.1 LLM Agents and Tool Use

Tool use lets an LLM emit commands, observe API or function results, and adapt later actions [31, 28]. The interface determines which operations the model sees, how it invokes them, and how errors return.

Early work established the main patterns for tool-augmented models. Schick et al. [28] showed that a language model could learn when to insert API calls, which API to call, and which arguments to provide from concise textual descriptions. Yao et al. [31] introduced the complementary pattern of interleaving reasoning traces with actions and observations, giving the model a way to track intermediate state and adapt to environmental feedback. This thought–act–observe loop informs all three harnesses in this thesis, each of which defines how the model acts and incorporates observations.

Later work showed that tool documentation quality materially affects reliability. Patil et al. [35] found that API-call errors often arise from hallucinated API names or incorrect argument signatures, not only from task reasoning failures. This is especially relevant in enterprise systems, where operations are numerous, documentation is uneven, and a plausible but incorrect call can modify the wrong platform object.

Controlled studies have begun to isolate tool specification format as an experimental variable. Sigdel and Baral [7] compare free-form documentation, JSON Schema, and JSON Schema with validation diagnostics while holding tool semantics and scaffold constant. Schema-based representations reduce *interface misuse*, meaning calls that violate the declared input contract, but not *semantic misuse*, meaning schema-valid calls that do not advance the task. This distinction maps onto the thesis harnesses: H1 constrains calls through JSON schemas, H2 through typed client functions, and H3 through higher-level domain operations intended to reduce invalid calls and poorly chosen valid calls.

At the system level, Wei [22] identifies tool systems, orchestration, context management, safety, and subagent architecture as recurring design dimensions across 70 public agent projects. This study uses the term *registry-oriented tools* to denote a finite catalogue of named tools whose signatures, schemas, or descriptions are registered with the agent before inference. The model selects its actions from this catalogue during execution. Protocol-based approaches such as the Model Context Protocol (MCP) are gaining adoption, but they still require choices about which operations to expose and how to describe them to the model. This supports treating harness design as an architectural variable.

The field now pays more attention to multi-tool reliability over longer trajectories, not only isolated call correctness. Xu et al. [9] frame current work around planning, trajectory construction, safety, efficiency, open-environment capability, and benchmark design, arguing that the main challenge is now system-level reliability of tool chains. Wu et al. [8] show the practical importance of interface design: their Thinker framework improves τ-bench

retail success from 68.3% to 82.6% with GPT-4o and from 49.6% to 81.9% with Llama-3.1 405B by adding state-machine-augmented tool interfaces without fine-tuning. Stateful benchmarks such as ToolSandbox further show that performance degrades when tools have state dependencies and conversational context must be maintained [45].

3.2 Code Generation for System Interaction

An alternative to structured tool calling is programmatic tool use: the model writes executable code, typically Python, against a client library rather than selecting one operation from a static schema. This changes both expressive power and failure modes. Tool calling emits fixed-schema operations whose outputs the model can only consume on a subsequent turn. Generated code can instead combine loops, conditionals, data transformations, and multiple API calls in a single program, each able to use the results of the last [46].

Liang et al. [46] introduced generated code as an action representation for robot control, showing that language models could compose low-level skill primitives into hierarchical policies. The important insight for this thesis is not robotics-specific: code gives the model variables, control flow, and reusable intermediate results, which are difficult to express through isolated tool calls. Gao et al. [47] made a similar point for mathematical reasoning. In Program-Aided Language Models, the LLM generates Python code and delegates computation to an interpreter; on GSM8K this improved accuracy from 58.8% with chain-of-thought prompting to 80.0%. The gain came from changing the output representation and execution substrate, not the model.

Once generated code enters the agent loop, the scaffold becomes part of the reasoning system. Huang et al. [48] show that language-model plans can fail when the model lacks grounded knowledge of available operations. Wang, Wang, et al. [49] find that the interaction scaffold, including error handling and feedback structure, can create larger performance gaps than model choice in multi-turn tool-use settings. Deng et al. [33] similarly show that software-engineering agents benefit from executing generated code, observing errors, and iterating. For H2 and H3, the sandbox, API documentation, exception messages, and retry policy are part of the harness being evaluated.

Code generation changes agent architecture, not only syntax. Trooskens et al. [50] propose a compiled-AI paradigm in which an LLM generates executable workflows once and later runs them deterministically without further inference, reducing token use substantially for repeated tasks. The thesis does not target such high-frequency compiled workflows, but the result illustrates the broader design space opened by code: determinism, amortised generation cost, and access to static analysis. H2 occupies a middle ground: it retains per-task generation while gaining the compositional advantages of code.

Felendler et al. [51] formalise this distinction at the protocol level. They compare *context-coupled* execution, where tool schemas, calls, and outputs are serialised into the model context at each turn, with *context-decoupled* execution, where the model generates a self-contained program that orchestrates the required operations and returns only the final result. Their CE-MCP architecture reduces token use, latency, and turn count while maintaining comparable task quality, but it also introduces new security risks because tool outputs and exception messages can influence executable code. Their results also show that the relative advantage depends on dependency structure: code handles linear chains and fan-out well, while stepwise tool use remains useful for adaptive loops and subjective filtering. This contingency matters for Ardoq tasks, which include both linear retrieval and multi-step workflows that may require revision.

Code generation stores intermediate values and composes several operations in one response, but shifts failures from schema misuse towards API, logic, and sandbox errors. H1 therefore relies on schema validation, while H2 requires execution feedback and bounded retry [7].

3.3 Agent Architectures, State, and Domain Abstraction

H3 exposes higher-level domain operations inside the Python sandbox without adding a planner, working-memory module, or reflection controller. It tests how much task structure the interface should encode and how its values should support composition.

Planning and memory research describes relevant enterprise-task failures. ReAct shows the value of connecting reasoning, action, and observation in iterative tool use [31], although subsequent work has cautioned that these gains are not automatic and can depend heavily on prompting conditions. Indeed, Verma et al. [52] argue that ReAct's performance on standard benchmarks is primarily attributable to exemplar–query similarity rather than the interleaved reasoning pattern itself. Planning surveys likewise show that task decomposition, refinement, and memory-augmented planning are active research areas, but they also underline that these mechanisms introduce additional control complexity [53]. This study doesn't involve explicit planning, and instead asks whether a domain module can reduce burdens such as preserving identifiers, passing parameters, and choosing suitable operations.

State management is the closest point of contact with the memory literature. Enterprise architecture tasks often require carrying component identifiers, workspace references, report rows, and traversal results across operations. As mentioned, in H1, that state is mediated purely through the conversation history, while H2 and H3 can persist state inside the Python runtime. The memory literature distinguishes richer mechanisms such as episodic stores, vector retrieval, and structured long-term memory [54]; those are outside this thesis's harness design. The relevant design question is how much useful intermediate state generated code can retain and how clearly domain-module return values expose it for subsequent operations.

Dependency-aware tool orchestration addresses a related problem: parameter flow across many tools. NaviAgent separates high-level task reasoning from low-level tool execution through a Tool World Navigation Model that encodes structural and behavioural dependencies between tools [55]. This thesis makes a simpler intervention. Rather than retrieving or precomputing a toolchain, H3 exposes reusable domain operations with signatures and return values shaped around common Ardoq workflows. For example, a traversal helper can return normalised node and edge structures with stable identifiers, while a report helper can bundle report discovery, filter validation, execution, and result formatting.

A developing line of work treats the agent-facing interface itself as the object of design rather than a wrapper over an existing surface. A recent systematisation frames such reusable, higher-level operations as a distinct "skills" layer sitting above atomic tools [56]. Pan [15] argues that CRUD APIs built for human interfaces are poorly suited to LLM planners and proposes an "agent-first" tool API with semantic verbs and normalised contracts, reporting higher task success than a CRUD baseline on the same platform and model. Mazrouei [16] shows that a domain-specific language designed for LLM generation can outperform general-purpose Python under an identical interpreter, especially on multi-step tasks. Chen et al. [57] hold the execution medium fixed and vary the abstraction level of agent-facing operations. They find that reusable higher-level skills cut token use substantially, while deep skill hierarchies degrade reliability. These results support the

premise behind H3, that the abstraction level of the interface is a performance lever. A related line lets the agent author its own tools in code: CREATOR and CRAFT have the model synthesise and retrieve reusable functions on demand [58, 59]. H3 differs in that its abstractions are human-designed and fixed per platform, which makes them a controlled experimental variable rather than a moving, agent-generated target. The studies above therefore either remain at the tool-calling boundary, use agent-synthesised abstractions, or evaluate on generic benchmarks. None places the comparison inside a code-execution harness over a schema-governed enterprise platform.

Planning, reflection, and long-term memory remain adjacent rather than implemented mechanisms. H2 and H3 both use generated Python and sandbox state; H3 changes the operations and values available to that code.

3.4 Enterprise AI and EA Automation

Enterprise architecture platforms are demanding for agent integration because they combine structured data, typed relationships, access control, and workflow semantics [17, 18]. In Ardoq, components are graph nodes with model-defined fields, references are directed typed edges, reports expose typed tabular results, and viewpoints encode reusable traversals. An agent must understand both the architectural meaning of components and the concrete metamodel of the workspace. Treating such a platform as a generic API surface risks failures that standard tool-use benchmarks do not capture [60].

Enterprise AI deployment also involves governance, auditability, and operational control, not only model accuracy [61]. EA platforms are relevant for harness design because they already encode governance over technology landscapes. Vendors have begun adding AI-assisted querying, reporting, and impact-analysis features [37, 62], but these products usually expose one interaction style rather than comparing alternative interfaces.

Recent enterprise-agent work addresses these constraints at several layers. Routine uses structured natural-language planning scripts with explicit steps, success criteria, and parameter-passing conventions; in one enterprise scenario it improved GPT-4o accuracy from 41.1% to 96.3% [63]. SkillForge shows that domain-contextualised skills grounded in tickets and technical documentation can outperform generic skills in cloud-support settings [64]. DADL tackles tool-library management with a declarative YAML format for serving many REST tools through one runtime, reducing tool-advertisement context cost by 142$\times$ in Code Mode [65]. Together, these studies support structure above the raw API.

Comparisons with established automation approaches reinforce the same point. Robotic Process Automation (RPA) remains faster and more reliable for stable repetitive workflows, while LLM agents require less development time and adapt more flexibly to changing interfaces [66]. POLARIS separates typed planning from governed execution through workflow contracts that specify allowed actions, data types, and transitions, reducing execution errors on back-office automation tasks [67]. These studies suggest that enterprise-agent reliability depends on the structure, governance, and recovery the harness provides.

Enterprise knowledge graph (EKG) research provides the closest conceptual parallel to Ardoq's metamodel. EKGs represent organisational components, relationships, and metadata through formal schemas, and LLM agents benefit when this structure is explicit. Rolim, Freitas, and Vidal [41] show that metadata-grounded semantic views improve reasoning and traceability over enterprise knowledge graphs. The same issue appears in Ardoq: the agent must discover component types, fields, references, and workspace boundaries before acting. The harness determines whether this graph structure reaches

the model as discrete tool schemas (H1), a typed client library (H2), or domain-level graph operations (H3).

Studies of MCP servers show that wrapping enterprise APIs into agent-facing tools is itself a design activity. Mastouri et al. [68] find that most MCP servers are REST-backed, that 92% expose tools as thin endpoint wrappers, and that the median server exposes only 19% of available REST operations. Multi-step workflows, administrative endpoints, and parameter-dense schemas are often omitted. This omission is technically consequential: exposing an entire REST API can create a large tool registry, consume context with dense schemas, increase security and governance risk, and present the agent with low-level operations whose safe use depends on state and procedural context not captured by the endpoint signature alone. Fault studies add that failures arise not only from the model but also from server configuration, tool registration, host communication, infrastructure settings, and documentation quality [69]. These findings support treating the harness as an interpretive layer between the platform and the model, not a neutral conduit.

The enterprise literature shows benefits from domain-specific planning, skills, tool libraries, and metadata grounding, but often varies several design choices at once, and does not provide a controlled within-platform harness comparison.

3.5 Benchmark Design for Agent Evaluation

Agent evaluation has moved from static question answering toward executable environments where agents must interact with systems to complete tasks. This shift matters because performance depends on action space, environment state, task structure, and verification method as well as model capability. Benchmarks such as API-Bank, AgentBench, and WebArena show that tool-use performance is sensitive to environment design and available-tool composition [70, 71, 34].

Pysklo, Zhuravel, and Watson [20] characterise benchmark design through five axes: interaction model, environment, evaluation signal, observability, and task horizon. These choices determine what conclusions a benchmark can support. Trace matching gives detailed diagnostics but penalises valid alternative strategies; outcome or state verification better captures completion, but says less about the path taken. In contrast, this thesis adopts outcome-oriented verification because Ardoq tasks can often be completed through more than one valid sequence of operations. This requires tasks with answers that can be reliably derived from platform state, which excludes ambiguous tasks and open-ended subjective EA judgements.

Stateful evaluation exposes failures that stateless benchmarks miss. Lu et al. [45] argue that real tool use involves mutable world state, accumulated conversational context, and feedback loops; ToolSandbox evaluates intermediate and final milestones over arbitrary trajectories. Their results show that even strong models struggle with state dependencies, argument canonicalisation, and missing information. Jang et al. [72] similarly show that dispersed parameter information across dialogue turns significantly degrades tool-use performance. These findings matter for Ardoq because agents must preserve intermediate query results, component identifiers, and workspace context across multi-step workflows.

Trajectory-aware benchmarks add diagnostics beyond final success. ASTRA-bench decomposes difficulty into referential, functional, and informational complexity and shows that argument generation becomes a bottleneck under high complexity [19]. TRAJECT-Bench separately measures tool selection, argument correctness, and dependency satisfaction, exposing errors such as similar-tool confusion and parameter-blind selection [73].

REAP uses real developer-agent sessions and fail-to-pass tests, giving more realistic signals for model and harness selection [74]. These approaches were deemed beyond the scope of this thesis.

Reliability also requires repeated execution and perturbation. Yao et al. [23] introduce pass$[k]$, showing that pass@k can overstate production reliability. Agents with 60% single-run success may achieve only 25% consistency across five trials. Gupta [21] extends this into $R(k, \epsilon, \lambda)$, measuring consistency, robustness to equivalent prompt variations, and tolerance of infrastructure failures. Their results show substantial degradation under modest perturbation, making single-run ideal-condition evaluation insufficient for production-facing claims.

Enterprise and API-centric benchmarks are the most direct methodological precedents. Liu et al. [71] reports large performance variation across operating-system, database, and knowledge-graph environments, showing that aggregate scores hide domain-specific weaknesses. Pysklo, Zhuravel, and Watson [20] evaluate code-executing agents on sandboxed replicas of enterprise productivity APIs using state-diff contracts that compare initial and final environment state. The thesis follows this direction through a real enterprise platform, deterministic post-execution verification, and repeated trials per harness-task combination.

Existing benchmarks are valuable, but they are not designed to compare realised harness architectures within a single case-study enterprise architecture platform. The thesis benchmark is targeted: it compares the Traditional tool-calling (H1), the Programmatic (H2), and the Agent-native domain module (H3) harnesses in a realistic, curated EA environment.

3.6 Summary and Gap

Prior work establishes that interface representation affects reliability, code adds state and control flow, and domain abstractions can reduce or displace coordination work [7, 8, 46, 47, 57]. Enterprise platforms add schema-governed data and workflows that generic benchmarks only partly capture [17, 18]. Their evaluation requires executable environments, objective verification, repeated trials, and trajectory diagnostics [20, 45, 72, 21].

However, the specific comparison remains open. Tool-specification studies usually stay within structured calling [7]; code studies use benchmarks without EA metamodels, enterprise contexts, and governed workflows [50]; and work on agent-first interfaces either remains at the tool boundary, uses agent-synthesised abstractions, or evaluates generic settings [15, 57, 16]. Enterprise frameworks also vary interface, planning, and recovery together [63, 64]. Thus, although the agent architecture space has been mapped [22], the trade-offs of the specific designs under study have not been measured within an enterprise EA platform.

This thesis addresses the gap through a controlled comparison on the Ardoq platform. H1–H2 is centred on invocation style and its consequences for state and observations; H2–H3 is centred on abstraction level over a shared execution model. The model, platform, tasks, and verification remain fixed, but each contrast compares a complete realised harness rather than an independently manipulated scalar property. The following chapters specify the method, implementation, and results.

4 Methodology

This chapter outlines how the three harnesses were compared and how that comparison answers the research questions. It covers the experimental design, the benchmark suite and its validation, the harness configurations, the oracle-based verification, and the metrics and statistical tests used to interpret the results.

4.1 Research Design and Comparisons

The study is a case study on a single platform, Ardoq. The three harnesses are compared on a shared task suite while everything except the harness itself is held fixed: the Ardoq deployment, model configuration, prompts, answer contracts, verifiers, and operational budgets are common to all three. Only their model-visible interfaces, documentation, state placement, and error mechanics differ, as the treatment under study.

The 24 fixed *tasks* are the paired unit of analysis. A *cell* combines one task with one harness, and a *run* is one of seven stochastic attempts within that cell. The seven runs per cell measure how stable each outcome is under non-determinism; the task remains the unit of analysis. The main campaign contains 72 task–harness cells and 504 total runs, as illustrated by Figure 4.1.

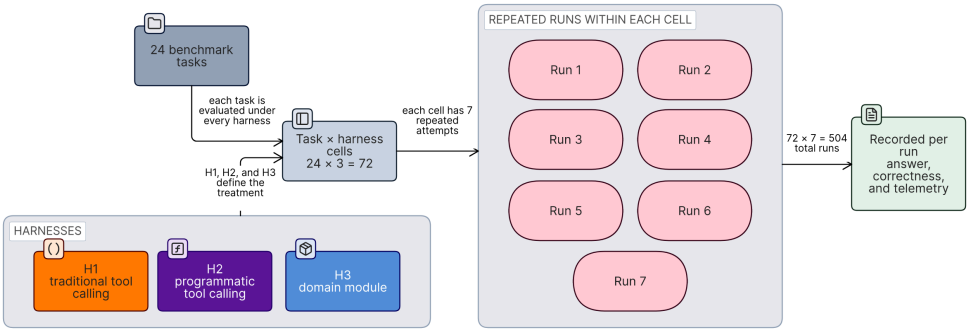


Figure 4.1: Experimental hierarchy of the evaluation campaign. A task and a harness condition combine to form a cell (72 total cells). Each cell is evaluated over seven stochastic run attempts to capture sampling variability, resulting in 504 total runs.

The two primary comparisons follow the sub-questions established in Chapter 1. SQ1 is the difference between H1 and H2 under the fixed benchmark and operational budgets, and SQ2 is the corresponding difference between H2 and H3. SQ3 examines how those differences vary across tasks and task characteristics. H1 and H3 are compared implicitly through the overall three-harness results, but they are not a planned contrast.

H1 exposes 20 low-level operations as JSON-schema tools. H2 makes the same operations available as Python functions through a single `execute_code` tool. H3 retains that execution model but replaces the low-level functions with an EA-oriented domain module. Table 4.1 summarises the three harnesses along the LLM-visible interface, invocation style, and comparison role.

Table 4.1: Controlled comparison axes for the three harnesses.

Harness	LLM-visible interface	Invocation style	Primary comparison role
H1	Traditional tool-calling harness	Structured tool calls	Baseline
H2	Tools translated to functions	Python code through `execute_code`	SQ1: staged transition from H1
H3	EA-oriented domain module	Python code through `execute_code`	SQ2: staged transition from H2

Implementation testing showed two avoidable interface costs before the final campaign was run. H1's raw tool observations could be large and repetitive, while H2's source-style API documentation and typed return objects made the code-execution interface heavier than necessary. Because these choices would also matter for a production implementation, two small design-validation studies using A/B testing were run before fixing the final H1 and H2 configurations. Each study used T01–T05 with seven runs per variant. The purpose was not to test formatting or value representation as general causal factors, but to check whether a more compact implementation could reduce interaction cost without an observed loss in task completion on the subset. H1 and H2 each tested 2 variants, *a* and *b*. H1b and H2b were selected for the main harness comparison on that basis; subsequent references to H1 and H2 denote those canonical variants unless stated otherwise. H3 has only one configuration and was not A/B tested. Section 4.4 gives the scope and limits of these studies.

Table 4.2 links each sub-question to its comparison and evidence. Exact pass is the main correctness measure; resources and latency describe operational behaviour; traces and architecture-specific events support diagnosis. Interface representation can alter completion without changing the model [7, 8], so the harness is an experimental condition rather than a neutral implementation detail.

Table 4.2: Traceability from research sub-questions to the evaluation design.

Question	Primary comparison	Headline evidence	Supporting evidence
SQ1	H1 versus H2	Exact pass, tokens, cost, all-run latency	Final context, LLM-visible actions, terminal outcomes
SQ2	H2 versus H3	Exact pass, tokens, cost, all-run latency	Generated code, tool latency, terminal outcomes
SQ3	Variation across tasks	Per-task pass differences	Topic tags, difficulty, representative traces

Failure analysis uses three evidence types: terminal outcomes, observed interaction events, and diagnostic interpretations from agent self-reflection or trace reading. A wrong answer without an execution fault does not identify its cause. Terminal outcomes are therefore primary, recorded mechanics are supporting evidence, and interpretations remain provisional.

4.2 The Ardoq Benchmark Suite

The benchmark contains 24 analytical tasks selected from 142 candidates. It supports the harness comparison and is also a thesis contribution: no standard benchmark existed for Ardoq-centred agents. Each task translates a realistic EA workflow question into a repeatable, objectively verifiable case on the same local deployment. The complete task catalogue is provided in Appendix A.3.

The 24 tasks are numbered T01–T24 in a single sequence; the identifiers are labels and do not encode analytical categories. The analysis groups tasks by concept tags and by survey-rated difficulty rather than by any naming convention: the tags describe the retrieval pattern, computation pattern, and platform resources that the harnesses must handle, while difficulty captures practitioners' perceived effort (detailed in Section 4.2.3).

4.2.1 Task Construction

The benchmark was designed to include multi-step tasks with checkable answers. Work in enterprise architecture platforms spans tasks with widely varying levels of complexity. Simple direct retrieval tasks provide limited evidence about harness architecture because they require little coordination across operations. By contrast, open-ended questions such as whether an application should be retired depend on organisational context and professional judgement, and have no unique correct answer. The selected tasks target the middle ground between these extremes, covering multi-step workflows while remaining objectively answerable from platform state. These tasks reproduce analysis used before decisions, not the decisions themselves. This design enables controlled evaluation of realistic, multi-step analysis using deterministic oracle-based verification.

Task construction involved several steps, but started from Ardoq's Solution Guides and their assets in the local seed environment. These packages configure reports, dashboards, viewpoints, workspaces, fields, types, relationships, and example data for recurring customer use cases. The resulting database is a multi-solution organisation rather than an isolated synthetic graph.

Before task candidate generation, a script queried the solutions included in the local environment and produced a summary for each one. A summary contained the solution description, its metamodel triples, and its bundled analytical assets. Candidate tasks were then created in two ways:

- **Manual exploration.** The platform and its data were browsed directly, and questions were formulated from practical experience with Ardoq and an understanding of EA workflows.
- **Agentic exploration.** Claude Code autonomously explored the environment using the solution summaries and four Python live-data probe utilities for reports, dashboards, viewpoints, and metamodel triples. It was also given the task criteria and a desired spread of task complexity, and drafted additional candidate questions, including compute functions.

The two methods produced 142 candidates covering topics such as application retirement, risk controls, capability coverage, and cost.

Each candidate used the same case fields. A case stores the task identifier, `what` field, `why` field, natural-language question, task-specific output type, tags, and verification function. The `what` field names the answer being requested, while the `why` field records the workflow reason the question matters. These fields are retained in the evaluation records and used in generated task tables, plots, and diagnostic summaries, so the benchmark preserves

both the operational question and its EA motivation. The model must return structured output rather than free text, which makes exact comparison possible while preserving the possibility of multiple valid retrieval strategies. Listing 4.1 sketches the case structure for a representative task.

```python
class EvaluationCase:
    code: str                   # canonical task identifier, e.g. "T01"
    what: str                   # what the answer represents
    why: str                    # why the question matters for EA
    question: str               # natural-language prompt given to the agent
    output_type: type           # task-specific structured-output schema
    tags: list[str]             # resource, computation, and domain tags

    def compute(self, deps) -> Answer:
        # independent oracle: re-derives the ground truth from the
        # live deployment using the same operations the agent has
        ...

    def verify(self, output, deps) -> VerifyResult:
        expected = self.compute(deps)
        actual = output.as_answer()
        return VerifyResult(passed=(actual == expected), ...)
```

Listing 4.1: Conceptual structure of an evaluation case. The fields capture the task, its metadata, and the structured-output contract; the `verify` method scores an agent's answer by re-deriving the ground truth independently at evaluation time.

Selection of the approved tasks was entirely manual. Each candidate contract and oracle was executed, inspected, and refined or excluded when the oracle failed, returned an empty or unstable answer, required presentation logic that the agents could not access, or admitted several plausible interpretations. For example, some dashboard widgets apply filters to report data, and some viewpoints impose path requirements that are only available in the user interface. This manual testing and refinement was the most tedious part of task construction: it kept hard tasks while excluding ambiguous questions or cases whose expected answer was not stable and unambiguous in the live deployment.

Screening reduced 142 candidates to a target 24 tasks. All the retained tasks were tried with H1 and H2 during development. This helped identify ambiguous data, hidden filters, and presentation behaviour that the agents could not observe. Testing with H3 was avoided as much as possible, to avoid influencing the domain-module design based on task outcomes. However, early candidates were run with H3, to help with finding implementation bugs and ensuring matching functionality with H1/H2. Early runs with H3 did not result in adding task-specific domain operations, examples, or guardrails, however. The prompts, contracts, verifiers, task suite, and harnesses were then fixed for the final experiment campaign.

Selection was based on a common capability criterion. Every approved task has a reference solution expressible through the same 20 low-level tool operations shared by H1 and H2, without direct database access, private API shortcuts, or product UI affordances. This ensured there was a guaranteed agent-accessible solution, through the tool interface. H3 reaches the same task-relevant platform data through its domain module. This establishes that a valid low-level solution path exists; it does not imply that each harness can discover and execute that path within 45 turns and 90 seconds. The fixed budgets are part of the measured operating condition.

The tasks differ in the kinds of composition they require, using the dimensions from Sec-

tion 2.4. Those five dimensions carry author-assigned ordinal scores (low, moderate, high) used only as descriptive structural proxies, not as independent experimental factors. Per-task difficulty is a separate construct: it is the practitioner rating from the validation survey (Section 4.2.3), on a one-to-five scale, rather than an author estimate.

Most tasks require at least two retrieval or analysis steps, such as discovering and reading a report, traversing from filtered components, joining retrieved sets, or aggregating values with field-specific null handling. T19 illustrates the low-coordination end: it reads two compact, named reports and performs a direct set difference, needing little schema discovery, fan-out, or intermediate state despite containing two retrievals.

The suite is read-only, which avoids state restoration but excludes conclusions about create, update, or delete operations.

4.2.2 Task Taxonomy

Each task carries overlapping tags from a controlled vocabulary. Resource tags identify the platform resources the task requires: reports, dashboards, viewpoints, components, metamodel, or graph traversal. Computation tags describe operations such as aggregation, ranking, set operations, date filtering, hierarchy, and drill-down. Domain tags record the EA use case. Resource and computation tags support harness analysis, while domain tags ensure breadth across the Ardoq Solutions catalogue. The author-assigned tags are neither balanced factors nor statistically independent categories.

Table 4.3 gives a compact overview of the 24 approved tasks. It omits full prompts, ground-truth answers, and verifier logic; those details are reported in Appendix A.3.

4.2.3 Task-suite Validation Survey

Because the benchmark is also a thesis deliverable, its tasks were reviewed for realism independently of the harness comparison. Seven respondents, all software developers at Ardoq, one of them the author's supervisor, were asked to take an enterprise-architect perspective and rate each of the 24 tasks on three five-point dimensions: *difficulty* (how hard the task would be to carry out), *usefulness* (how relevant its answer is to EA work), and *frequency* (how often such a question arises in practice). The instrument shown to respondents is reproduced in Appendix A.9. No identifying information is retained; only the aggregate role summary and per-task mean and median ratings are reported.

The respondents are platform-internal rather than independent external EA practitioners, and the sample is small, so the survey gives indicative rather than confirmatory validation (more details in Section 4.8). Its principal use in the analysis is the difficulty dimension: the per-task survey mean is the difficulty score used throughout the task-sensitivity analysis. The survey results are reported alongside the rest of the empirical findings in Chapter 6.

4.2.4 Oracle-Based Verification

Each task module recomputes the expected answer from the live Ardoq API at verification time. The verifier compares this result with the typed agent output without inspecting its reasoning or requiring a particular trace.

Exact equality is defined per output type rather than as raw object identity. Name and identifier collections are compared as unordered sets, so a correct answer is independent of output ordering unless the task explicitly requires a ranking with a stated tie-break. Numeric answers are avoided where a discrete count, set, or boolean suffices. Where an average or rate is intrinsic to the question, for example the cost averages of T08 and T24 or the compliance rate of T14, the expected and actual values are rounded to the

Table 4.3: Overview of the 24-task benchmark suite.

Task	Workflow question
T01	Which risks with a raw exposure above 15 currently have no mitigating Controls deployed against them?
T02	Which servers support applications scheduled to retire within the next 12 months with no named successor planned, and what is each server's hosting type and lifecycle phase?
T03	Which process activities have no role assigned to them, and what is the parent process for each?
T04	Which business capabilities are not realised by any application, and which organisational units are responsible for each?
T05	For each value-stream stage in the 'Business Value Stream Stages (Capabilities)' report, which business capabilities realise it, and at what maturity level?
T06	What are the five most expensive physical servers by total direct cost, and for each, its annual cost, physical location, and deployed technology products?
T07	For data assets classified Restricted or Confidential and held on a data store, which store holds each, and which applications access them?
T08	From the applications-and-attributes report, what is the average total direct app cost for each service-level category, and how many integrations does each category have?
T09	Among vulnerabilities marked as candidate, which technology products does each impact, partitioned by whether the product's review date is past, future, or absent?
T10	For each organisational unit, how many people are directly assigned to it, and how many belong to it indirectly across all child units recursively?
T11	Which business capabilities at the highest criticality level (5) are not supported by any AI capability, using the business-capabilities-supported-by-AI viewpoint?
T12	Using the application integration map viewpoint, which portfolio applications integrate with at least one SAP-supplied application, and which business capabilities do those applications realise?
T13	Which technical-debt items are marked deliberately delayed, which applications do they impact, and which organisational units consume those applications?
T14	Which technology products are mandated by approved architecture standards, which of those have a live deployment, which have none, and what is the compliance rate?
T15	Using the metamodel, is there a directed chain of references connecting an 'Application' component type to a 'Risk' component type, and if so, what is one valid path?
T16	Which deployed products carry the 'approved' flag but no longer appear in the standardised-products report, and which appear on that report without the approved flag set?
T17	Which at-risk applications also have an end-of-support technology product in their technology stack, traced via the 'Application Technology stack - TPM' viewpoint?
T18	Using the AI governance dashboard, which AI systems currently have no owner assigned, and what are their names and lifecycle phases?
T19	Which EU DORA requirements appear in the 'EU DORA requirements total' report but have not been mapped to any controls in the 'EU DORA mapped requirements' report?
T20	Which applications are owned by the organisational units Darryl Kerrigan is directly connected to?
T21	For each component-type section of the data-quality dashboard, how many components does it show, how many distinct reports does it draw from, and which report is used by the most widgets?
T22	According to the metamodel, which component type is referenced by the greatest number of distinct other component types, and what is that count?
T23	Using the retiring application landscape impact viewpoint, which business capabilities do all Adobe applications collectively realise, and which directly connected organisational units consume them?
T24	Using the 'all business capabilities' report, what is the average maturity score for differentiating capabilities (market differentiation ≥ 3) versus commodity ones (< 3), ignoring null maturity?

task-specified precision and compared within a fixed tolerance. Each task fixes its own comparison rule in its verifier; the per-task rules are documented in Appendix A.3.

The oracle is independent of the agent trace, but not of the shared client code. It is separate from the agent run and uses the same function definitions that generate the H1 tools, plus ordinary Python for deterministic post-processing. It has no direct database access, unrestricted API, or shortcut unavailable to the harnesses. A computable oracle therefore establishes a valid low-level solution path; the comparison tests whether agents can discover and compose it.

Each oracle was validated during task construction by executing its compute function against the live instance and manually checking that it produced one stable, interpretable answer before approval. The dated answer snapshots in Appendix A.3 record those checks rather than experimental outcomes. At verification time, the oracle re-executes its canonical query path and never reuses or observes the agent's trace. However, it shares low-level client functions with H1 and H2.

This avoids brittle trace matching and subjective LLM-as-a-judge scoring, but it does not measure procedural quality. Traces and generated code are retained and occasionally inspected and analysed manually, because outcome-only evaluation can sometimes mask questionable successful procedures [75].

Where an output can be flattened into comparable answer elements, the verifier also records precision, recall, F1 score, extra-item count, and missing-item count. These measures diagnose over-retrieval and under-retrieval but do not replace the task-specific exact-pass rule. They are not treated as equally meaningful for scalar, Boolean, ranked, and nested structured outputs.

4.3 Architecture Configurations

All harnesses use the exact same frontier model through the same Ardoq-internal LiteLLM proxy route. Claude Sonnet 4.6 (model identifier `claude-sonnet-4-6`) was chosen, due to its balance between cost efficiency and frontier-level intelligence. The runner supplies neither temperature nor seed, leaving the provider default temperature of 1.0, to mirror the production environment. Seven repetitions per cell provide a cost-bounded view of sampling variability.

Table 4.4 records the common configuration. Prompts share a controlled skeleton, while interface-specific instructions and documentation remain part of each condition. Runs were recorded on 15 and 16 June 2026 against the same seeded deployment.

Table 4.4: Experimental configuration and campaign size.

Property	Configuration
Main comparison	24 tasks, 3 harnesses, 7 runs per cell; 504 runs
Design-validation studies	T01 to T05, 2 variants per study, 7 runs per cell; 70 runs per study
Model route	`claude-sonnet-4-6` through a common LiteLLM proxy route
Sampling	Provider default temperature 1.0; no supplied seed
Run limits	45 turns and 90 seconds; two malformed-output retries
Scheduling	Harness workers concurrent; tasks and runs sequential within each harness
Platform	One local, seeded Ardoq deployment; read-only tasks

The seed snapshot was pulled on 3 June 2026. The implementation, benchmark source, result archives, and database snapshot are internal and are not published with the thesis.

One turn is one model request, whether the model returns an answer or requests one or more actions. The 45-turn and 90-second limits were chosen during development to bound campaign time and cost. Informal development runs indicated that attempts still unresolved near these limits rarely recovered, although this observation was not recorded as a formal pilot study, but chosen based on engineering judgement. Turn-limit and timeout outcomes are therefore reported separately from wrong answers. A timeout is checked only after the current model or tool node has completed and before another model request begins; it does not interrupt an in-flight call. Consequently, elapsed time can exceed 90 seconds, but each retained tool request has a corresponding return value.

The model route used its default five-minute prompt cache. Cache reads were disabled on the first turn of each run, while cache writes remained enabled. This prevents later repetitions from receiving a pre-heated cache from earlier runs and gives each harness the same first-turn baseline. Later requests within the same run could reuse eligible prefixes established by that run.

4.3.1 Traditional Tool-Calling Harness Configuration

H1 exposes low-level Ardoq platform operations as JSON-schema tools registered with the agent framework. The catalogue contains 20 tools across component, entity-resolution, traversal, report, metamodel, viewpoint, and dashboard domains, all of which are registered for every task. The model emits one or more structured calls, the framework validates and dispatches them, and the serialised results become observations in the conversation history.

4.3.2 Programmatic Harness Configuration

H2 exposes the same underlying operation set as pre-bound async Python functions inside a Docker-hosted sandbox. The LLM-visible interface contains a single tool, `execute_code`. Its description includes generated Python documentation for the full API. Variables and imports persist across code executions within one task run, allowing the model to retain intermediate identifiers and results. A fresh sandbox session identifier is created for every run.

4.3.3 Domain-Module Harness Configuration

H3 uses the same sandbox and `execute_code` interaction model as H2, but changes the pre-imported namespace. Instead of Python functions, it exposes an EA-oriented domain

module with declarative query predicates, typed domain models, metamodel inspection, report-analysis helpers, and a chainable traversal builder. H3 therefore varies abstraction level without adding an external planner, memory store, or separate orchestration controller.

4.3.4 Prompt and Documentation Protocol

All harnesses share the same role framing, answer procedure, and instruction to return either the typed output or an explicit failure. H1 then describes its tools, while H2 and H3 describe persistent Python environments using raw-API and domain-module idioms. The shared prompt text was written once, and the three prompts appear in Appendix A.5. H1 carries 20 JSON schemas; H2 and H3 embed their complete references in `execute_code`. No condition receives task-specific hints or a preselected subset of operations: every task sees the full, task-invariant interface of its harness, and the agent must identify the appropriate resources and actions itself. These unfiltered definitions also create a fixed per-harness context cost that is measured separately.

4.4 Harness-Configuration Design-Validation Studies

Two design-validation studies, each structured as an A/B comparison, fixed how H1 serialised results and how H2 documented its API and represented values. They select conditions for the main comparison rather than test the research questions independently. Each study used the first five approved tasks, T01–T05, with seven runs per task and variant. The first five tasks were selected for convenience rather than sampled to represent the full benchmark, but they were manually checked to provide an informative configuration check. Table 4.5 summarises the variants, and Section 6.2 reports the observed outcomes. The *B* versions were selected for the main comparison, and referred to as the canonical variants.

Table 4.5: Harness variants. H1b and H2b are canonical for the main comparison.

Variant	Distinguishing choice	Role
H1a	Tool results serialised as raw structured objects, including null fields	A/B reference
H1b	Tool results serialised as compact, type-specific string formats, including pipe-delimited formats for tabular outputs	Canonical
H2a	API documented as full Python class source via `inspect.getsource`; functions return typed objects	A/B reference
H2b	API documented as inline dict schemas; functions return plain dictionaries	Canonical
H3	Single configuration; domain module documented as syntax-stubbed class definitions	Canonical

4.4.1 H1: How Should Tool Results Be Returned?

H1a serialises each tool result as the raw structured object the API returns, including null fields. This matches how Ardoq's production agents currently return tool results, so it is the incumbent baseline. H1b instead returns results through compact, type-specific string encodings: null and empty fields are omitted where they add no task-visible information, and row-oriented outputs from reports and dashboards use pipe-delimited tables. The two variants register identical tool schemas and dispatch identical calls; they differ only in how the result of each call is written back into the conversation. Because H1 re-sends

the full conversation on every turn (Section 2.5), result serialisation drives context growth directly. The study therefore reports total token usage and final context size alongside the pass rate. The question, framed against the production baseline, is whether H1b's more efficient encoding is a safe improvement over the current approach or whether dropping nulls and flattening tables removes information the model relies on.

4.4.2 H2: How Should the Code Environment Expose the API?

H2a documents the available operations by emitting their full Python class source used in the Pydantic tool definitions, through `inspect.getsource`. It has the bound sandbox functions return typed Pydantic objects, the most faithful rendering of the underlying API and the naive starting point for a code-execution interface. H2b documents the same operations as inline dictionary schemas and has the functions return plain dictionaries and ordinary Python data structures. The variants use the same low-level operations and sandbox, but two properties changed together: documentation form and returned Python value type. Their individual effects cannot be separated. The motivating hypothesis was that full class definitions add context without improving operation use, and that ordinary dictionaries are sufficient for generated code. The study records pass rate, tool-definition size, final context, tokens, and cost. The compact variants selected from these limited checks are H1b and H2b; where the suffix is immaterial, the thesis refers to them as H1 and H2.

4.5 Experimental Procedure

The runner loads tasks, constructs the cases, and starts one worker per harness. Workers share the model gateway and Ardoq deployment, while tasks and repetitions run sequentially within each worker. Tasks ran in a fixed order, with the seven repetitions for each task adjacent.

For each task and repetition, the runner creates a fresh logical sandbox session where applicable and drives the agent's model–tool loop. H2 and H3 do not start a new Docker container per run; instead, a persistent FastAPI sandbox container allocates a new session namespace keyed by a fresh identifier. Variables and imports persist within that run's namespace, but not across runs. The run terminates when the agent returns a structured result, exceeds the turn budget, reaches the timeout boundary, or encounters an unhandled error.

On normal completion, the task verifier recomputes the oracle answer and compares it with the model output. The run is classified as pass, wrong answer, explicit failure (*"giving up"*), turn-limit exhaustion, timeout, or unexpected error. A wrong answer, turn-limit result, or timeout triggers one separate reflection call with the same model and a typed failure schema. The reflection sees the task and run history, including pending tool returns at a timeout boundary, and the oracle answer. It selects from the fixed categories reproduced in Appendix A.7 and gives a short reason. All 131 eligible non-passing runs in the main campaign produced a reflection record. Reflection is agent self-report, does not change the terminal outcome, and is excluded from the run's model-usage totals. An explicit failure already contains the agent's stated category and therefore does not trigger a second call.

The wall-clock timer starts when the per-run function is entered, after case construction. The recorded total includes model gateway transit, model requests, tool execution, sandbox work for H2 and H3, orchestration, and oracle verification. It excludes Docker startup, offline task loading, and reflection.

Each completed run is stored with its configuration, messages, tool schemas, outputs,

oracle verdict, usage, latency, terminal outcome, and reflection. This permits campaign batches to be merged and audited without reconstructing the interaction from aggregate tables.

4.6 Evaluation Metrics

The analysis distinguishes headline outcomes from supporting and architecture-specific diagnostics.

Headline outcomes. Exact pass is the primary correctness measure. A run passes only when its typed output satisfies the task-specific verifier. Total model tokens, estimated cost, and recorded end-to-end latency measure the operational burden under the fixed budgets. All runs, including wrong answers and budget-bound outcomes, enter these operational summaries.

Supporting behavioural measures. These include turns, final context size, LLM-visible actions, model and tool latency components, terminal outcome counts, and applicable collection precision, recall, and F1. Final context size is the input-token count of the final model request. An LLM-visible action is one H1 tool call or one H2/H3 `execute_code` call. It is not a platform-operation count: one code action may perform several Ardoq calls.

Provider usage metadata records base input, cache reads and writes, and output tokens for every request. Total tokens include all reported input categories and output. "New input" is recorded input minus cache-read tokens, so it retains base input and cache-write tokens. Per-run cost applies the June 2026 price schedule for the model route separately to base input, cache reads, cache writes, and output. The reported cost is therefore cache-adjusted rather than a flat list-price calculation. Because first-turn cache reads are disabled, the measured cache-read tokens provide an even comparison across runs, because it avoids the reuse of a pre-warmed cache by an earlier repetition.

Architecture-specific diagnostics. H1 records malformed tool calls and framework retries. H2 and H3 record sandbox syntax, runtime, and API events plus generated code lines; a code line is a non-empty, non-comment source line. H2 also logs calls made to its low-level functions inside the sandbox. H3 composes platform work inside domain operations and does not expose equivalent inner-call telemetry. Inner-call counts are therefore not compared across all three harnesses, only H1 and H2. These diagnostics help explain a result but are not interchangeable measures of interface complexity.

4.7 Analysis Plan

The analysis begins with run counts, means, distributions, and task-level patterns. Inferential tests use the task as the unit of pairing. For each harness, the seven runs in a task are first aggregated to a task mean. The 24 paired task means then enter a two-sided Wilcoxon signed-rank test for exact pass rate, cost, total tokens, all-run wall-clock latency, model latency, tool latency, and turns. The primary comparisons are H1 versus H2 for SQ1 and H2 versus H3 for SQ2; H1 versus H3 is secondary. All-run resource and latency values enter these tests, so budget-bound outcomes remain part of the operating-cost comparison.

Zero paired differences are discarded using Wilcoxon's zero method, absolute differences receive average ranks when tied, and the implementation chooses the exact or asymptotic calculation supported by the observed ties and zeros. Results report the test statistic,

median paired difference, and rank-biserial correlation. The tests are implemented with SciPy and statsmodels in the generated analysis pipeline. With only 24 paired tasks and coarse pass rates based on seven runs, the tests support rather than replace the task-level evidence.

Exact pass is the primary metric for testing whether the harnesses differ; the tests of tokens, cost, latency, and turns provide supporting evidence. Holm's correction is applied separately for each metric. For exact pass, it adjusts the H1–H2 and H2–H3 p-values together; total token use has its own pair, and the same approach is used for each remaining metric. The correction therefore does not treat every metric and comparison as one large family.

Terminal outcomes and internal error events are reported descriptively as counts and proportions. No inferential claim is based on the agent's reflection categories. Collection precision, recall, F1, and extra- and missing-item counts are also descriptive because their interpretation depends on output structure and they are unavailable or less meaningful for some tasks. The failure analysis reports these measures on non-passing eligible runs so that successful exact matches do not inflate the diagnosis of partial answers.

SQ3 remains exploratory. Per-tag correctness and resource use are reported with task counts, recognising that tags overlap and are unbalanced. Spearman rank correlations relate the survey-derived difficulty score (per-task mean rating, 1–5) to per-task mean pass rate, cost, tokens, turns, and latency for each harness. These tests are not corrected for multiplicity and are interpreted as descriptive evidence of monotonic association, not as causal difficulty effects, due to the low volume of tasks and iterations.

Practical quantities lead the interpretation: percentage-point pass differences, additional completed runs, paired task patterns, token differences, and cost per run.

4.8 Validity and Limitations

4.8.1 Internal Validity

The staged contrasts hold the benchmark, model, platform, output contracts, verification, caching, and budgets constant, but each harness is a package of related design choices. H1 versus H2 changes action representation together with state placement, documentation, observation flow, and error handling. H2 versus H3 changes domain operations, return structures, hidden composition, and observability. Prompt and documentation differences are constitutive parts of those treatments. The study therefore estimates differences between the three implemented conditions, not pure effects of invocation style or abstraction in all possible implementations.

Harness workers also share a model gateway and Ardoq deployment. Concurrent scheduling and transient service load may affect latency unevenly, even though the same scheduling policy applies to each condition. The fixed turn and time budgets can interact differently with the architectures; this is intended when measuring bounded completion, but it does not estimate unconstrained capability. Development involved manual inspection of tasks, oracles, prompts, and traces, and all three harnesses were exercised during construction. The scored results did not change the tasks or harnesses, but the author developed the benchmark and the harnesses in the same project. Task realism, usefulness, and difficulty were rated in the validation survey, but the seven respondents are platform-internal developers, one the author's supervisor, rather than independent external EA practitioners, so this is indicative rather than independent review. Direct lookup tasks are underrepresented, and development runs informed the retained task definitions. The

resulting suite may favour compositional cases on which harness differences are easier to observe. The design-validation subset was selected by convenience, and its selected H1b/H2b runs are reused in the main totals.

4.8.2 Construct Validity

Exact pass measures agreement with a deterministic answer contract, not explanation quality or procedural appropriateness. Tokens and cost measure model interaction burden rather than total compute or intrinsic algorithmic efficiency. LLM-visible actions are not comparable platform-operation counts, generated code lines are a rough diagnostic, and agent reflections are self-reports. The difficulty score is a survey mean from seven internal respondents and is treated as a coarse continuous summary, while tags overlap and are not balanced factors. Collection metrics and retained traces add context, but they do not eliminate these gaps between measures and constructs.

4.8.3 Statistical Conclusion Validity

The study has 24 paired tasks and seven stochastic runs per cell. This supports paired task-level comparison but gives coarse per-task pass rates and limited power for small differences. Runs within a task do not provide independent evidence of task breadth. Ties and zero differences between harnesses reduce the effective sample for signed-rank tests, subgroup cells can contain only one or two tasks, and latency and token distributions are skewed. Difficulty and tag analyses are therefore exploratory, and descriptive effect sizes remain central.

4.8.4 External Validity and Scope

The comparison uses one model route, one seeded Ardoq deployment, and one implementation of each harness. The seed contains Solution Guide assets and fictional demonstration data rather than live customer records. It is structurally rich but cannot reproduce production drift, organisation-specific modelling, or every Ardoq installation. Other models and EA platforms may respond differently to schemas, code, and domain documentation. In particular, the H3 result applies to this module's names, operation boundaries, validation rules, and assumptions, not to all agent-native interfaces.

The 24 tasks cover analytical workflows. They do not test mutation, approval workflows, concurrent users, external integrations, or long-running operations. Security is also a deliberate scope exclusion. The experimental sandbox and domain module are research implementations, not claims of production hardening. A production deployment would need separate analysis of isolation, permissions, auditability, data leakage, resource limits, and safe write operations.

4.8.5 Measurement and Oracle Validity

The oracle is independent of the agent trace but shares low-level client code with H1 and H2, so a common client or interpretation error could affect both execution and verification. Manual answer checks reduce but cannot remove this risk. Total latency includes oracle verification without a separate verifier timer, and inner platform calls are not observed equivalently in H3. Latency attribution therefore relies on the model and tool components, while cross-harness conclusions use only measurements available under comparable boundaries.

5 Implementation

This chapter explains how the experimental conditions were built, focusing on what the model can discover, invoke, retain, observe, and repair.

5.1 Implementation goals and architecture

The implementation builds three harnesses that interact with one Ardoq deployment. H1 presents twenty low-level operations as individual tools. H2 exposes the same operations as asynchronous Python functions through `execute_code`. H3 retains that execution environment but replaces the functions with an agent-native domain module.

Figure 5.1 shows the runtime paths. All conditions use the same evaluation runner, model route, typed answer contract, and authentication-bound API client layer. H2 and H3 also use the same Docker-hosted executor. Sharing the executor rules out its Python version, container configuration, session policy, and startup behaviour as sources of H2-to-H3 difference. The remaining H2–H3 differences are namespace, documentation, returned structures, guard rails, and work hidden inside H3 operations.

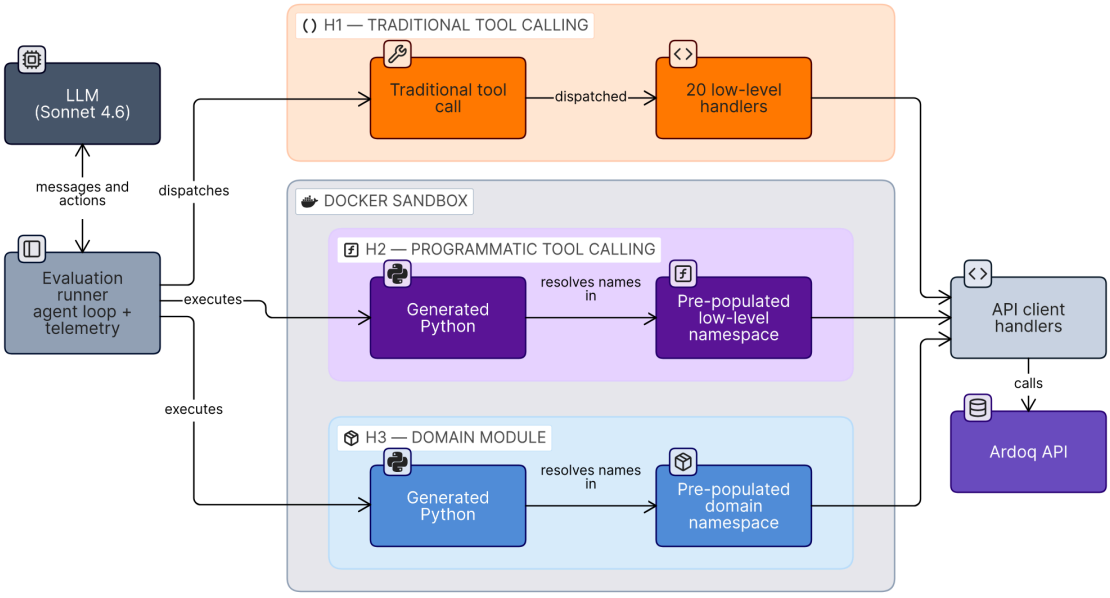


Figure 5.1: Implementation architecture. The evaluation runner supplies model-visible interface definitions and dispatches actions. H1 invokes low-level handlers directly. H2 and H3 execute generated Python against different pre-populated namespaces in the shared execution service. All three paths reach the same authentication-bound client layer and Ardoq deployment.

Table 5.1 separates shared parts from harness-specific parts using the harness responsibilities and state locations defined in Chapter 2.

Table 5.1: Common and condition-specific parts of the implementation.

Concern	Common foundation	H1	H2	H3
Action space	Ardoq client layer	20 low-level tools	Same 20 logical operations	Domain module
Description and discovery	Generated from implementation artifacts	JSON schemas	Function reference	Patterns and SDK reference
Invocation	Agent loop and typed final output	JSON arguments	Python snippet	Python snippet
State handling	Environment and orchestration state	Conversation history	Session namespace	Session namespace
Observation	Tool result enters history	Serialised return value	Printed output or traceback	Printed output or traceback
Failure handling	Framework limits and terminal failure type	Argument and tool errors	Python and API errors	Python, API, and domain-validation errors

5.2 Common experimental foundation

5.2.1 Agent and answer contract

All three agents use `pydantic-ai` and the same Claude Sonnet 4.6 deployment. The framework assembles history, registers tools, records usage, applies prompt caching, and validates the final Pydantic output.

For a given task, every harness receives the same natural-language question and must return the same task-specific success type or a common explicit failure type. A success object may contain, for example, a list of server records or a mapping from capabilities to organisational units. The explicit failure object contains a categorical cause and a short explanation. The agent returns it as its final structured output; it is not an exception. Malformed structured output instead activates the framework's output-validation retry. This differs from H2 or H3 code exception handling: a Python traceback is returned as an ordinary tool observation, after which the model may choose to submit another snippet.

The oracle is not part of the agent contract. The evaluation runner invokes it after the agent terminates, as described under oracle-based verification in Chapter 4. Consequently, the harness cannot use the oracle result while constructing its answer.

5.2.2 Shared low-level operations and their production origin

H1 and H2 share one ordered catalogue of twenty logical operations. The catalogue extends the seventeen production tools described in Table 2.3. Three operations were added for the benchmark: `get_component_aggregations` computes values and `filter_components` filters and applies field-level conditions within a type, while `list_component_types` enumerates valid types. These additions extend the existing client and backend.

Table 5.2 gives the complete operation set. The same ordered list is the source of truth for H1 registration and H2 namespace construction. H3 has different operation boundaries, although it reaches the task-relevant data through the same deployment and client API handlers.

Table 5.2: Complete low-level operation catalogue shared by H1 and H2.

Domain	Operations
Components	`search_components`, `get_component_fields`, `get_component_aggregations`, `filter_components`
Entity resolution	`resolve_entity_by_name`
Traversal	`traverse_one_hop`
Reports	`list_reports`, `get_report_definition`, `get_report_data`, `get_report_column_suggestions`, `get_report_aggregations`
Metamodel	`list_component_types`, `get_metamodel`, `get_metamodel_fields`
Viewpoints	`list_viewpoints`, `get_viewpoint_definition`, `get_viewpoint_data`
Dashboards	`list_dashboards`, `get_dashboard_definition`, `get_dashboard_data`

Authentication and communication logic is hidden from the model. The framework or namespace wrapper injects the deployment URL, API key, and organisation label. The function then calls an authentication-bound client handler, which issues the corresponding REST request. This keeps credentials outside model-generated arguments and gives H1 and H2 a common lower layer.

5.2.3 Evaluation cases, runs, and persistence

An evaluation case binds a task specification to one harness. At the start of each repetition, the runner creates a run record and installs harness-specific dependencies, including a fresh H2/H3 session identifier.

The runner records requests, messages, schemas, timing, usage, actions, generated code, and observable errors. After termination, it adds the output, oracle verdict, terminal outcome, and any eligible reflection. A serialisable snapshot omits credentials and live callables but retains the task code and configuration.

Before a campaign, the runner creates a JSON result file and then replaces it atomically after each run under an asynchronous write lock. This preserves completed work during interruption. Snapshots support later merging, and merged results retain their source provenance.

5.3 From implementation artifacts to model-visible interfaces

Each interface has three parts: an executable function, wrapper, or SDK class; a generated schema or reference; and the format returned to the model (as illustrated in Figure 5.2).

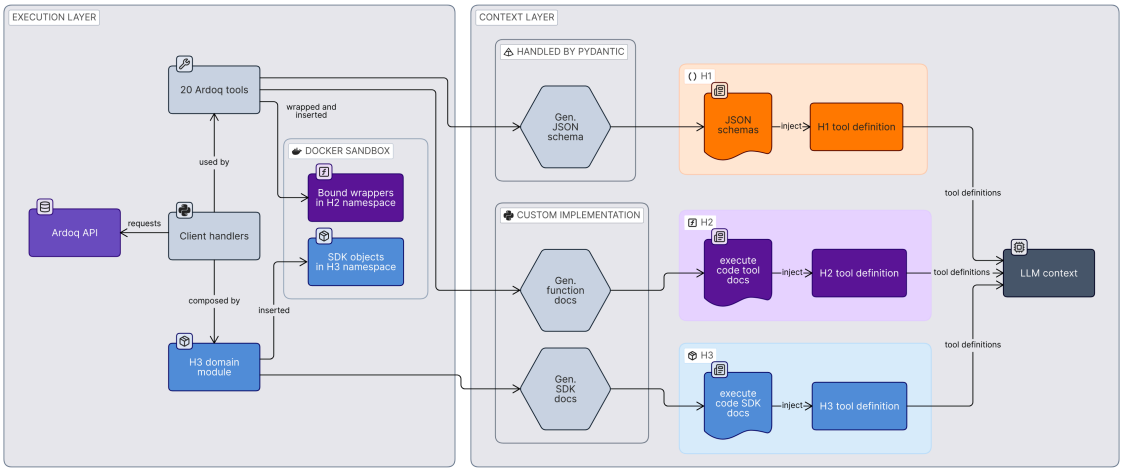


Figure 5.2: Construction of the model-visible interfaces. H1 schemas and H2 documentation originate from the shared low-level operation definitions. H2 wrappers and H3 SDK objects are separate execution artifacts placed in the Python namespace. H3 documentation is generated from the domain-module source.

For H1, `pydantic-ai` converts each low-level function's signature and docstring into a JSON tool definition. Its parameter schema omits the injected connection context. Returned Pydantic objects then pass through the selected H1 serialiser before entering conversation history.

For H2, a namespace factory wraps each low-level function as a credential-bound asynchronous callable. A documentation generator renders the callable's Python signature and referenced Pydantic models and enumerations. One `execute_code` schema carries the complete API reference.

H3 starts from a separately designed domain module over the common client handlers. Its namespace factory inserts public classes and constructors under known names. An abstract-syntax-tree generator emits class stubs with signatures and docstrings while replacing method bodies with ellipses. Curated examples are prepended to that reference. Generated code therefore executes against a pre-populated namespace; it does not import either the H2 functions or H3 SDK.

Table 5.3 reports the frozen interface sizes, measured with LiteLLM's token counter for the evaluated model route. These are sizes before any task run begins, not complete-run usage. The serialisation and documentation variants summarised in its final column are defined in Sections 5.4.2 and 5.6.2.

Table 5.3: Static model-visible interface footprint of the harnesses, including the A/B reference variants H1a and H2a. H1a and H1b are statically identical and differ only in runtime serialisation; canonical variants are H1b, H2b, and H3.

Harness	System prompt	Tool definition	Combined	Runtime values
H1a	581	9,868	10,449	Raw Pydantic serialisation, including null fields
H1b	581	9,868	10,449	Compact strings derived from Pydantic returns
H2a	420	11,857	12,277	Typed Pydantic objects in Python
H2b	423	6,573	6,996	Dictionaries and primitives in Python
H3	427	5,112	5,539	Domain dataclasses and collections

5.4 H1: Traditional tool-calling harness

5.4.1 Runtime path and state placement

H1 supplies all twenty low-level schemas with each model request. The framework validates selected calls, injects connection context, invokes the functions, and appends their serialised results to the history. The agentic loop ends with a typed success or failure, after which the runner performs oracle verification and, for eligible non-passing outcomes, a separate reflection call. A request may contain several independent calls, but any later call that needs an identifier, field name, or relationship discovered in an earlier observation requires another model request.

Retrieved identifiers and records are stored only in conversation history because H1 has no task-visible execution state. The model must recover them from observations for dependent calls. Every domain operation is therefore traceable in the history, although internal reasoning is not.

5.4.2 Observation serialisation variants

H1a and H1b expose identical tool names, descriptions, and parameter schemas. They differ only in observation encoding. H1a returns the default serialisation of the Pydantic result, including nested structure and null fields. H1b removes null fields and whitespace from JSON. Search and entity-resolution outputs use a pipe-delimited table where the result consists mainly of rows. H1a is the existing production behaviour, while H1b is the proposed compact alternative.

Compact serialisation reduces observation volume without changing operation coverage, but it can affect how the model interprets absent or nested values. It is an empirically checked encoding choice, not a cognitively equivalent one. Listing 5.1 and Listing 5.2 show the two observation shapes for the same report lookup.

The H1 variants differ only in how tool observations are written back into the conversation. The following excerpt shows the same kind of report lookup in the reference encoding and in the compact encoding used for the main comparison.

```
reports=[ReportSummary(id='821ca245745bfeedd4d9f62c',
```

```
  name='Business Value Stream Stages (Capabilities) - BVS',
  description='This is the main Business Value Stream report for BVS...')]
total_count=1

columns=[ReportColumn(key='_id',
  type=<ReportColumnType.field: 'field'>,
  label='Component ID',
  data_type=<ReportColumnDataType.text: 'Text'>,
  sort=None, is_fixed=None, original_label=None,
  description='Ardoq component ID for each row...'),
  ...]
```

Listing 5.1: Excerpt of H1a raw object-style observation for T05.

```
total=1
id | name | description
821ca245745bfeedd4d9f62c | Business Value Stream Stages (Capabilities) - BVS |
     This is the main Business Value Stream report for BVS. It shows the Value
     Stream Stages which are the children of the Value Streams

rows_count=29 showing=29
_id | name | IsRealizedBy--outgoing
7522ecbb86d3efebe536cd27 | Release / Deploy | CI/CD Pipelines, Deployment
    Automation, Release Notes, Version Control
43afe05fcc19328e39a712d4 | Onboard New Hire | Identity and Access Management,
    Onboarding Management
...
```

Listing 5.2: Excerpt of H1b compact observation for the same report lookup.

5.5 Shared Python execution service

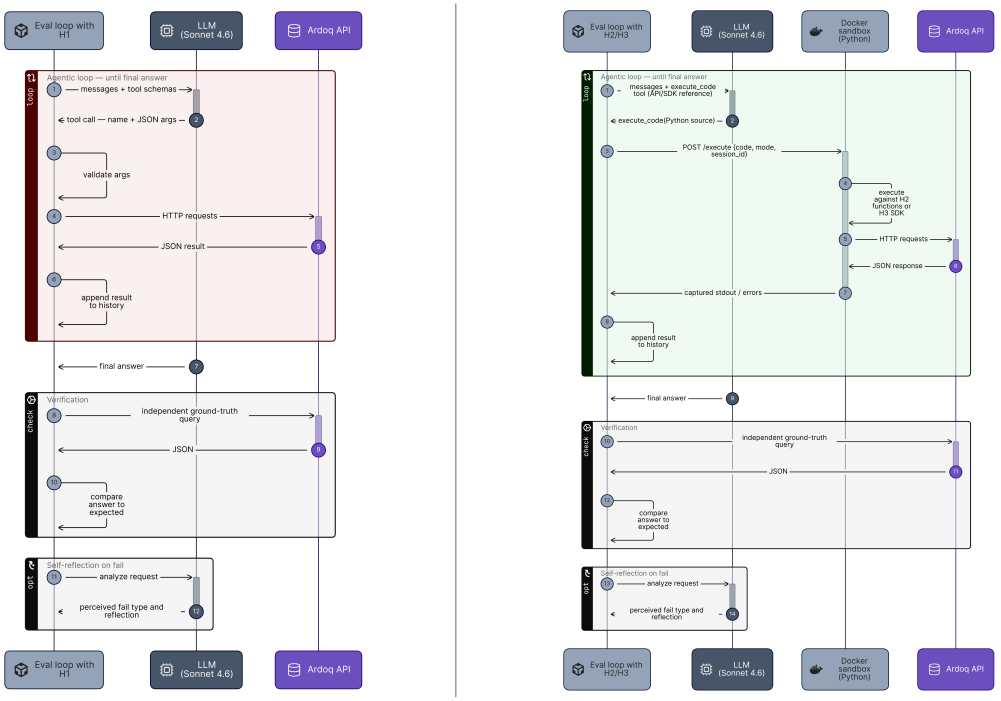
H2 and H3 send generated source, a namespace mode, and a session identifier through `execute_code` to a persistent Docker-hosted FastAPI service. The service:

1. selects or creates the namespace associated with the session;
2. compiles the source with Python's top-level-`await` flag;
3. executes it against pre-bound H2 functions or H3 SDK objects;
4. captures output streams, exceptions, and available inner-call records;
5. returns a formatted observation to the agent; and
6. retains the namespace for a later action in the same run.

Figure 5.3a shows the treatment-specific runtime loop for the traditional tool-calling harness (H1). Figure 5.3b shows the corresponding path for the programmatic harnesses (H2/H3). The evaluation loop sends generated Python to the Docker-hosted service, whose namespace mode selects H2's Ardoq functions or H3's domain module. In both panels, the agent run is followed by the same oracle verification and optional reflection procedure.

5.5.1 Session namespaces and concurrency

Each H2/H3 run receives a fresh UUID whose namespace stores variables, imports, and functions across code actions. Values created before an exception remain under normal Python semantics. Namespaces persist until the container stops, but fresh unguessable identifiers prevent later campaign runs from selecting earlier state.



(a) H1: Traditional tool-calling harness.

(b) H2/H3: programmatic code execution.

Figure 5.3: Complete evaluation sequences for the traditional tool-calling and code-execution harnesses. The coloured agentic loops show the treatment-specific runtime paths; oracle verification and eligible post-run reflection use the common evaluation procedure.

Harness campaigns may run concurrently, while cases within one harness proceed sequentially. Per-request output capture uses `ContextVar`-backed stream wrappers. This prevents concurrent asyncio tasks from writing into one another's stdout or stderr buffer. The evaluation protocol submits one code action at a time for a given agent run, so no two snippets intentionally mutate the same session namespace concurrently.

A snippet can retain thousands of identifiers or rows while printing only a count, sample, or final projection. Printed content becomes conversation state; unprinted values remain in execution state until inspected.

5.5.2 Observations, errors, and limits

The service returns captured output, a structured execution error, and available platform-call records as labelled observation sections. Normalised success and error fields support later classification without creating another model action.

The client applies a 30-second timeout to each executor request. The formatted model observation is capped at 50,000 characters after execution. This cap limits what enters conversation state but does not truncate Python objects already retained in the session namespace. The system prompt therefore tells the model to print summaries and required answer fields rather than entire catalogues. Table 5.4 collects the behaviourally relevant properties of the execution service.

Table 5.4: Property summary of the Python execution service.

Property	Implementation
Runtime	Persistent Docker service using Python 3.13
Namespace selection	Request mode selects H2a, H2b, or H3 bindings
Asynchronous code	Snippets support top-level `await`
Within-run state	Namespace dictionary persists under a run UUID
Across-run separation	Each run receives a fresh UUID
Output isolation	Per-request `ContextVar` buffers
Executor request timeout	30 seconds at the client
Model-observation cap	50,000 characters after response formatting

The service is a sandbox for this read-only experiment, not a hardened production sandbox. Production use would require threat-model-specific controls for credentials, isolation, network access, resources, session disposal, and auditing (Section 2.5).

5.6 H2: Programmatic harness

5.6.1 Wrapped-function namespace

Beyond the wrapped callables described in Section 5.3, the H2 namespace also exposes `json`. Each wrapper records its function name, arguments, elapsed time, and error flag, so one model-visible `execute_code` action can contain several low-level calls, including calls launched together with `asyncio.gather`. The runner stores these inner records separately from the single model-visible action. This distinction is necessary later: an LLM tool-call count measures submitted snippets for H2, while the inner log measures wrapped low-level operations.

5.6.2 Documentation and value variants

The code-execution tool description explains session persistence, top-level `await`, explicit printing, and the observation cap. It then includes a generated reference for all twenty functions.

H2a renders each signature and performs a breadth-first walk over referenced Pydantic models and enumerations. The Python source of the classes in the tool signatures is included with `inspect.getsource`. Those same classes are injected into the sandbox runtime for the agent to use. H2b instead expands each model into a compact dictionary schema, renders enum members as literal unions, merges model unions into nullable fields, and bounds recursive expansion at three levels. The excerpt in Listing 5.3 and Listing 5.4 shows the difference. H2a is the baseline implementation: expose the Python source and preserve the typed objects returned by the underlying functions. H2b deliberately translates both layers into simpler structures for model-generated code: the documentation becomes inline dictionary schemas, and return values are recursively converted from Pydantic models to dictionaries and from enumerations to primitive values, so generated code uses key access rather than Pydantic attribute access.

The H2 variants make the same raw Ardoq operations available through `execute_code`, but render their documentation differently. H2a exposes source-like function and model definitions; H2b keeps the same call contract but compresses the return shape into an inline dictionary schema.

```python
def search_components(regex: str) -> ComponentSearchToolOutput:
    """
    Search for components by name using a regular expression.

    {...truncated arg descriptions...}
    """
    pass

class ComponentSearchToolOutput(BaseModel):
    components: list[ComponentSearchResult] = Field(
        description='List of matching components'
    )
    total_count: int = Field(
        description='Total number of matching components'
    )

class ComponentSearchResult(BaseModel):
    id: str = Field(alias='_id')
    name: str
    type_name: str = Field(alias='typeName')
    workspace_name: str = Field(alias='workspaceName', default='')
```

Listing 5.3: Excerpt of H2a source-style documentation for `search_components`.

```
`search_components(regex: str)`
  -> `{"components": [{"id": str,
                       "name": str,
                       "type_name": str,
                       "workspace_name": str}],
       "total_count": int}`

"""Search for components by name using a regular expression."""
```

Listing 5.4: Excerpt of H2b inline-schema documentation for the same operation.

The H2 design-validation study changes two linked properties: H2a documents and returns Pydantic objects, while H2b uses inline dictionary documentation and returns dictionaries. Both expose the same logical low-level operation coverage. H2b is the canonical condition selected by the process in Section 4.4.

5.7 H3: Agent-native domain module harness

H3 implements the agent-native definition from Section 2.6; it is not a universal template. Six principles guided its public interface:

1. use enterprise-architecture vocabulary rather than endpoint-shaped names;
2. make query and traversal operations composable and declarative in Python;
3. normalise platform responses into stable domain objects;
4. move recurring discovery, batching, hydration, and graph coordination into reusable code;
5. reject bounded classes of invalid or unbounded actions with diagnostic errors; and
6. generate the reference from current signatures and docstrings.

5.7.1 Design rationale and programming model

The SDK is declarative for selecting components and describing graph traversals. A `Q` expression records type, name, workspace, or field predicates; `ComponentQuery.fetch()`

materialises the selection; and `TraversalBuilder` accumulates hops until `build()` executes them. The SDK selects the lower-level calls, batching, and hydration, while hiding implementation details from the agent.

This is bounded declarative programming, not a general query optimiser. Construction is local; `fetch()` and `build()` are explicit asynchronous execution points; and each traversal hop still requires platform work. The design changes who coordinates that work. Table 5.5 summarises these programming stages.

Table 5.5: Programming stages in the H3 SDK.

Stage	Representative expression	Execution	Result
Describe	`Q.type(...)` combined with field predicates	Local construction	Query expression
Retrieve	`await query.fetch()`	Remote, asynchronous	Hydrated domain objects
Compose	`builder.hop(...)`	Local plan construction	Traversal specification
Execute	`build()` or `traverse()`	Remote, asynchronous	Normalised `Subgraph`
Analyse	`nodes_of()`, `where()`, or `count_by()`	Local, synchronous	Derived domain value
Observe	`print(...)`	Conversation boundary	Selected model-visible output

Returned domain objects, rather than API calls, are H3's main values passed between operations. Query results directly seed traversal or viewpoint execution, and the resulting `Subgraph` supports filtering, aggregation, and further queries. Listing 5.5 shows this value flow. Generated code does not need to extract identifiers into a second request structure or join reference endpoints back to separately fetched records.

```
apps = await ComponentQuery(
    Q.type("Application") &
    Q.field("lifecycle_phase").eq("Phasing Out")
).fetch()

graph = await (
    TraversalBuilder(apps)
    .hop("Is Supported By", direction="outgoing",
         target_type="Server")
).build()

servers = graph.nodes_of("Server")
print([(s.name, s.get_field("server_hosting")) for s in servers])
```

Listing 5.5: Composition from a declarative component query to a traversal result.

The same predicate vocabulary applies to component queries, report rows, and subgraph filters. Arguments are generally flexible to accept Python callables, components, identifiers, or derived ID sets, so H3 constrains common operations without replacing task-specific Python with a fixed workflow engine.

The interface separates awaited retrieval methods such as `fetch()` and `build()` from synchronous analysis over materialised values. Frozen dataclasses and non-mutating filters make session state predictable and allow repeated analysis without another fetch, as demonstrated in Listing 5.6.

```
report = (await ReportQuery(
    Q.name("application portfolio")
).fetch())[0]

live = report.where(
    Q.field("lifecycle_phase").eq("Live")
)
costs = live.group_sum(group_by="business_unit", sum_by="annual_cost")
print(costs)
```

Listing 5.6: One remote report fetch followed by local, immutable analysis.

Normalisation reduces the code needed to reshape returned data as well as orchestration. Components expose stable identity and fields, references resolve their endpoints, reports carry metadata and rows, and subgraphs remove dangling relationships. H3 is therefore more than a renaming of H2's functions.

5.7.2 Public namespace and model-facing guidance

Table 5.6 lists the eighteen pre-populated names in the H3 namespace. Snippets require no imports, removing module-path and symbol resolution errors observed during development.

Table 5.6: Exact public namespace available to H3 generated code.

Category	Names	Role
Query construction	`Q`, `ComponentQuery`, `ReportQuery`, `ViewpointQuery`, `DashboardQuery`	Compose filters and fetch domain objects
Graph execution	`TraversalBuilder`	Validate and execute chained graph hops
Domain objects	`Component`, `Reference`, `Report`, `Viewpoint`, `Dashboard`, `Widget`, `Subgraph`	Normalised platform data and local analysis
Metamodel and values	`Metamodel`, `ComponentTypeDetail`, `MetamodelFieldDef`, `FieldSummary`	Schema inspection and summaries
Utility	`json`	Standard JSON encoding and decoding

The `execute_code` description combines runtime rules, six hand-written composition patterns, and a generated SDK reference. The doc generation uses Python's `ast` module to remove private members and method bodies while preserving public signatures, types, properties, and docstrings in ordered sections. This keeps the structural reference aligned with the source. The composition patterns remain manual and are reproduced with extended examples in Appendix A.6.

5.7.3 Moving work into the SDK

The SDK moves parts of the five burdens from Section 2.4 between generated and reusable code (Table 5.7). The main change is that retrieved objects can be reused directly in traversal and local analysis.

Table 5.7: How H3 redistributes compositional work.

Burden	H3 mechanism	Work retained by generated code
Schema discovery	`Metamodel` constrains later query and hop specifications	Select intended domain concepts and inspect ambiguity
Dependency depth	Query results directly seed `TraversalBuilder` or viewpoint traversal	Specify the desired path and order
Fan-out	Collections remain typed values while the SDK batches hydration and caps frontiers	Choose seeds and sufficiently narrow filters
Intermediate volume	`Report` and `Subgraph` retain materialised data in execution state	Decide which summaries or fields enter the conversation
Local computation	Reusable predicates, filters, grouping, and aggregation compose over domain objects	Define task-specific computation and answer projection

For example, a component query resolves types, retrieves identifiers, batches hydration, and returns objects that can form a traversal frontier. The builder retains a normalised subgraph across hops, while report and subgraph analysis runs locally over retrieved data. Table 5.8 gives representative model-visible operations alongside the coordination they hide.

Table 5.8: Examples of model-visible H3 operations and hidden coordination.

H3 operation	Hidden coordination	Interaction pattern
`ComponentQuery.fetch()`	Type resolution, identifier query, batch field hydration	Several bounded platform calls
`ReportQuery.fetch()`	Report discovery, metadata filtering, and row retrieval	Discovery plus one retrieval per match
`Metamodel.fetch_full()`	Metamodel retrieval and parallel type-detail fetches	Several platform calls
`TraversalBuilder.build()`	Hop validation, frontier management, and graph normalisation	At least one call per hop
`Subgraph.aggregate()`	Numeric aggregation over retained domain objects	Local computation only

5.7.4 Guard rails and diagnostic feedback

`Q.name()`, `Q.type()`, and `Q.workspace()` reject regular expressions without literal alphabetic characters, preventing wildcard-only catalogue retrieval.

`TraversalBuilder` validates sufficiently specified hops against the metamodel and reports valid relationships after a mismatch. It checks schema compatibility, not whether the path answers the task. Frontiers above 200 components similarly raise a diagnostic that requests narrower filters. Listing 5.7 shows a representative traversal configuration and subsequent local analysis.

```
servers = await ComponentQuery(Q.type("Server")).fetch()

```

```
graph = await (
    TraversalBuilder(servers)
    .hop("Is Located At", direction="outgoing",
         target_type="Location")
    .hop("Deploys To", direction="incoming",
         source_type="Technology Product")
).build()

print(graph.summary())
```

Listing 5.7: Representative use of H3 traversal and local subgraph analysis.

5.8 Example workflow across the three harnesses

T02 asks for servers supporting applications scheduled to retire within twelve months with no named successor, together with each server's hosting type and lifecycle phase. It exercises schema discovery, date filtering, successor checking, graph traversal, and enrichment. Table 5.9 summarises the first run of each harness from the main campaign. The table omits returned rows but preserves the operations and state transitions.

Table 5.9: T02 as carried out by each harness in representative main-campaign traces.

Harness	Model-visible progression	Intermediate state
H1	Discover reports, types, and metamodel; retrieve the one-year report; traverse from returned application IDs; fetch server fields	IDs and records re-enter later JSON calls from conversation observations
H2	Inspect types, fields, and relationships; retrieve and filter applications; store application IDs; check successors; traverse to servers; enrich records	Python lists, dictionaries, and maps persist in the session namespace
H3	Fetch one metamodel object; query applications into `Component` objects; pass those objects directly into successor and server traversals; inspect hydrated server objects	Domain objects, dates, and subgraphs persist in the session namespace

The stages are comparable, but their boundaries differ. H1 carries parameters between model turns; H2 keeps them in Python while retaining low-level operations; H3 carries state in compatible domain objects while the module handles recurring retrieval mechanics. Chapter 6 tests the consequences.

5.9 Telemetry, validation, and measurement boundaries

The evaluation records telemetry at six boundaries: provider responses, runner timers, model-visible message history, executor logs, agent termination, and oracle verification. These boundaries matter because the three harnesses expose different amounts of internal work. A count taken at the conversation boundary is available for all harnesses, but it does not measure the same kind of work as an HTTP-call log inside H2. The analysis therefore treats run-level usage, latency, outcome, and model-visible actions as cross-harness measures, while inner-call records and reflection records are diagnostic evidence. Table 5.10 records the capture points; Section 4.6 defines their aggregation.

Table 5.10: Telemetry capture points and cross-harness comparability.

Signal	Capture point	Availability	Comparability
Tokens and cache accounting	Provider usage metadata on each model response	H1, H2, H3	Common
Model-response and action timing	Runner timers around model requests and model-visible tool actions	H1, H2, H3	Common boundary, with different work hidden behind each action
Model-visible actions	Structured tool-call parts in message history	H1, H2, H3	Common count; one action may represent different platform work
Tool-output size returned to the conversation	Tool-return message parts	H1, H2, H3	Common conversation boundary
Malformed calls and retries	Framework validation records	H1, H2, H3	Common mechanism; incidence depends on interface design
Generated source and line count	`execute_code` arguments	H2, H3	Common code boundary
Syntax, runtime, and API errors	Executor responses and sandbox call records	H2, H3	Common broad classes
H2 internal wrapper calls	Instrumented namespace wrappers	H2 only	Diagnostic, not cross-harness
Final structured answer or failure state	Agent termination object	H1, H2, H3	Common
Oracle verdict and terminal outcome	Evaluation runner after agent termination	H1, H2, H3	Common
Post-run reflection diagnosis	Separate reflection call for eligible non-passing outcomes	H1, H2, H3	Diagnostic self-report only

The latency split follows the same boundary. Provider timestamps separate model time from model-visible tool time. H2 and H3 executor work is included in tool time because both enter the sandbox through `execute_code`. H2 then adds wrapper-level timings for each low-level function called inside the snippet. H3 does not expose an equivalent inner log: SDK methods may combine validation, hydration, batching, traversal, and several REST requests under one domain operation. A flat internal call count would not have the same meaning as the H2 wrapper log, so it is not used as a cross-harness metric.

End-to-end run latency starts when the runner enters a case and ends after oracle verification. It includes model requests, model-visible tool execution, sandbox work, orchestration overhead, and verifier execution. It excludes Docker startup, offline task loading, post-run reflection, and figure generation.

Validation also occurs at several boundaries. Before a run, a health endpoint checks that the backend is reachable. During a run, framework schemas, executor exceptions, API errors, and H3 domain checks return structured feedback to the agent. After termination, the task oracle computes the expected answer and compares it with the final typed output. These checks support benchmark validity, not general software assurance.

This measurement contract supports the two comparisons from Methodology: tool calling versus code over the same low-level operations, and low-level functions versus an agent-native domain module in the same executor. Chapter 6 evaluates the resulting correctness, resource use, latency, and failure patterns.

6 Evaluation

This chapter compares the three harnesses, locates their task-level differences, and interprets the results for agent–system interface design. Measurements are kept separate from proposed mechanisms.

6.1 Overview

The main campaign contains seven runs in each of 72 task–harness cells: 168 runs per harness and 504 total. H1 and H2 denote the canonical *H1b* and *H2b* variants selected in Section 6.2.

SQ1 compares tool calling with programmatic composition over the same low-level coverage; SQ2 compares low-level Python functions with the agent-native domain module; and SQ3 locates their task sensitivity. Exact pass is the primary outcome. Resources, terminal outcomes, and telemetry support interpretation.

H2 passes far more often than H1 and uses fewer model resources, especially for graph, set, cross-source, and state-heavy workflows. H3 reduces token traffic and final context relative to H2 while giving up only one exact answer; per-run model cost also falls from $0.11 to $0.10, while mean latency changes only descriptively from 34.8s to 30.8s. The H2–H3 step is therefore mainly a gain in interaction efficiency, with little change in correctness.

6.1.1 Task-suite Validation

Before interpreting harness behaviour, the suite itself was checked for realism through the validation survey described in Section 4.2.3. Table 6.1 pools the seven respondents' ratings across the 24 tasks. Usefulness is the strongest signal, at a mean of 3.70 out of 5 (median 4); most tasks have a median usefulness of 4 or higher, so the questions map onto answers the respondents regard as relevant to EA work. Frequency is lower at 2.57 (median 2), consistent with periodic analytical or deep-dive questions rather than daily lookups. Rated difficulty averages 2.99 and ranges from 1.71 to 4.00 across tasks (per-task ratings in Appendix A.3), concentrated in the middle of the scale; this per-task survey mean is the difficulty score used in the task-sensitivity analysis below. Because the respondents are platform-internal developers rather than external practitioners, these ratings validate task realism only indicatively.

Table 6.1: Task-suite validation survey results: averages per rating dimension, pooled over all rated tasks ($n = 7$ developers acting as EA practitioners, 1–5 scale). Each row pools 168 task–respondent ratings: one rating for each included task from each respondent for that dimension.

Metric	Mean	Median	Std	Min	Max
Difficulty	2.99	3.00	1.11	1	5
Usefulness	3.70	4.00	0.92	2	5
Frequency	2.57	2.00	0.84	1	4

6.2 Harness-Configuration Design-Validation Results

This section documents why H1b and H2b were used as the canonical H1 and H2 conditions in the main comparison. These results explain the configuration choice; the primary research-question evidence begins in Section 6.3.

The two configuration checks used A/B comparisons on T01–T05. Both compact encodings (H1b and H2b) reduced token use without an observed correctness loss on the subset. For H1, compact observation encoding used type-specific string formats that omitted null and empty fields where possible, with pipe-delimited representations for tabular outputs. H1b passed 20/35 runs, compared with 18/35 for the raw-object H1a variant. Mean total tokens fell from 334.5k to 230.3k, a 31% reduction; the final context window fell further, from 44.3k to 23.9k (46%); and estimated cost per run dropped from $0.28 to $0.18 (36%). This supports H1b as the lower-cost encoding with no observed exact-pass loss on the selection subset.

For H2, the compact variant bundled two changes: inline dictionary schemas replaced full source-style documentation, and plain dictionaries replaced typed return objects. H2b passed 33/35 runs, compared with 32/35 for H2a. Mean total tokens fell from 192.5k to 116.9k, a 39% reduction, with the final context window down 30% and estimated cost down from $0.17 to $0.12 per run (29%).

Table 6.2 reports both families side by side. The selected H1b and H2b observations for T01–T05 are also included in the 24-task main totals. The table documents the configuration choice, and from this point forward, *H1* is equivalent to *H1b*, and *H2* is equivalent to *H2b*.

Table 6.2: Design-validation A/B studies (5 tasks, 35 runs per variant). Each family compares its compact variant (b) against the verbose baseline (a); the b variants were selected as the canonical H1 and H2 used in the main comparison. Token and context values are mean $\pm$ std.

Metric	H1a	H1b	H2a	H2b
Pass rate	51.43%	**57.14%**	91.43%	**94.29%**
Total tokens	$334.5k \pm 420.3k$	$\mathbf{230.3k \pm 232.7k}$	$192.5k \pm 93.1k$	$\mathbf{116.9k \pm 40.3k}$
Final context	$44.3k \pm 27.3k$	$\mathbf{23.9k \pm 7.8k}$	$21.5k \pm 5.1k$	$\mathbf{15.1k \pm 2.5k}$
Cost per run (USD)	0.28 ± 0.23	$\mathbf{0.18 \pm 0.11}$	0.17 ± 0.06	$\mathbf{0.12 \pm 0.03}$

6.3 Main Harness Comparison

Table 6.3 shows the overall performance summary, per harness. H2 raises exact pass from 66/168 runs (39.29%) for H1 to 154/168 (91.67%), an increase of 88 runs and 52.38 percentage points (pp). Mean tokens fall from 329.7k to 113.2k, turns from 12.9 to 8.7, and estimated cost from $0.22 to $0.11 per run. The correctness gain comes with fewer tokens and turns. Table 6.4 shows the terminal outcomes behind this difference. H1 combines 50 wrong answers with 52 budget-bound outcomes. H2 has equal counts of wrong answers and timeouts, while H3 fails mainly through wrong answers; neither code harness reaches the turn limit.

H3 returns 153 exact answers (91.07%), one fewer than H2. It has lower descriptive means for tokens (91.8k), turns (7.9), and latency (30.8 s), with mean cost at $0.10 per run compared with H2's $0.11. The paired analysis tests whether these changes are consistent across tasks.

Table 6.3: Overall performance summary (24 tasks, 168 runs per harness). Numeric values are mean ± one standard deviation.

Metric	H1	H2	H3
Pass Rate	39.29%	**91.67%**	91.07%
Total Tokens	$329.7k \pm 389.3k$	$113.2k \pm 73.8k$	$\mathbf{91.8k \pm 46.6k}$
Latency (s)	40.5 ± 34.5	34.8 ± 21.0	$\mathbf{30.8 \pm 15.4}$
Turns	12.9 ± 10.8	8.7 ± 4.2	$\mathbf{7.9 \pm 3.1}$
Cost (USD)	0.22 ± 0.19	0.11 ± 0.05	$\mathbf{0.10 \pm 0.03}$

Table 6.4: Terminal run-outcome counts per harness.

Metric	H1	H2	H3
Correct	66	**154**	153
Wrong answer	50	**7**	14
Turn limit reached	9	0	0
Timeout	43	7	**1**

Figure 6.1 shows that code execution reduces both wrong answers and budget-bound failures relative to H1. H3 times out less often than H2 but returns more wrong answers.

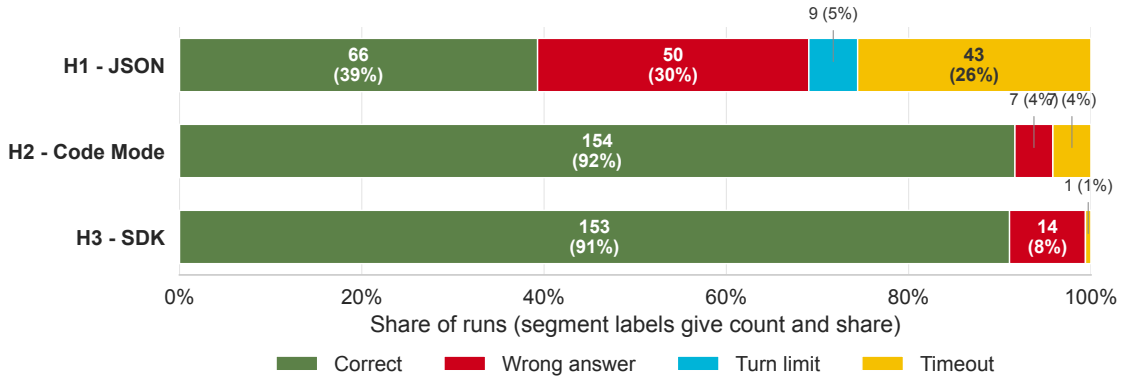


Figure 6.1: Terminal outcomes by harness. Bars are normalised to 100%; each segment label gives the run count with its share of all runs in parentheses. A correct outcome means that the strict verifier accepted the final answer.

Table 6.5 reports the two primary comparisons, H1–H2 and H2–H3, after first aggregating the seven runs to one mean per task. It includes the headline correctness, token, context-window, latency, and turn outcomes; paired cost tests are reported with the full statistical diagnostics in Table A.1. H2 exceeds H1 on every task with a non-zero pass-rate difference, giving rank-biserial $r_{\text{rb}} = 1.000$ and Holm-adjusted $p_{\text{adj}} < 0.001$. Tasks with identical pass rates contribute zero differences and are discarded by the signed-rank test; the effect therefore means that every *differing* task favours H2, not that H2 improves all 24 tasks.

Table 6.5: Primary staged contrasts on per-task means (seven runs per task). Positive differences indicate that the later harness is higher. r_{rb} is rank-biserial correlation; Holm correction is applied across the two planned contrasts within each outcome.

Outcome	Contrast	Mean Δ	Median task Δ	r_{rb}	p_{adj}
Exact pass rate	H1→H2	$+52.4$pp	$+64.3$pp	$+1.000$	< 0.001
	H2→H3	-0.6pp	$+0.0$pp	-0.194	0.608
Total tokens	H1→H2	-216.5k	-100.4k	-0.980	< 0.001
	H2→H3	-21.4k	-8.7k	-0.513	0.027
Final context window	H1→H2	-11.8k	-7.8k	-1.000	< 0.001
	H2→H3	-1.5k	-1.3k	-0.713	0.001
Wall-clock latency	H1→H2	-5.7s	$+1.7$s	-0.033	0.900
	H2→H3	-4.0s	-1.1s	-0.320	0.356
Turns	H1→H2	-4.14	-1.50	-0.732	0.007
	H2→H3	-0.82	-0.07	-0.268	0.281

The H2–H3 exact-pass difference is small and inconsistent: the median task difference is zero, $r_{\text{rb}} = -0.194$, and adjusted $p_{\text{adj}} = 0.608$. H3 uses fewer total tokens ($r_{\text{rb}} = -0.513$, adjusted $p_{\text{adj}} = 0.027$), a result qualified by the decomposition in Section 6.5.

Campaign latency falls by 5.7 s from H1 to H2 and 4.0 s from H2 to H3, but neither paired change is detected (adjusted $p_{\text{adj}} = 0.900$ and 0.356). Turns fall consistently only from H1 to H2 ($p_{\text{adj}} = 0.007$; H2–H3 $p_{\text{adj}} = 0.281$).

Cost per attempted run follows the same pattern as token use. As shown in Table 6.3, H1 costs $0.22 per run on average, compared with $0.11 for H2 and $0.10 for H3. This is a descriptive campaign average rather than an uncertainty-bearing estimate of future deployment cost, but it shows that H1 combines lower completion with higher resource use per attempt.

6.4 Where the Differences Occur

Figure 6.2 provides the main task-level overview, alongside the two Δ columns showing H2 minus H1 and H3 minus H2.

Four tasks are solved in every run by all three harnesses: T03, T08, T18, and T24. A second group shows mainly an efficiency difference. On T02, for example, H1 succeeds in 6/7 runs and H2 and H3 in 7/7, but H1 uses 314k mean tokens against 146k for H2 and 137k for H3. For compact or report-led tasks, H1 can still pass, but at higher token cost.

On T07, T13, T14, T15, T19, and T22, H1 is 0/7 while both code harnesses are 7/7. These tasks involve graph traversal, set construction, metamodel search, or cross-source composition. T11 and T17 are similar. H1 usually invokes the platform successfully but does not complete the composition within its budgets.

The T13 traces make the contrast concrete. The task asks for org-unit/application pairs where a unit consumes an application impacted by a delayed debt item. Figure 6.3 compares the three complete model-turn traces; the full code excerpts and verifier output are reproduced in Appendix A.8. H1 returns a wrong answer after 13 turns and 249k tokens:

	H1	Δ H2−H1	H2	Δ H3−H2	H3
T01 high-exposure risks without controls	0/7	+6	6/7	+1	7/7
T02 retiring-application servers	6/7	+1	7/7	+0	7/7
T03 activities without assigned roles	7/7	+0	7/7	+0	7/7
T04 unrealised capabilities and owners	5/7	+1	6/7	+0	6/7
T05 value-stream capability maturity	2/7	+5	7/7	+0	7/7
T06 highest-cost physical servers	5/7	+2	7/7	+0	7/7
T07 confidential-data access paths	0/7	+7	7/7	+0	7/7
T08 application cost by service level	7/7	+0	7/7	+0	7/7
T09 vulnerabilities by review date	6/7	+1	7/7	+0	7/7
T10 recursive organisation headcount	0/7	+4	4/7	+1	5/7
T11 critical capabilities without AI support	0/7	+7	7/7	-1	6/7
T12 supplier integration-map dependencies	0/7	+0	0/7	+1	1/7
T13 delayed-debt consumer traversal	0/7	+7	7/7	+0	7/7
T14 standards deployment gaps	0/7	+7	7/7	+0	7/7
T15 application-to-risk metamodel path	0/7	+7	7/7	+0	7/7
T16 approved-versus-standardised products	1/7	+6	7/7	-2	5/7
T17 at-risk application technology stacks	0/7	+7	7/7	-1	6/7
T18 unowned AI systems	7/7	+0	7/7	+0	7/7
T19 unmapped DORA requirements	0/7	+7	7/7	+0	7/7
T20 person-to-owned-applications traversal	6/7	+1	7/7	-1	6/7
T21 data-quality dashboard profile	6/7	+0	6/7	+0	6/7
T22 metamodel in-degree	0/7	+7	7/7	+0	7/7
T23 retiring-application landscape	1/7	+5	6/7	+1	7/7
T24 capability maturity comparison	7/7	+0	7/7	+0	7/7

Figure 6.2: Task-level exact correctness and paired harness contrasts. Count columns show correct runs out of seven; each Δ column sits between the two harnesses it compares and gives the signed difference in correct-run counts.

it resolves the two delayed debt items early, then issues the required application and consuming-unit hops as six separate `traverse_one_hop` calls across turns 6–10, each result re-entering the conversation as a fresh observation. Without a place to hold and deduplicate the intermediate set, it submits nine pairs (recall 1.0, precision 0.56), four of them incorrect. H2 and H3 pass in eight turns and 94k/83k tokens, respectively. They query the debt items, traverse both hops, and deduplicate the pairs inside execution state, returning exactly the five expected pairs.

The trace supports the state-placement explanation, though the bundled H1–H2 contrast does not isolate it from documentation, observation flow, or error handling.

6.4.1 Sensitivity of the H2–H3 Transition

H3's one fewer pass comes from a few task-level shifts rather than a broad regression. It trails on T11, T16, T17, and T20; leads on T01, T10, T12, and T23; and ties on the other 16 tasks. The paired test detects no systematic effect (Table 6.5). The moved tasks instead show where the abstraction fits or misfits a workflow. Trace references use *Txx#y* for task *Txx*, run *y*; for example, H3 T16#4 is the fourth H3 run of T16.

T16 provides the clearest observed regression, by asking the agent to compare approved deployed technology products with the active standardised-products report and return the two mismatch lists in each direction. In H3 T16#4 and T16#5, the generated query retrieves all 152 technology products whose `approved` field is true and subtracts the standardised report. It does not apply the task's *deployed* constraint. T16#4 therefore returns a large false-positive set after only six turns and 61.7k tokens. In H2 T16#4, the program explicitly reads the "Total deployed Tech Products" report, joins it by identifier with the standardised report, and passes after 11 turns and 135.3k tokens. The case shows the H3 trade-off: a concise domain query can make an under-specified selection easy and

H1

13 turns; 249k tokens

1: list_reports()
list_viewpoints()
list_component_types()
→ reports, viewpoints, component types
2: get_report_definition()
get_metamodel_fields()
→ report schema, Debt Item fields
3: get_report_data()
filter_components()
→ two delayed debt items
4: get_viewpoint_data()
→ 42 components, 26 references
5: get_report_data()
→ 32 rows, requested columns empty
6: get_report_definition()
→ report query, column names
7: traverse_one_hop()
→ four applications, other assets
8: traverse_one_hop()
→ five pairs
9: traverse_one_hop()
→ one application, one technology service
10: traverse_one_hop()
traverse_one_hop()
→ one pair, one application
11: traverse_one_hop()
→ one pair
12: get_report_data()
→ application lists, organisational-unit lists
13: result:DelayedDebtOrgUnits
→ wrong answer

Wrong answer: recall 1.00, precision 0.56; four incorrect pairs.

H2

8 turns; 94k tokens

1: list_reports()
→ 19 reports
2: get_report_definition() ×2
→ report definitions, debt_action column
3: get_report_column_suggestions()
→ five action values
4: get_report_data()
→ two delayed debt items
5: traverse_one_hop()
calc application IDs from results
→ four applications
6: traverse_one_hop()
→ Consumes, Owns references
7: **calc** pairs from Consumes references
→ five pairs
8: result:DelayedDebtOrgUnits
→ exact pass

Exact pass: recall 1.00, precision 1.00; five expected pairs.

H3

8 turns; 83k tokens

1: Metamodel.fetch_full()
→ component types
2: mm.component_type_detail()
→ Debt Item fields, relationships
3: ComponentQuery().fetch()
→ two delayed debt items
4: TraversalBuilder.hop().build()
g.nodes_of()
→ four applications
5: mm.ref_types_between()
mm.component_type_detail()
→ Consumes, Owns
6: TraversalBuilder.hop().build()
→ five Consumes edges
7: **calc** pairs from Consumes edges
→ five pairs
8: result:DelayedDebtOrgUnits
→ exact pass

Exact pass: recall 1.00, precision 1.00; five expected pairs.

Figure 6.3: Complete T13 run-1 traces at model-turn granularity. Line numbers are one-indexed model turns. H1 entries show the recorded JSON tool calls; H2 and H3 entries show each recorded execution call followed by the relevant in-sandbox calls or computation. Full code and verifier output: Appendix A.8.

efficient while hiding the omitted scope constraint. Because much of its domain interface is focused on graph structure and traversal, H3 leaned towards graph queries, instead of relying on the user-specified reports.

The other H3 losses show related failure patterns. In H3 T11#2, the harness retrieves the criticality-5 capabilities but computes the complement over the smaller viewpoint subgraph, producing low recall. In H3 T17#4, it selects an expiring approved-products quality report rather than the report for deployed technology products ending support within one year. In H3 T20#5, it discovers both *Belongs To* and *Owns* relations between people and organisational units, but looks for organisational units that own Darryl rather than organisational units that Darryl owns. It therefore keeps only Executive Management and misses COO Office, Operations, and their applications. These failures are not mainly metamodel discovery failures: they instead concern scope, report provenance, and edge direction after a plausible abstraction has been selected.

H3's advantages show the complementary fit. T01 and T23 benefit from domain-module functions that preserve endpoint type constraints: H3 T01#1 restricts the mitigating source to controls, while H3 T23#1 filters relationship sources to business capabilities. T10 is weaker evidence because H3 still fails two runs, but its successful runs show the domain module can carry assignment and hierarchy semantics compactly. T12 is the strongest positive trace: H3 T12#7 passes in 10 turns and 115.4k tokens, while H2 T12#3 times out after 22 turns and 362.8k tokens despite having computed the necessary intermediate data. Because H3 is still only 1/7 on T12, this is a completion example rather than a stable correctness advantage.

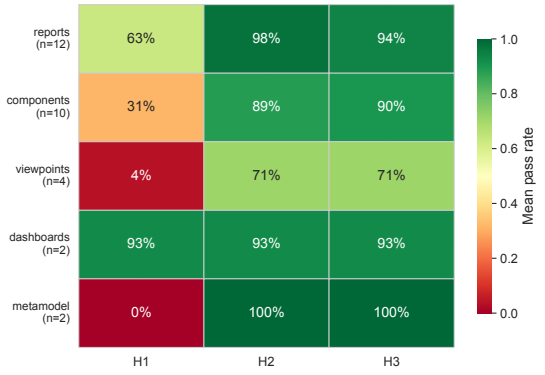
H3's clearer advantage is operational: mean tokens fall from 113.2k to 91.8k, with lower total-token use on 17 of 24 tasks, and final context falls from 14.6k to 13.1k (Table 6.5). Per-run cost also falls from \$0.11 for H2 to \$0.10 for H3 ($p_{\text{adj}} = 0.012$; Table A.1), although the absolute difference is small. H3 therefore reduces total token traffic, final-context size, and model cost while giving up only one exact answer; mean latency is lower descriptively (34.8 s to 30.8 s), but the paired latency change is not detected ($p_{\text{adj}} = 0.356$).

6.4.2 Task Characteristics

The required forms of composition from Section 2.4 help interpret the discriminating tasks but are not independent factors. T13 combines dependent typed traversal with retained intermediate identifiers; T14 and T19 construct and compare sets; T15 and T22 require metamodel discovery and local graph computation; and T07 and T17 join sizeable graph-derived collections. The observed gaps occur where several burdens co-occur, so the campaign cannot attribute a separate causal effect to any one dimension.

The overlapping tag views in Figure 6.4 provide a descriptive check. H1 has 4% mean pass on the four viewpoint tasks and 0% on the two metamodel tasks, compared with 71% and 100% for H2 and 71% and 100% for H3. H1 is stronger on report and dashboard work, where retrieved rows are generally closer to the requested answer. Component and viewpoint tasks are also costly for H1, at 447k and 407k mean tokens, compared with 157k and 206k for H2 and 100k and 130k for H3.

The technique tags in Figure 6.5 tell the same story for the better-populated groups. Across 14 graph-traversal tasks, mean pass is 30% for H1, 88% for H2, and 90% for H3; mean token use is 427k, 123k, and 96k. Across six set-difference tasks, pass is 14%, 95%, and 90%. Sparse rows such as hierarchy, set intersection, and date filter contain only one task and are case labels rather than stable category estimates. Because tags overlap, differences between rows are descriptive and cannot be interpreted as indepen-

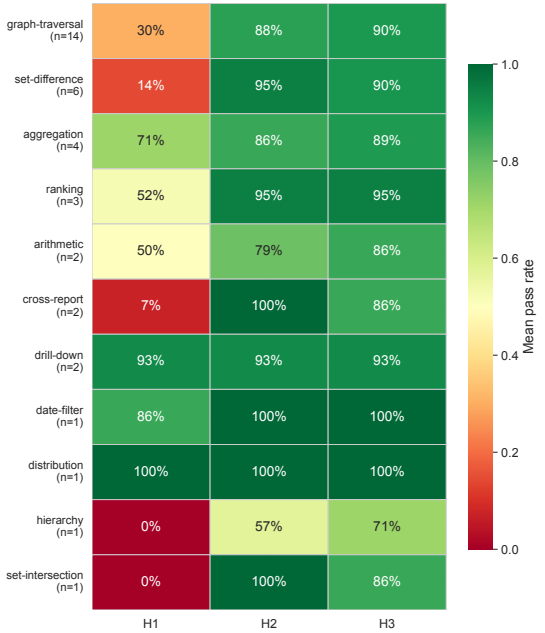


(a) Pass rate by API resource type tag.

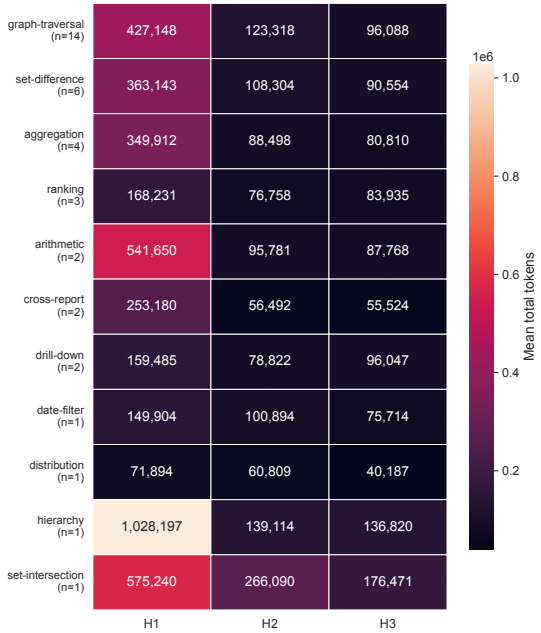


(b) Mean token consumption by API resource type tag.

Figure 6.4: API resource tag correctness and token use by harness.



(a) Pass rate by reasoning technique tag.



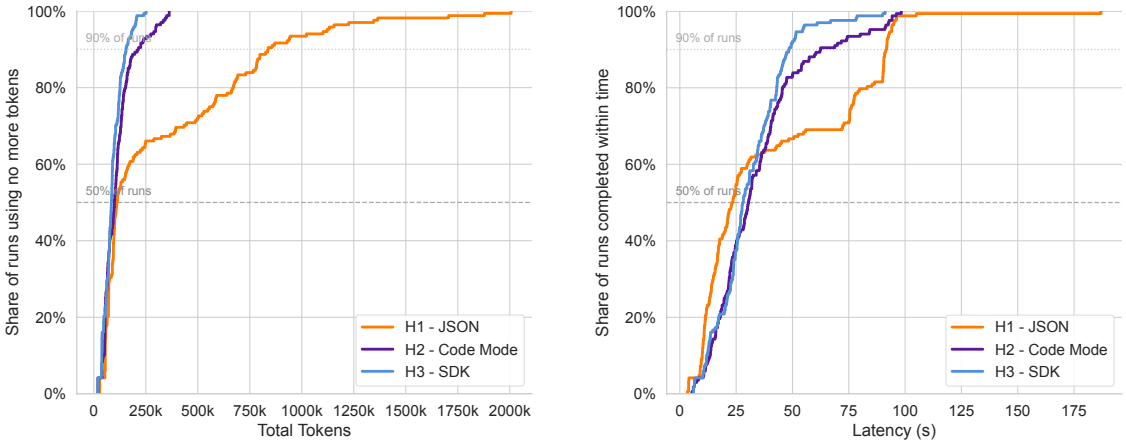
(b) Mean token consumption by reasoning technique tag.

Figure 6.5: Reasoning technique tag correctness and token use by harness.

dent tag effects.

The survey-derived difficulty score does not explain much variation. Rounded to integer levels it spans only levels two through four (three, seventeen, and four tasks), giving limited spread, and its Spearman correlations with per-task pass rate and resource use are weak for every harness ($|\rho| \leq 0.31$), and none is significant (all $p > 0.15$; Table A.2). Resource use trends weakly upward with difficulty for H1 and H2 but not H3, and pass rate shows no consistent association. Appendix A.2 therefore retains these diagnostics rather than using them as primary SQ3 evidence.

6.5 Operational Behaviour



(a) Token consumption. Curve height is the share of runs using no more than the x-axis value.

(b) Wall-clock latency. Curve height is the share of runs completed within the x-axis value.

Figure 6.6: Cumulative distributions of total token consumption (left) and wall-clock latency (right). At any x-axis value, the curve height gives the percentage of runs that used no more than that many tokens or completed within that time. Curves farther left indicate lower resource use or faster completion; horizontal reference lines mark 50% and 90% of runs.

Figure 6.6 shows H1's long resource tail, reaching approximately two million tokens, and the effect of the 90-second boundary on latency. Medians and interquartile ranges by outcome therefore complement means.

Table 6.6 reports resource use by terminal outcome. H1's passing runs use a median 93.9k tokens, six turns, and 16.4 s. Its non-passing runs use 413.5k tokens, 18 turns, and 72.2 s. The split does not show that token use causes failure, because difficult tasks and long unsuccessful searches also cause more interaction. It does show that H1's aggregate cost is driven by runs that spend much more model effort without returning an exact answer. H2 and H3 show the same direction but a much smaller gap: their non-passing median token counts are 207.5k and 104.6k.

Failed H1 traces often repeat retrieval or carry large intermediate sets across turns. This supports a high-cost failure tail; "runaway context" applies only to traces with demonstrated observation growth.

Table 6.6: Resource use conditioned on terminal exact-pass outcome. Values are median [interquartile range]; the split is descriptive and does not imply that resource use causes failure.

Harness	Outcome	n	Total tokens (k)	Wall time (s)	Turns
H1	Pass	66	93.9 [70.8, 127.2]	16.4 [11.8, 23.5]	6.0 [5.0, 7.0]
	Non-pass	102	413.5 [91.2, 775.0]	72.2 [16.8, 90.4]	18.0 [6.0, 28.8]
H2	Pass	154	96.2 [56.6, 131.2]	29.5 [19.6, 40.6]	8.0 [5.0, 10.0]
	Non-pass	14	207.5 [131.8, 325.3]	76.6 [39.8, 93.9]	14.0 [9.2, 20.5]
H3	Pass	153	82.6 [57.3, 117.5]	27.0 [20.2, 39.3]	7.0 [6.0, 10.0]
	Non-pass	15	104.6 [96.4, 132.5]	37.8 [33.4, 48.9]	9.0 [8.0, 11.5]

6.5.1 Token Usage and the Context Window

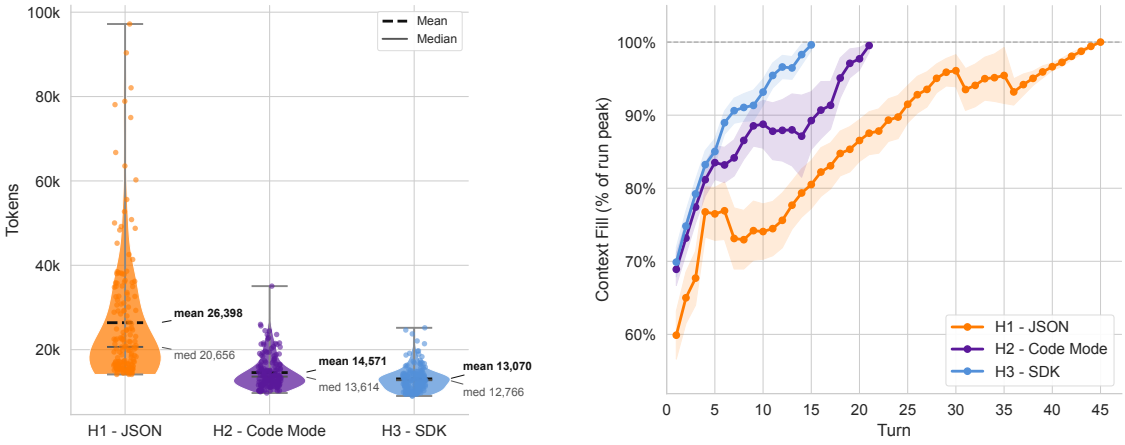
Two token measures describe different costs. Total tokens (Table 6.7) measure cumulative traffic; the final context (Figure 6.7a) measures the last input and determines whether a workflow fits a fixed budget. H1 averages 329.7k total tokens but a 26.4k final window, so most of its traffic is context re-read turn after turn rather than newly retained state. H2 and H3 are leaner on both axes (113.2k and 91.8k total; 14.6k and 13.1k final), and their windows vary less in this campaign. H1's final window has a standard deviation of 15.5k and reaches 97.2k on its worst run, against peaks of 35.1k and 25.2k for H2 and H3. The code-execution harnesses therefore keep individual model calls smaller and lower-peaked under the evaluated tasks and budgets.

Table 6.7: Absolute token usage and context window size per harness (mean $\pm$ std). *Total tokens* is cumulative model traffic over every turn; *Final context* is the size of the last window sent to the model and *Peak context* its single largest observed value. The gap between total and final tokens reflects context re-read across turns.

Harness	Total tokens	Final context	Peak context	Output
H1	$329.7k \pm 389.3k$	$26.4k \pm 15.5k$	$97.2k$	$1.9k \pm 1.6k$
H2	$113.2k \pm 73.8k$	$14.6k \pm 3.9k$	$35.1k$	$2.0k \pm 1.3k$
H3	$\mathbf{91.8k \pm 46.6k}$	$\mathbf{13.1k \pm 3.0k}$	$\mathbf{25.2k}$	$\mathbf{1.6k \pm 0.9k}$

Figure 6.7b shows how each window fills. Static prompts and documentation occupy 60–70% of the initial input. H2 and H3 peak within roughly ten to twenty turns, because each code action folds retrieval, traversal, and aggregation into one observation. H1 instead climbs gradually across as many as forty-five turns, each tool result adding to the window; this is the per-turn view of the accumulation that produces its long total-token tail.

Figure 6.8 shows how the final window is allocated. H1's largest component is accumulated tool output, 10.3k tokens or 39% of the final window: retrieved rows and intermediate results serialised into the dialogue. Tool results are much smaller for H2 and H3, at 2.7k and 2.4k tokens, around 19% each, because intermediate state can remain in the sandbox. Tool definitions are instead their largest component, at 7.1k tokens (48%) for H2 and 6.1k (46%) for H3. The model's emitted call arguments are a separate component, close to 10% of the window for H2 and H3 but only 4% for H1, while user messages are negligible throughout. Retained model reasoning is a further 4.5k tokens (17%) for H1



(a) Final context window size per run.

(b) Context fill per turn as a fraction of each run's peak window.

Figure 6.7: Final context window size per harness (left) and context fill per turn as a fraction of each run's peak window (right).

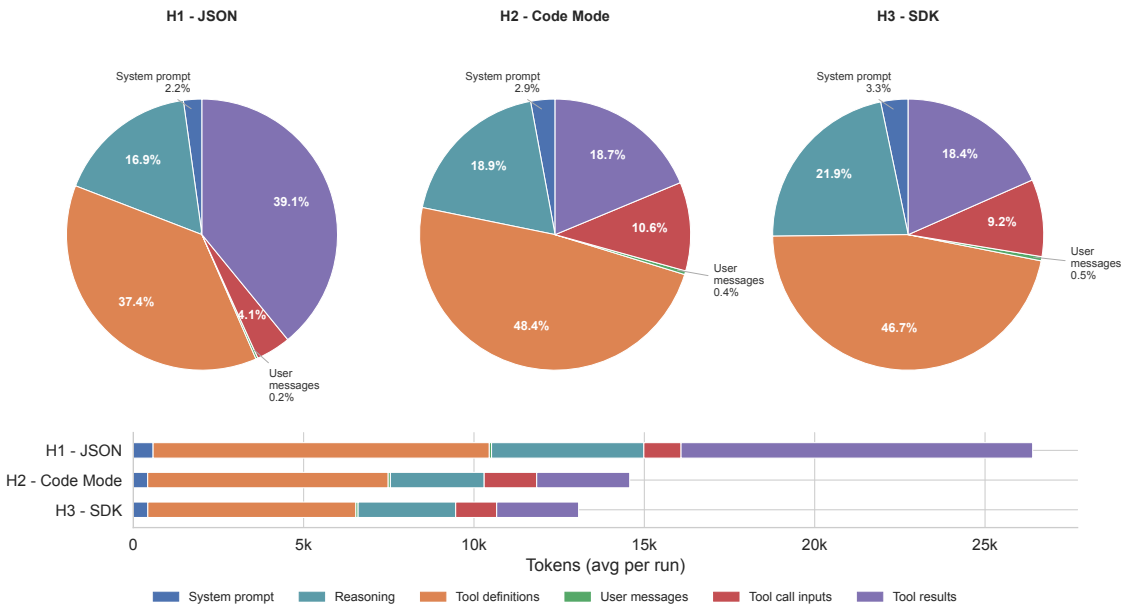
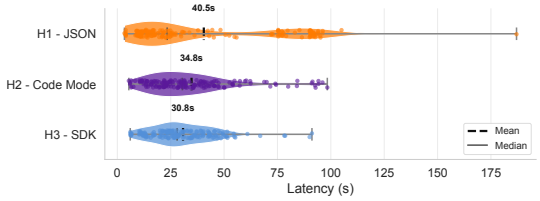


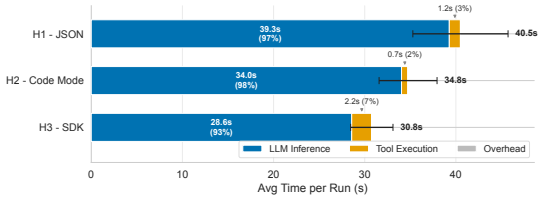
Figure 6.8: Average context window composition per harness: breakdown by component type (top) and across harnesses (bottom).

and 2.8k and 2.9k (19% and 22%) for H2 and H3, and the static system prompt is a small fixed remainder near 0.5k.

6.5.2 Latency



(a) Total latency distribution per harness. Dashed line marks the mean, solid line the median.



(b) Mean latency stacked into LLM inference, tool execution, and overhead with 95% CI error bars. Computed on runs with full instrumentation.

Figure 6.9: Wall-clock latency distribution and component breakdown per harness.

Model inference dominates mean latency: 97% for H1, 98% for H2, and 93% for H3 (Figure 6.9). H3 spends more recorded tool time than H2 (2.2s versus 0.7s) because domain operations hide more work inside each action. The 90-second boundary shapes H1's slow tail.

Neither paired latency difference is detected, as shown in Table 6.5. H3's lower mean is therefore a campaign average, not evidence that every task is faster. Among runs that avoid timeout and turn exhaustion, H1 averages 19.6 s, compared with 32.2 s for H2. This subset excludes the long H1 failures at the operating boundary, so it should not be read as evidence that H1 is generally faster.

Generated source is also only a diagnostic. Per `execute_code` call, H3 produces less Python source than H2, with 12.1 ± 7.9 total lines compared with 13.9 ± 10.0 for H2. This is consistent with recurring coordination being encoded in reusable operations. Source length does not measure implementation complexity, cognitive effort, hidden platform work, or maintainability, and is therefore not used as a headline benefit.

6.6 Failure Behaviour

Failure analysis separates terminal outcomes, execution events, partial-answer quality, and interpretation. H1 averages only 0.10 malformed calls and 0.10 framework retries per run. Its 102 non-passing runs are therefore not primarily schema-validation failures. For H2 and H3, sandbox events are uncommon and usually recoverable. As Table 6.8 shows, six of eight H2 runs with runtime errors still pass, as do all eleven such H3 runs.

Table 6.8: Observable sandbox-error incidence and recovery. A run is recovered when it contains at least one sandbox error but nevertheless terminates with an exact pass.

Harness	Runs with error	Recovered to pass	Recovery rate	Errors / affected run
H2	8 / 168	6 / 8	75.00%	1.0
H3	11 / 168	11 / 11	100.00%	1.4 ± 0.5

Table 6.9 restricts partial credit to failed collection answers. H1's mean recall (0.92) is much higher than its precision (0.69), with 3.27 extra items and 0.88 missing items on

average. Its typical collection failure is therefore more often over-inclusion than loss of the last expected items. The pattern is especially visible on T13, T16, T17, and T23, where failed H1 runs often retain recall of 1.0 while including irrelevant components. The extra and missing item counts for H3 were heavily inflated by a few badly wrong answers.

Table 6.9: Partial-credit quality for non-passing collection-answer runs only (mean ± std). Extra and missing counts distinguish over-inclusion from under-inclusion.

Harness	Eligible failed runs	Precision	Recall	F1	Extra items	Missing items
H1	41	0.69 ± 0.26	0.92 ± 0.19	0.76 ± 0.22	3.27 ± 2.92	0.88 ± 1.47
H2	7	0.74 ± 0.31	0.93 ± 0.09	0.78 ± 0.26	2.71 ± 2.14	1.29 ± 1.11
H3	13	0.62 ± 0.32	0.76 ± 0.27	0.59 ± 0.29	25.54 ± 49.52	6.85 ± 12.34

H2 has only seven eligible failed collection runs, so its similar aggregate precision and recall have a small denominator. H3's 13 eligible failures are more heterogeneous. T16 failures have recall 1.0 but very low precision because the missing deployment constraint admits a large product set. In contrast, the failed T11 and T17 runs under-retrieve relevant capabilities or technology products. This mixture produces lower mean precision and recall and very large variance in extra and missing counts. The result does not show when a partial answer would be useful in practice; it diagnoses the direction of error under the strict exact-pass contract.

Every failing run also carries a post-hoc self-reflection in which the agent names a probable failure cause. These reports record how the agent perceives a run from inside the loop, not a verified diagnosis; the verifier remains the only ground truth, and they are read here as the agent's perspective on its own failures rather than as evidence of their causes. Manual trace inspection found that the reports often aligned with the observed issue, but this alignment was not coded as a separate measure. Their content differs systematically by harness; Appendix A.7 (Table A.9) lists representative examples for each.

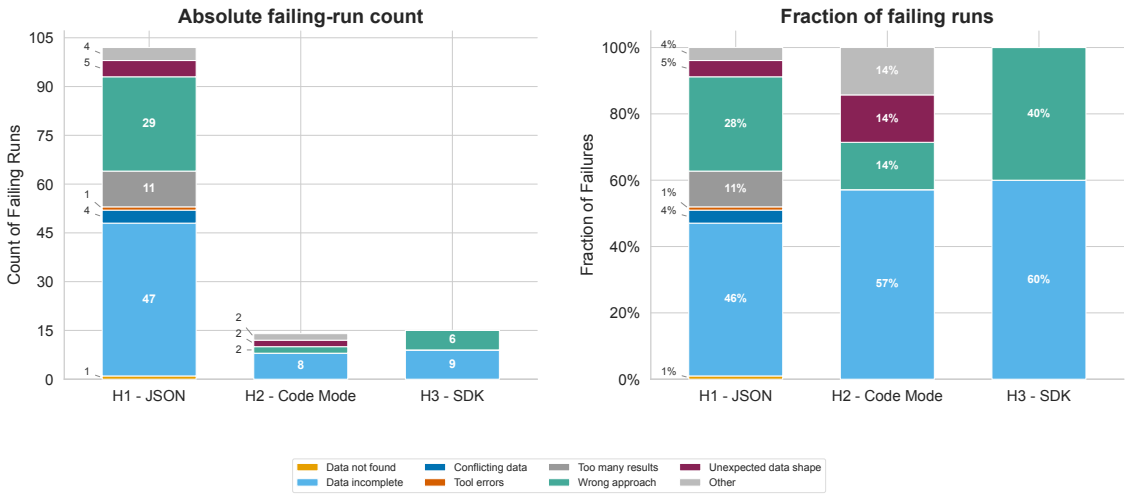


Figure 6.10: Agent self-reported failure type distribution per harness: fraction of failing runs (left) and absolute count (right).

Figure 6.10 shows their distribution. Across all three harnesses the most common self-perceived cause is incomplete data (46% of H1 failures, 57% for H2, 60% for H3): even the code-execution harnesses, on the rare runs they fail, more often read the failure as missing information than as a flawed method. "Wrong approach" is the next most frequent

category (28% for H1, 14% for H2, 40% for H3). H3's larger share is consistent with the semantic-selection shortcut documented for the T12 boundary case (Section 6.6.1), where the agent matches on a supplier name instead of following the supplier relation.

"Too many results" appears only under H1 (11% of its failing runs) and never under H2 or H3. None of these eleven runs submitted an over-inclusive answer that the verifier rejected; all reached the time or turn budget first (nine timeouts and two turn-limit terminations) while still expanding an over-broad traversal. Their reflections describe the same shape: a multi-hop expansion that requires many sequential per-entity lookups, including one call per person and per unit, one per at-risk application, and a stack with "300+ Deploys To targets", which cannot be completed within the limit. The label therefore marks budget exhaustion during over-broad retrieval, the serial-call cost of holding and growing the working set in conversation, rather than a submitted answer with too many items. It is the same tendency that, when an answer is returned, surfaces as over-inclusion in the verifier (Table 6.9: H1 precision 0.69 against recall 0.92, with 3.27 extra items on average): one cause, expressed either as a run that never finishes or as an answer that over-includes. The remaining categories (conflicting data, unexpected data shape, tool errors, and data not found) are individually rare and occur almost only under H1. The self-reports broadly track the measured failure modes, but they remain unvalidated and are not used as causal evidence.

6.6.1 The T12 Boundary Case

T12 asks which applications integrate with at least one SAP-supplied application in the application integration map, and which business capabilities those integrating applications realise.

All harnesses have a valid solution path for T12, yet only one of 21 runs passes. H1 and H2 fail all seven runs, while H3 passes once and fails six times. The traces show different reasons for failure rather than an impossible verifier.

In H1 T12#1, the agent first finds nine applications by matching the supplier name, then corrects this shortcut by resolving the supplier organisation and following *Supplies* to all eleven applications. It continues interpreting the application–interface–application graph through serial calls and reaches the timeout without submitting an answer. In H2 T12#3, generated code obtains the same eleven supplied applications, reconstructs the exact nine integrating applications, and retrieves the exact ten realised capabilities. The run then times out before returning its typed final result. This is a completion failure despite the necessary data having been computed.

H3 exhibits a different recurrent failure. In H3 T12#1, it selects applications whose names contain the supplier string rather than following the supplier relationship. This omits two supplied applications whose names do not contain the string, misclassifies one of them as an external integrator, and changes the downstream capability set. Five completed H3 wrong answers repeat this same shortcut, although H3 T12#7 follows a valid path and passes. The task therefore exposes both budget exhaustion during graph interpretation and semantic selection error. Harness design helps with the former but does not guarantee correct interpretation of the supplier relation.

As Table 6.10 reports, excluding T12 raises pass rates to 40.99%, 95.65%, and 94.41% and lowers mean tokens without changing the ranking or conclusions.

Table 6.10: Sensitivity of headline outcomes to T12. The complete campaign remains the primary result; the exclusion shows whether the ranking depends on this boundary task.

Scope	Harness	Correct / total	Pass rate	Total tokens
All tasks	H1	66 / 168	39.29%	$329.7k \pm 389.3k$
All tasks	H2	154 / 168	91.67%	$113.2k \pm 73.8k$
All tasks	H3	153 / 168	91.07%	$91.8k \pm 46.6k$
Excluding T12	H1	66 / 161	40.99%	$313.1k \pm 389.1k$
Excluding T12	H2	154 / 161	95.65%	$104.2k \pm 60.5k$
Excluding T12	H3	152 / 161	94.41%	$90.4k \pm 46.7k$

6.7 Discussion

Paired task analysis shows whether differences repeat across tasks; task groups and traces suggest mechanisms. Taken together, the results show that harness architecture affects performance by changing where intermediate state, coordination, and domain semantics live.

6.7.1 SQ1: Stateful Composition and Execution Responsibility

H2 improves correctness and efficiency together, especially on compositional tasks. H1's low malformed-call rate rules out schema validation as the main explanation. The more plausible mechanism is state placement: H1 repeatedly serialises dependent work through the dialogue, while H2 retains identifiers and performs joins or aggregation in Python before returning a compact result.

Several diagnostics support this interpretation and weaken alternative explanations. H1 averages only 0.10 malformed calls and 0.10 framework retries per run, so its 102 non-passing runs are rarely invalid JSON calls. Its failed collection answers are also not simple non-retrieval failures: their mean recall is 0.92, while precision is 0.69, indicating frequent over-inclusion after substantial relevant retrieval. The context-window breakdown points to the same mechanism. H1 retains 10.3k tokens of tool results in the final window, compared with 2.7k for H2, because intermediate rows, identifiers, and partial sets re-enter the conversation. H2 can keep those values in execution state and print only selected summaries or final projections.

The task evidence follows the same pattern. T13 illustrates the mechanism in one trace, while the broader gains on traversal, set construction, metamodel lookup, and cross-source composition show that the effect is not trace-specific. This interpretation agrees with work on executable control flow [46, 14] and conversational state dependence [45]. SQ1 therefore supports the realised programmatic harness: executable state, local data structures, compact printed observations, and recoverable execution errors.

The implication is not merely to replace JSON with Python. It is to give dependent work a bounded execution environment where values can be retained and transformed without repeated conversational mediation. The corresponding cost is an infrastructure responsibility: isolation, inspection, limits, recovery, and observability become part of the harness.

6.7.2 SQ2: Semantic Abstraction and Governance Responsibility

H3 changes how the agent works more than how well it does. It returns one fewer exact answer than H2, a difference the paired test does not detect ($p_{\text{adj}} = 0.608$, median task difference zero) and that could be run-to-run variation, while total tokens and final context fall consistently across tasks. Per-run model cost also falls from \$0.11 to \$0.10 on average, with a paired median task difference of $-\$0.0095$ ($p_{\text{adj}} = 0.012$), while mean latency falls from 34.8 s to 30.8 s without a detected paired change ($p_{\text{adj}} = 0.356$). The notable effect is not the single missing answer but the change in failure composition: H3 reaches the budget far less often than H2 (one timeout against seven) yet returns more wrong answers (fourteen against seven). The abstraction makes the agent more decisive, completing compositions that H2 leaves running, while sometimes committing to a selection assumption that a lower-level program would have kept open to question.

This mixed result matches evidence that abstraction level is a performance lever, not a monotonic improvement [57, 16]. The H3 result is consistent with higher-level skills reducing token traffic, but it does not show that higher abstraction generally improves reliability. In this campaign, abstraction reduced total tokens and final context while leaving exact pass statistically indistinguishable from H2 and shifting the failures toward semantic mistakes.

The trace evidence separates two kinds of semantic responsibility. H3 helps when a domain-module function preserves endpoint-type, relationship-source, or hierarchy semantics that generated code would otherwise need to rediscover, as in T01, T23, and the successful H3 run on T12. It hurts when the selected operation, report, or traversal fixes the wrong scope, provenance, direction, or selection rule, as in T16, T17, T20, and the repeated T12 supplier-name shortcut. The central issue is therefore governance of abstraction boundaries. Once semantics move into maintained code, correctness depends on names, operation scope, validation rules, examples, and escape routes back to lower level inspection. The experiment did not detect a systematic correctness difference, but 24 tasks do not establish equivalence between H2 and H3. SQ2 therefore supports H3 when reusable, governed domain-module functions justify transferring semantic responsibility into maintained code: its value is leaner interaction and reuse at minimal correctness cost, not a correctness upgrade over H2.

6.7.3 SQ3: Task Sensitivity

The largest harness differences occur on graph traversal, metamodel analysis, set construction, cross-report comparison, and viewpoint-based composition. These tasks require several of the compositional burdens introduced in Section 2.4 at once: dependent calls, retained intermediate identifiers, local computation, schema discovery, and choice among platform abstraction layers. H1 remains adequate on several compact report, dashboard, and aggregation tasks where the platform already returns data close to the final answer.

The task results separate three workflow regimes. In direct or asset-led retrieval, such as dashboards, compact reports, and some aggregations, the benefit of code execution is smaller because the retrieved asset already resembles the answer. In stateful composition, including graph traversal, set-difference, metamodel, and cross-report work, H2 and H3 dominate because they can retain identifiers, branch over collections, and compute locally. In semantically loaded abstraction, where correctness depends on relationship direction, report provenance, viewpoint scope, or component-type constraints, H3 can either preserve the intended semantics or make a wrong shortcut cheaper to execute.

The tag views support these regimes without making the tags causal factors. H1 has

4% mean pass on viewpoint tasks and 0% on metamodel tasks, compared with 71% and 100% for both code harnesses. Across graph-traversal tasks, mean pass is 30% for H1, 88% for H2, and 90% for H3; across set-difference tasks, it is 14%, 95%, and 90%. These rows overlap and some contain few tasks, but they show where the aggregate result comes from. T12 marks the boundary for the H2–H3 transition: H3's single successful run shows that a domain module can reduce enough orchestration to complete a difficult composition, while the remaining failures show that abstraction still cannot ensure correct interpretation or timely completion.

SQ3 therefore answers the main research question in structural terms. Harness architecture affects performance by deciding which coordination remains probabilistic and conversational, which state and computation become executable, and which domain semantics are embedded in reusable code. It can reduce coordination burden, but it cannot eliminate semantic interpretation or completion risk.

6.7.4 Practical Implications

The empirical and engineering implications are asymmetric. Moving from H1 to H2 adds sandboxing, generated-code inspection, documentation, retry handling, and timeout controls. The benchmark nevertheless gives strong evidence that this infrastructure is worthwhile for the evaluated compositional workflows. Moving from H2 to H3 adds operation design, validation logic, normalised return structures, and abstraction governance. Its justification depends more on reuse and organisational control than on a general empirical advantage.

The practical sequence follows the same pattern. H1 fits narrow workflows when direct calls are sufficient, when each platform action should remain explicitly visible in the conversation, or when code-execution infrastructure is not justified. H2 is the stronger default for multi-step analytical workflows when sandboxing and generated-code inspection are acceptable. H3 should be adopted more selectively: it is most defensible when recurring analyses justify maintained domain operations and when the organisation can govern their names, assumptions, validation rules, documentation, and low-level escape routes. A domain module that hides too much can make the agent faster at the wrong interpretation.

Table 6.11 separates measured results from engineering work that was not evaluated. The guidance is scoped to analytical queries; write operations, approval gates, human usability, and production governance were not evaluated.

Table 6.11: Practical decision framework separating benchmark evidence from engineering commitments and bounded guidance.

Harness	Benchmark evidence	Engineering commitment	Bounded guidance
H1	Competitive on several compact tasks whose platform outputs were close to the answer; weak on high-composition workflows and costly when failing.	JSON-schema and serialisation maintenance; direct model-visible tool traces; no code-execution service.	Use for narrow workflows when direct calls are sufficient or execution infrastructure is unjustified; do not treat it as the default for the evaluated compositional tasks.
H2	Highest exact pass; large, consistent gain over H1 with lower tokens, turns, and campaign cost.	Sandbox isolation, code inspection, API documentation, limits, retries, credentials, and observability.	Prefer for the evaluated multi-step analytical workflows when code execution is operationally acceptable.
H3	Close to H2 in exact pass (one fewer run); lower total tokens, final context, and model cost, with no detected latency change.	Domain-operation design, validation, semantic governance, documentation, and low-level escape strategy.	Adopt selectively when stable recurring operations justify moving coordination and semantics into maintained domain code.

6.7.5 Threats to Validity and Generalisability

The detailed validity account is given in Section 4.8. The main boundary for these findings is that each harness is an implemented condition, not a single isolated design variable. Provided results are also scoped to this single case-study.

The results also depend on the benchmark setting: one model route, one seeded Ardoq deployment, one implementation of each harness, and 24 analytical tasks. Exact pass measures agreement with the oracle, not explanation quality or procedure quality. Task tags and difficulty scores locate patterns but do not identify causal task attributes. H3 internal work is less observable than H2 wrapped calls, so comparisons rely on model-visible actions and run-level resources rather than symmetric platform-operation counts. The A/B studies justify the compact H1 and H2 variants used in the main comparison, but they are design-validation checks on a five-task subset rather than independent confirmation of SQ1 or SQ2. The T12 exclusion shows that the headline ranking does not depend on one boundary case, but it does not remove the limits of the fixed benchmark.

Table 6.12 consolidates the finding, primary evidence, and claim boundary for each research question. Within these boundaries, the evidence supports H2 as the strongest default for the evaluated compositional workflows and H3 as a selective abstraction trade-off rather than a correctness upgrade. The main research question can therefore be answered as follows: harness architecture changes Ardoq-agent behaviour by allocating work among the conversation, execution state, and maintained domain code. The largest observed gain comes from moving stateful composition out of the conversation. Raising the same execution model to a domain module mainly reduces interaction cost and changes failure composition, rather than improving exact correctness.

Table 6.12: Research-question findings, primary evidence, and claim limitations.

Question	Finding	Primary evidence	Limitations
SQ1	H2 improves exact pass and model efficiency together: $+52.4$pp pass, -216.5k mean tokens per run, and no turn-limit outcomes.	Tables 6.3 and 6.5; Figs. 6.1 and 6.2.	Realised bundled transition on one model, platform, task suite, and fixed budgets.
SQ2	H3 cuts interaction at a minimal pass cost: -21.4k mean total tokens, -1.5k mean final context window, and a -0.6pp pass difference relative to H2.	Tables 6.5 and 6.7; Fig. 6.7; task matrix and T16 trace.	One domain-module design; hidden composition prevents symmetric inner-call comparison.
SQ3	Gaps concentrate on workflows requiring graph, set, schema, and cross-source composition; compact report-led tasks show smaller differences.	Fig. 6.2; resource and technique tag heatmaps; T13 trace case.	Tags overlap and sparse rows are case evidence, not independent causal factors.

7 Conclusion

This thesis treated the agent–platform interface as a performance-bearing architectural variable and located three concrete harnesses in a two-dimensional design space of invocation style and abstraction level (Figure 2.7). H1 uses traditional tool calling over low-level operations; H2 keeps those operations but moves invocation into code execution; H3 keeps code execution but raises the operations to an agent-native domain module. The staged comparison is organised around two axes: H1–H2 changes invocation style and state placement, while H2–H3 changes abstraction level over a shared execution model. The controlled study compared the three on 24 read-only Ardoq tasks, with seven runs per task–harness cell and typed live-API oracles.

Within this space, the clearest result came from the change in invocation style, not the change in abstraction level. Moving dependent work into an executable environment improved completion and efficiency together; raising that same environment to a domain module made the interaction leaner with little change in correctness. The harness governs how work is divided among the model, the conversation, the runtime, and reusable domain code, and that division is where the measured differences came from.

7.1 Answer to the Research Question

SQ1—invocation style. Replacing H1 with H2 produced the largest and most consistent effect. Exact pass rose from 39.29% to 91.67% and mean tokens fell from 329.7k to 113.2k, with fewer turns and no turn-limit outcomes; every non-zero task-level difference favoured H2. The gaps concentrated on graph traversal, set construction, metamodel analysis, and cross-source composition, tasks where H1 repeatedly serialised intermediate values through the conversation while H2 retained and transformed them in code. SQ1 supports the implemented programmatic harness as a bundle, not code syntax isolated from state, documentation, and observation flow.

SQ2—abstraction level. Replacing H2's low-level functions with the H3 domain module changed how the agent worked more than how well it did. Exact pass fell by one run (153 versus 154 of 168, $p_{\text{adj}} = 0.608$), while total tokens fell from 113.2k to 91.8k, with lower total-token use on 17 of 24 tasks, and final context fell from 14.6k to 13.1k. Cost followed the same direction, falling from \$0.11 to \$0.10 per run ($p_{\text{adj}} = 0.012$). Latency should be read more cautiously: H3's mean was lower than H2's (30.8 s versus 34.8 s), but the paired task comparison did not support a reliable latency reduction ($p_{\text{adj}} = 0.356$). The abstraction bought compaction at little correctness cost, not a correctness gain. Its failures also shifted in kind: H3 reached the budget far less often than H2 but returned more wrong answers, so the module made the agent more decisive while roughly maintaining accuracy, completing compositions that H2 left running while sometimes committing to an under-specified interpretation. Task-level gains, where a domain operation preserved type or hierarchy semantics, roughly offset losses, where an operation or report fixed the wrong scope, provenance, or edge direction. Twenty-four tasks do not establish equivalence, but within this study the module delivered leaner interaction for a one-run drop in exact pass.

SQ3—task sensitivity. The differences tracked workflow structure. H1 stayed competitive when reports, dashboards, or aggregations returned data close to the answer, and weakened when tasks combined dependent retrieval, retained identifiers, local computa-

tion, and schema discovery. Overlapping tags and survey-derived difficulty make these descriptive rather than causal task effects.

7.2 Contributions

The thesis makes four contributions.

1. **An architectural framing and controlled empirical comparison.** The study treats the harness as a performance-bearing variable and stages two interpretable transitions within one platform. Programmatic composition improved completion and efficiency; the agent-native domain module then reduced interaction traffic at minimal correctness cost.
2. **An objectively verifiable enterprise architecture benchmark.** The 24-task suite translates analytical Ardoq workflows into typed contracts and trajectory-independent live-API oracles. It covers the platform's main query, traversal, and metamodel operations while permitting different valid solution paths, requiring retrieval and computation that can inform professional EA judgement without automating it. Additionally, a human expert survey confirms the real-life applicability of the created tasks. The proprietary code and seeded deployment make it an internal research artefact.
3. **A repeated-trial evaluation methodology.** The pipeline combines exact verification with task-paired analysis, resource telemetry, terminal outcomes, partial-answer diagnostics, and auditable traces, characterising stochastic variation while separating verified outcomes from observed mechanics and trace interpretations.
4. **Bounded guidance for enterprise agent-interface design.** The evidence consolidates into a conditional selection framework (Table 6.11) that ties each harness to the workflow structure and engineering commitment it suits, rather than a single ranking. It also provides actionable insight into the design of H1 and H2, both applicable to production deployments at Ardoq, by empirically comparing A/B variations of each.

Together, the contributions connect harness architecture to measured behaviour on a schema-governed enterprise platform, and the benchmark and pipeline support later harness tests under the same verification method.

7.3 Practical Implications

Code execution is an architectural investment, not a formatting convenience. For iteration, joins, aggregation, graph traversal, or schema search, it keeps intermediate data outside the conversation and supplies ordinary programming constructs. These benefits must be weighed against isolation, code inspection, timeouts, retries, credential boundaries, and observability. Short report-led workflows may not justify that infrastructure.

For organisations facing the workflow types evaluated here, the practical sequence is to retain H1 for narrow, direct interactions, adopt H2 as the default for compositional analysis when code execution is operationally acceptable, and invest in H3 only when stable recurring operations justify the additional abstraction-governance cost. In this campaign, estimated model cost per successful run was approximately $0.56 for H1, $0.12 for H2, and $0.11 for H3. These figures exclude the engineering and operating costs of sandboxing, security controls, observability, and domain-module maintenance.

An agent-native domain module transfers coordination and semantic assumptions into maintained platform code. This can improve compactness, but makes abstraction design part of product governance. Enterprise architects and governance stakeholders may

help define stable concepts, permitted operations, validation, report-selection rules, directionality guarantees, and audit requirements. Because the benchmark did not evaluate human usability or governance, this remains an implementation implication.

7.4 Scope of the Conclusions

These conclusions cover three implemented harnesses, one frontier model, one seeded Ardoq deployment, fixed budgets, and 24 read-only tasks. The deployment contains rich solution assets but no live customer records. Mutation, approval workflows, concurrent users, external integrations, and sustained production operation remain untested, and other models, datasets, and platforms may behave differently.

The staged design compares complete harnesses rather than independently isolating syntax, state, documentation, observation flow, validation, or abstraction. It also does not quantify implementation effort, maintenance, security, auditability, or human oversight. The result is a bounded architectural choice, not a universal ordering or claim of production readiness.

7.5 Future Work

The design space points to the most direct extension. The three harnesses occupy three of its four corners (from Figure 2.7); the missing one is high-level domain operations exposed through traditional tool calling rather than code execution. Building it would vary abstraction within the tool-calling style and isolate whether H3's compaction came from the domain abstraction or from the move to code execution. It would also show what tool definitions result once domain operations have to be expressed as discrete calls rather than a code module.

Beyond the missing corner, the priority is replication across model families, platforms, datasets, and independently constructed task suites, to test whether the H1–H2 result generalises beyond Ardoq and whether models benefit equally from execution state. Replications should retain paired tasks and repeated runs while adding prompt perturbations and controlled infrastructure failures.

H3 is one point in a larger design space, and several directions could improve it. The most direct is to widen the module by adding operations, so that more common analyses are served by a single domain call rather than assembled by hand. The module could also be built on a different programming paradigm, such as a more object-oriented or a more functional one, to test which style the model uses more reliably. A larger change is to move away from low-level primitives and encode workflows directly: instead of exposing the building blocks of an analysis, the module would expose the analysis itself. Each of these choices raises a question the study leaves open, namely how to design the SDK well and where an abstraction becomes too abstract, hiding so much that the model can no longer tell what an operation will do. Further out, the agent could maintain the module itself and form a complete feedback loop. As it answers user queries, it would add new operations or revise existing ones to match the questions it encounters, either during a run or afterwards, so the module grows in step with how the platform is actually used.

Production-oriented evaluation should add write permissions, approval gates, rollback, and state restoration, with procedure-aware verification that checks data sources, traversal constraints, provenance, and prohibited shortcuts alongside final answers. Longitudinal and multi-user studies could then measure drift, concurrency, recovery, and oversight. Cost is the other open variable: the campaign priced model usage but left out sandbox operation, security controls, observability, interface maintenance, and the governance

each domain abstraction requires, with documentation drift a recurring cost specific to H3 whose value depends on its descriptions still matching the platform they wrap. A full accounting would show whether the leaner interaction of H2 and H3 holds once the supporting infrastructure is counted.

7.6 Closing Perspective

Agent capability depends on the interface through which the model observes, composes, and executes work. In this study, the transition that kept dependent operations and intermediate values in an executable environment produced the clearest gain; adding a domain abstraction made interaction leaner while barely affecting correctness. A harness allocates work among the model, context, runtime, and domain code. That allocation is a first-class architectural decision.

This matters beyond Ardoq. As organisations move LLMs from pilots into the systems where work actually happens, the value they capture will depend less on the model alone than on how it is connected to applications, data models, permissions, and procedures. The interface decides which work the model reasons through token by token and which it offloads to code or to the platform, and that choice affects not only whether a task succeeds but what it costs to run. The question does not fade as models improve. A more capable model still acts through some interface, and a better one frees that capability for the parts of a problem that genuinely need judgement.

The agent-native direction pushes this further, asking what an interface would look like if it were designed for the agent rather than inherited from human developers. This thesis gives one controlled answer, on one platform, rather than a general theory. What it shows is that the harness is where enterprise agents can be made more capable and more efficient, and that designing it well deserves the same care as the choice of model and data.

Bibliography

[1] Alex Singla et al. *The State of AI in 2023: Generative AI's Breakout Year*. McKinsey & Company, 2023. url: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-in-2023-generative-ais-breakout-year.

[2] Alex Singla et al. *The State of AI in Early 2024: Gen AI Adoption Spikes and Starts to Generate Value*. McKinsey & Company, 2024. url: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-2024.

[3] *2024 Work Trend Index Annual Report: AI at Work Is Here. Now Comes the Hard Part*. Microsoft and LinkedIn, 2024. url: https://www.microsoft.com/en-us/worklab/work-trend-index/ai-at-work-is-here-now-comes-the-hard-part.

[4] *The Economic Potential of Generative AI: The Next Productivity Frontier*. McKinsey Global Institute, 2023. url: https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier.

[5] *AI in Action: Beyond Experimentation to Transform Industry*. World Economic Forum, 2025. url: https://reports.weforum.org/docs/WEF_AI_in_Action_Beyond_Experimentation_to_Transform_Industry_2025.pdf.

[6] Flavio Calvino, Jelmer Reijerink, and Lea Samek. *The Effects of Generative AI on Productivity, Innovation and Entrepreneurship*. Tech. rep. OECD Artificial Intelligence Papers No. 39. OECD, 2025. doi: 10.1787/b21df222-en. url: https://www.oecd.org/en/publications/the-effects-of-generative-ai-on-productivity-innovation-and-entrepreneurship_b21df222-en.html.

[7] Akshey Sigdel and Rista Baral. "Schema First Tool APIs for LLM Agents: A Controlled Study of Tool Misuse, Recovery, and Budgeted Performance". In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2603.13404. url: https://doi.org/10.48550/arxiv.2603.13404.

[8] Yunnan Wu et al. "The Art of Tool Interface Design". In: *ArXiv.org* (2025). doi: 10.48550/arxiv.2503.21036. url: http://arxiv.org/abs/2503.21036.

[9] Heng Xu et al. "The Evolution of Tool Use in LLM Agents: From Single-Tool Call to Multi-Tool Orchestration". In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2603.22862. url: https://doi.org/10.48550/arxiv.2603.22862.

[10] MIT News Office. "Explained: Generative AI's Environmental Impact". In: *MIT News* (2025). url: https://news.mit.edu/2025/explained-generative-ai-environmental-impact-0117.

[11] Anthropic. *Code Execution with MCP: Building More Efficient AI Agents*. Anthropic Engineering Blog. 2025. url: https://www.anthropic.com/engineering/code-execution-with-mcp.

[12] Matt Carey. *Code Mode: Give Agents an Entire API in 1,000 Tokens*. Cloudflare Blog. 2026. url: https://blog.cloudflare.com/code-mode-mcp/.

[13] Kenton Varda, Sunil Pai, and Ketan Gupta. *Sandboxing AI Agents, 100x Faster*. Cloudflare Blog. 2026. url: https://blog.cloudflare.com/dynamic-workers/.

[14] Xingyao Wang et al. "Executable Code Actions Elicit Better LLM Agents". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 50208–50232. url: https://proceedings.mlr.press/v235/wang24h.html.

[15] Kai Pan. "Agent-First Tool API: A Semantic Interface Paradigm for Enterprise AI Agent Systems". In: *arXiv preprint arXiv:2605.10555* (2026). url: https://arxiv.org/abs/2605.10555.

[16] Saif Khalfan Saif Al Mazrouei. “Anka: A Domain-Specific Language for Reliable LLM Code Generation”. In: *arXiv preprint arXiv:2512.23214* (2025). url: https://arxiv.org/abs/2512.23214.

[17] Luca Mariotti et al. “Combining Large Language Models with Enterprise Knowledge Graphs: A Perspective on Enhanced Natural Language Understanding”. In: *Frontiers in Artificial Intelligence* 7 (2024). doi: 10.3389/frai.2024.1460065. url: https://www.frontiersin.org/articles/10.3389/frai.2024.1460065.

[18] Emanuele Laurenzi, Nicolas Mathys, and Thomas Martin. “An LLM-Aided Enterprise Knowledge Graph (EKG) Engineering Process”. In: *AAAI Symposium Series*. 2024. url: https://bibbase.org/network/publication/laurenzi-mathys-martin-anllmaidedenterpriseknowledgegraphekgengineeringprocess-2024.

[19] Zidi Xiu et al. “ASTRA-bench: Evaluating Tool-Use Agent Reasoning and Action Planning with Personal User Context”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2603.01357. url: https://doi.org/10.48550/arxiv.2603.01357.

[20] Hubert M. Pysklo, Artem Zhuravel, and Patrick D. Watson. “Agent-Diff: Benchmarking LLM Agents on Enterprise API Tasks via Code Execution with State-Diff-Based Evaluation”. In: *Open MIND* (2026). doi: 10.48550/arxiv.2602.11224. url: https://openalex.org/W7128767013.

[21] Aayush Gupta. “ReliabilityBench: Evaluating LLM Agent Reliability Under Production-Like Stress Conditions”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2601.06112. url: https://doi.org/10.48550/arxiv.2601.06112.

[22] Hu Wei. “Architectural Design Decisions in AI Agent Harnesses”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2604.18071. url: https://doi.org/10.48550/arxiv.2604.18071.

[23] Shunyu Yao et al. “A Benchmark for Tool-Agent-User Interaction in Real-World Domains”. In: *CoRR* abs/2406.12045 (2024). arXiv: 2406.12045. url: https://arxiv.org/abs/2406.12045.

[24] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

[25] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[26] Xinzhe Li. “A Review of Prominent Paradigms for LLM-Based Agents: Tool Use (Including RAG), Planning, and Feedback Learning”. In: *arXiv (Cornell University)* (2024). doi: 10.48550/arxiv.2406.05804. url: http://arxiv.org/abs/2406.05804.

[27] OpenAI. *Function Calling and Other API Tools*. 2026. url: https://developers.openai.com/api/docs/guides/function-calling.

[28] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2302.04761. url: http://arxiv.org/abs/2302.04761.

[29] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 3rd. Pearson, 2016. isbn: 9781292153964.

[30] Cole Stryker. *Types of AI agents*. May 2026. url: https://www.ibm.com/think/topics/ai-agent-types.

[31] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv (Cornell University)* (2022). doi: 10.48550/arxiv.2210.03629. url: http://arxiv.org/abs/2210.03629.

[32] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2303.11366. url: http://arxiv.org/abs/2303.11366.

[33] Xiang Deng et al. “SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?” In: *CoRR* abs/2310.06770 (2023). arXiv: 2310.06770. url: https://arxiv.org/abs/2310.06770.

[34] Shuyan Zhou et al. “WebArena: A Realistic Web Environment for Building Autonomous Agents”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2307.13854. url: http://arxiv.org/abs/2307.13854.

[35] Shishir G. Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2305.15334. url: http://arxiv.org/abs/2305.15334.

[36] The Open Group. *The TOGAF Standard, 10th Edition*. The Open Group, 2022. url: https://publications.opengroup.org/c220.

[37] Ardoq. *Graph Databases and Enterprise Architecture: A First-Principles Guide*. https://www.ardoq.com/knowledge-hub/graph-databases-ea. 2025.

[38] Ardoq. *Enterprise Architecture Platform Overview*. https://www.ardoq.com/platform-overview. 2026.

[39] Ardoq. *What Are Ardoq's Solution Guides, and Who Can Use Them?* https://help.ardoq.com/en/articles/43948-what-are-ardoq-s-solution-guides-and-who-can-use-them. 2025.

[40] Ardoq. *Getting Started with Application Rationalization*. https://help.ardoq.com/en/articles/44015-getting-started-with-application-rationalization. 2026.

[41] Tulio Vidal Rolim, José Renato S. Freitas, and Vânia Maria Ponte Vidal. “Metadata-Driven Construction of Semantic Views in Enterprise Knowledge Graphs with LLM Agents”. In: *Anais Estendidos do XL Simpósio Brasileiro de Banco de Dados (SBBD 2025)*. 2025. doi: 10.5753/sbbd_estendido.2025.248167. url: https://doi.org/10.5753/sbbd_estendido.2025.248167.

[42] Model Context Protocol. *Specification: Architecture and Base Protocol*. Protocol revision 2025-03-26. 2025. url: https://modelcontextprotocol.io/specification/2025-03-26/basic/index.

[43] Ardoq. *Using the Ardoq MCP Server*. Ardoq Help Center. 2026. url: https://help.ardoq.com/en/articles/415375-using-the-ardoq-mcp-server (visited on 06/19/2026).

[44] Nelson F Liu et al. “Lost in the middle: How language models use long contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173.

[45] Jiarui Lu et al. “ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities”. In: *arXiv (Cornell University)* (2024). doi: 10.48550/arxiv.2408.04682. url: http://arxiv.org/abs/2408.04682.

[46] Jacky Liang et al. “Code as Policies: Language Model Programs for Embodied Control”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2209.07753. url: http://arxiv.org/abs/2209.07753.

[47] Luyu Gao et al. “PAL: Program-Aided Language Models”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2211.10435. url: http://arxiv.org/abs/2211.10435.

[48] Wenlong Huang et al. “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *International Conference on Learning Representations (ICLR)*. 2022. url: https://arxiv.org/abs/2201.07207.

[49] Xingyao Wang, Yizhong Wang, et al. “MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback”. In: *CoRR* abs/2309.10691 (2023). arXiv: 2309.10691. url: https://arxiv.org/abs/2309.10691.

[50] Geert Trooskens et al. “Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2604.05150. url: https://doi.org/10.48550/arxiv.2604.05150.

[51] Yuval Felendler et al. “From Tool Orchestration to Code Execution: A Study of MCP Design Choices”. In: *Open MIND* (2026). doi: 10.48550/arxiv.2602.15945. url: https://openalex.org/W7130604422.

[52] Mudit Verma and Subbarao Kambhampati. “On the Brittle Foundations of ReAct Prompting for Agentic Large Language Models”. In: *arXiv (Cornell University)* (2024). doi: 10.48550/arxiv.2405.13966. url: http://arxiv.org/abs/2405.13966.

[53] Xu Huang et al. “Understanding the planning of LLM agents: A survey”. In: *arXiv (Cornell University)* (2024). doi: 10.48550/arxiv.2402.02716. url: http://arxiv.org/abs/2402.02716.

[54] Zeyu Zhang et al. “A Survey on the Memory Mechanism of Large Language Model based Agents”. In: *arXiv (Cornell University)* (2024). doi: 10.48550/arxiv.2404.13501. url: http://arxiv.org/abs/2404.13501.

[55] Yan Jiang et al. “NaviAgent: Bilevel Planning on Tool Navigation Graph for Large-Scale Orchestration”. In: *ArXiv.org* (2025). doi: 10.48550/arxiv.2506.19500. url: http://arxiv.org/abs/2506.19500.

[56] Yanna Jiang et al. “SoK: Agentic Skills – Beyond Tool Use in LLM Agents”. In: *arXiv preprint arXiv:2602.20867* (2026). url: https://arxiv.org/abs/2602.20867.

[57] Shiqi Chen et al. “SkillCraft: Can LLM Agents Learn to Use Tools Skillfully?” In: *arXiv preprint arXiv:2603.00718* (2026). url: https://arxiv.org/abs/2603.00718.

[58] Cheng Qian et al. “CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023. url: https://aclanthology.org/2023.findings-emnlp.462/.

[59] Lifan Yuan et al. “CRAFT: Customizing LLMs by Creating and Retrieving from Specialized Toolsets”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2024. url: https://arxiv.org/abs/2309.17428.

[60] Christin Pagels, Simon Hacks, and Rob Bemthuis. “Large Language Models for Analyzing Enterprise Architecture Debt in Unstructured Documentation”. In: *University of Twente Research Information* (2026). doi: 10.48550/arxiv.2604.00046. url: https://research.utwente.nl/en/publications/f37a25c9-3821-4510-9438-8808d98ad66a.

[61] Vivek Acharya. “Governing the Agentic Enterprise: A Governance Maturity Model for Managing AI Agent Sprawl in Business Operations”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2604.16338. url: https://doi.org/10.48550/arxiv.2604.16338.

[62] SAP LeanIX. *AI Capabilities*. https://help.sap.com/docs/leanix/ea/ai-capabilities. 2024.

[63] Guancheng Zeng et al. “Routine: A Structural Planning Framework for LLM Agent System in Enterprise”. In: *ArXiv.org* (2025). doi: 10.48550/arxiv.2507.14447. url: http://arxiv.org/abs/2507.14447.

[64] Xingyan Liu et al. “SkillForge: Forging Domain-Specific, Self-Evolving Agent Skills in Cloud Technical Support”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2604.08618. url: https://doi.org/10.48550/arxiv.2604.08618.

[65] Axel Dunkel. “DADL: A Declarative Description Language for Enterprise Tool Libraries in LLM Agent Systems”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2605.05247. url: https://doi.org/10.48550/arxiv.2605.05247.

[66] Petr Průcha, Michaela Matoušková, and Jan Strnad. “Are LLM Agents the New RPA? A Comparative Study with RPA Across Enterprise Workflows”. In: *arXiv (Cornell University)* (2025). doi: 10.48550/arxiv.2509.04198. url: https://doi.org/10.48550/arxiv.2509.04198.

[67] Zahra Moslemi et al. “POLARIS: Typed Planning and Governed Execution for Agentic AI in Back-Office Automation”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2601.11816. url: https://doi.org/10.48550/arxiv.2601.11816.

[68] Meriem Mastouri et al. “From REST to MCP: An Empirical Study of API Wrapping and Automated Server Generation for LLM Agents”. In: *ArXiv.org* (2025). doi: 10.48550/arxiv.2507.16044. url: http://arxiv.org/abs/2507.16044.

[69] Mina Taraghi, Mohammad Mehdi Morovati, and Foutse Khomh. “Real Faults in Model Context Protocol (MCP) Software: A Comprehensive Taxonomy”. In: *J. ACM* 37.4 (2026), 111:1–111:40. url: https://openalex.org/W7134291035.

[70] Minghao Li et al. “API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2304.08244. url: http://arxiv.org/abs/2304.08244.

[71] Xiao Liu et al. “AgentBench: A Comprehensive Benchmark to Evaluate LLMs as Agents”. In: *arXiv (Cornell University)* (2023). doi: 10.48550/arxiv.2308.03688. url: http://arxiv.org/abs/2308.03688.

[72] Kyochul Jang et al. “DICE-BENCH: Evaluating the Tool-Use Capabilities of Large Language Models in Multi-Round, Multi-Party Dialogues”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2506.22853. url: http://arxiv.org/abs/2506.22853.

[73] Pengfei He et al. “TRAJECT-Bench:A Trajectory-Aware Benchmark for Evaluating Agentic Tool Use”. In: *ArXiv.org* (2025). doi: 10.48550/arxiv.2510.04550. url: http://arxiv.org/abs/2510.04550.

[74] Smriti Jha et al. “REAP: Automatic Curation of Coding Agent Benchmarks from Interactive Production Usage”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2604.01527. url: https://doi.org/10.48550/arxiv.2604.01527.

[75] Hongliu Cao, Ilias Driouich, and Eoin Thomas. “Beyond Task Completion: Revealing Corrupt Success in LLM Agents through Procedure-Aware Evaluation”. In: *arXiv (Cornell University)* (2026). doi: 10.48550/arxiv.2603.03116. url: https://doi.org/10.48550/arxiv.2603.03116.

A Appendix

A.1 Ardoq Definitions

This section defines the Ardoq platform terms used throughout the thesis. It is intended to help readers who are not already familiar with Ardoq's terminology.

Component A typed architectural entity, such as an application, process, risk, or data entity. Components can contain custom fields.

Component Type A template or category that defines the structure and properties of components, for example Application, Database, or Process.

Dashboard A composition of widgets, often backed by reports, that presents several analytical views together.

Field A model-defined data attribute attached to a component type or reference type, such as text, numbers, or dates.

Metamodel The schema-level graph of component types, reference types, fields, and relationship triples used to interpret architectural data.

Model A schema that governs one or more workspaces by defining valid component types, reference types, hierarchy rules, and fields.

Reference A typed, directed relationship between two components. References may cross workspace boundaries.

Reference Type A template defining the nature of relationships between component types, for example "hosts", "depends on", or "communicates with".

Report A queryable, filterable tabular view of architectural data.

Traversal Navigation through the graph by following typed references from a seed set.

Viewpoint A reusable traversal template with a start type and typed paths through the architecture graph.

Workspace A model-governed container that scopes related architectural components and source references.

A.2 Full Statistical Diagnostics

Chapter 6 reports a compact table containing only the two primary architectural contrasts and the outcomes needed for the main argument. Table A.1 retains all three pairwise comparisons and the additional diagnostic metrics, with a separate Holm family containing all three pairs for each metric. Its adjusted values therefore need not equal the two-primary-contrast values in Table 6.5. Table A.2 records the exploratory difficulty correlations discussed but not foregrounded in the main chapter.

Table A.1: Pairwise Wilcoxon signed-rank tests on per-task aggregated metrics. Unit of analysis: per-task mean across runs. n = number of tasks with data for both harnesses; 7 runs per task. Mean columns describe task-level average magnitudes; median task Δ summarizes the paired differences used by the signed-rank test. This secondary diagnostic table corrects across all three pairwise comparisons within each metric; the main table uses the two planned contrasts. p_{adj}: Holm–Bonferroni-adjusted value. r_{rb}: rank-biserial correlation (effect size; positive = B > A). Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, . $p < 0.10$.

Metric	Comparison	n	Mean A	Mean B	Mean Δ	Median task Δ	W	p	p_{adj}	r_{rb}
Pass rate	H1 vs H2	24	39.3%	91.7%	+52.4pp	+64.3pp	0	0.000	0.001***	+1.000
Pass rate	H1 vs H3	24	39.3%	91.1%	+51.8pp	+64.3pp	0	0.000	0.001***	+1.000
Pass rate	H2 vs H3	24	91.7%	91.1%	−0.6pp	+0.0pp	14	0.608	0.608	−0.194
Cost (USD)	H1 vs H2	24	0.2183	0.1140	−0.1043	−0.0583	1	0.000	0.000***	−0.993
Cost (USD)	H1 vs H3	24	0.2183	0.0967	−0.1216	−0.0681	0	0.000	0.000***	−1.000
Cost (USD)	H2 vs H3	24	0.1140	0.0967	−0.0173	−0.0095	63	0.012	0.012*	−0.580
Total tokens	H1 vs H2	24	329.7k	113.2k	−216.5k	−100.4k	3	0.000	0.000***	−0.980
Total tokens	H1 vs H3	24	329.7k	91.8k	−237.8k	−96.1k	2	0.000	0.000***	−0.987
Total tokens	H2 vs H3	24	113.2k	91.8k	−21.4k	−8.7k	73	0.027	0.027*	−0.513
New input tok.	H1 vs H2	24	26.5k	14.7k	−11.8k	−8.0k	0	0.000	0.000***	−1.000
New input tok.	H1 vs H3	24	26.5k	13.2k	−13.3k	−9.0k	0	0.000	0.000***	−1.000
New input tok.	H2 vs H3	24	14.7k	13.2k	−1.5k	−1.2k	45	0.002	0.002**	−0.700
Wall-clock latency (s)	H1 vs H2	24	40.5	34.8	−5.7	+1.7	145	0.900	0.900	−0.033
Wall-clock latency (s)	H1 vs H3	24	40.5	30.8	−9.7	−0.4	109	0.252	0.534	−0.273
Wall-clock latency (s)	H2 vs H3	24	34.8	30.8	−4.0	−1.1	102	0.178	0.534	−0.320
LLM latency (s)	H1 vs H2	24	39.3	34.0	−5.2	+1.7	146	0.922	0.922	−0.027
LLM latency (s)	H1 vs H3	24	39.3	28.6	−10.7	−1.5	100	0.160	0.320	−0.333
LLM latency (s)	H2 vs H3	24	34.0	28.6	−5.5	−2.3	73	0.027	0.081.	−0.513
Tool latency (s)	H1 vs H2	24	1.2	0.7	−0.5	−0.3	47	0.002	0.004**	−0.687
Tool latency (s)	H1 vs H3	24	1.2	2.2	+1.0	+0.3	97	0.136	0.136	+0.353
Tool latency (s)	H2 vs H3	24	0.7	2.2	+1.5	+0.9	20	0.000	0.000***	+0.867
Turns	H1 vs H2	24	12.85	8.71	−4.14	−1.50	31	0.003	0.010**	−0.732
Turns	H1 vs H3	24	12.85	7.89	−4.96	−1.00	41	0.010	0.019*	−0.645
Turns	H2 vs H3	24	8.71	7.89	−0.82	−0.07	84	0.281	0.281	−0.268

Table A.2: Spearman rank correlations between task difficulty (survey mean rating, 1–5) and per-task mean metrics, reported per harness. Unit of analysis: per-task mean over all runs. Reported as exploratory; no multiple-comparison correction applied. Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, . $p < 0.10$.

Metric	H1 ρ	H1 p	H2 ρ	H2 p	H3 ρ	H3 p
Pass rate	−0.238	0.263	+0.112	0.602	−0.060	0.782
Cost (USD)	+0.303	0.151	+0.131	0.543	−0.018	0.934
Total tokens	+0.288	0.173	+0.135	0.531	−0.171	0.423
Turns	+0.268	0.206	+0.194	0.364	−0.182	0.395
Latency (s)	+0.268	0.206	+0.198	0.354	−0.144	0.503

A.3 Benchmark Task Catalogue

This section documents the 24 read-only analytical tasks used in the controlled harness comparison. Each verifier independently recomputes the expected result against the live Ardoq API, so the recorded answers track the data rather than a hardcoded value. The dated answer sizes given for individual tasks are validation snapshots taken during task development, not experimental results.

Table A.3 lists every approved task with its canonical tags and structured output type. Per-task difficulty, usefulness, and frequency ratings from the validation survey (Section 4.2.3) are reported separately in Table A.4. The narrative entries that follow give the prompt, the verification approach, and the principal reasoning hazard for each task, in task-code order. The verbatim workflow questions appear in the methodology overview (Table 4.3).

Table A.3: The 24 approved benchmark tasks: canonical tags and structured output type.

Task	Tags	Output type
T01	graph-traversal, set-difference, arm	List of risk names
T02	reports, components, graph-traversal, alm	List of (server, hosting type, lifecycle phase)
T03	reports, components, bpm	List of (activity, parent process)
T04	reports, graph-traversal, set-difference, bcm	Map: capability $\rightarrow$ owning units
T05	reports, components, graph-traversal, bvs, bcm	List of (stage, capability, maturity)
T06	reports, graph-traversal, ranking, itlm	List of (server, cost, location, products)
T07	components, graph-traversal	Per-store entity and application groups
T08	reports, aggregation, distribution, apm	List of (service level, avg cost, integrations)
T09	components, graph-traversal, date-filter, tpm	Three maps: past, future, none
T10	graph-traversal, hierarchy, aggregation, om	List of (unit, direct, indirect)
T11	viewpoints, set-difference, eaim, eaig	List of capability names
T12	viewpoints, components, graph-traversal, bcm, apm	Three lists: SAP apps, integrators, capabilities
T13	components, graph-traversal, tdm, om	List of (org unit, application)
T14	components, graph-traversal, set-difference, tpm, rec	Deployed, gap, and compliance rate
T15	metamodel, graph-traversal	Boolean plus optional path
T16	reports, cross-report, set-difference, tpm	Two product lists
T17	reports, viewpoints, components, set-intersection, tpm, arm	List of application names
T18	dashboards, reports, drill-down, eaig	List of (name, lifecycle phase)
T19	reports, cross-report, set-difference, reg	List of requirement names
T20	graph-traversal, om, apm	Org units and applications

Continued on next page

Table A.3: *(continued)*

Task	Tags	Output type
T21	dashboards, reports, drill-down, aggregation, ranking	List of section profiles
T22	metamodel, graph-traversal, ranking	Type name and in-degree
T23	viewpoints, components, alm, bcm, om	Capabilities and org units
T24	reports, aggregation, arithmetic, bcm	Two averages plus a boolean

Table A.4: Task-suite validation survey results: ratings per task ($n = 7$ developers acting as EA practitioners, 1–5 scale). Each task was rated on difficulty, usefulness, and frequency of use; the mean and median across respondents are shown. Task codes are the canonical paper codes (the survey form's renumbering translated back).

Task	Diff. mean	Diff. med.	Use. mean	Use. med.	Freq. mean	Freq. med.
T01	1.71	2.00	4.00	4.00	3.00	3.00
T02	2.29	2.00	4.29	5.00	2.43	2.00
T03	2.71	2.00	4.00	4.00	2.71	3.00
T04	2.57	3.00	3.71	4.00	2.86	3.00
T05	3.00	3.00	4.00	4.00	2.86	3.00
T06	2.86	3.00	3.86	4.00	2.57	2.00
T07	3.71	4.00	4.00	4.00	2.71	3.00
T08	2.86	3.00	4.00	4.00	3.14	3.00
T09	3.29	3.00	3.71	4.00	2.86	3.00
T10	2.86	3.00	3.43	3.00	2.14	2.00
T11	3.00	3.00	3.43	4.00	2.43	3.00
T12	3.43	4.00	3.57	4.00	2.43	2.00
T13	3.00	3.00	3.29	3.00	2.71	3.00
T14	3.86	4.00	3.86	4.00	2.57	2.00
T15	3.00	3.00	3.29	3.00	2.29	2.00
T16	3.29	3.00	3.29	3.00	2.57	2.00
T17	2.86	3.00	4.14	4.00	2.57	3.00
T18	2.71	3.00	3.71	4.00	2.71	3.00
T19	3.29	3.00	3.86	4.00	2.29	2.00
T20	2.14	2.00	3.57	4.00	2.86	2.00
T21	4.00	4.00	3.14	3.00	2.14	2.00
T22	3.00	2.00	3.00	3.00	1.71	2.00
T23	2.71	3.00	4.43	5.00	2.71	2.00

Continued on next page

Table A.4: Task-suite validation survey results: ratings per task ($n = 7$ developers acting as EA practitioners, 1–5 scale). Each task was rated on difficulty, usefulness, and frequency of use; the mean and median across respondents are shown. Task codes are the canonical paper codes (the survey form's renumbering translated back).

Task	Diff. mean	Diff. med.	Use. mean	Use. med.	Freq. mean	Freq. med.
T24	3.71	4.00	3.29	3.00	2.43	2.00

A.3.1 T01: High-Exposure Risks Without Mitigating Controls

Prompt. Which risks with a raw exposure above 15 currently have no mitigating "Control"s deployed against them? Return the distinct names of those unmitigated high-exposure risks.

Configuration. Tags: `graph-traversal`, `set-difference`, and `arm`. The oracle selects Risk components whose `raw_exposure` exceeds 15, traverses incoming "Mitigates" references restricted to the "Control" type, and subtracts the mitigated risk identifiers from the filtered set. The typed output is a sorted list of risk names. Collection metrics compare risk-name strings. The validation snapshot from 2026-05-30 contains 11 risks.

Reasoning hazard. The incoming traversal returns risks as reference targets because controls mitigate risks. The answer requires set subtraction and must exclude control categories, which mitigate at a coarser granularity than individual controls.

A.3.2 T02: Servers Supporting Retiring Applications

Prompt. We need to assess infrastructure risk ahead of upcoming application retirements. Which servers support applications that are scheduled to retire within the next 12 months and have no named successor planned? For each server, include its hosting type and current lifecycle phase. Return the list sorted alphabetically by server name.

Configuration. Tags: `reports`, `components`, `graph-traversal`, and `alm`. The oracle fetches "Apps retiring in 1 year with no successor - ALM", traverses outgoing "Is Supported By" references to servers, and batch-fetches those servers to read `server_hosting` and `lifecycle_phase`. The typed output contains triples of server name, hosting type, and lifecycle phase. Collection metrics compare these triples. The validation snapshot from 2026-05-30 contains eight servers.

Reasoning hazard. The custom fields are available only after the graph batch fetch, not from traversal stubs.

A.3.3 T03: Process Activities Missing Role Assignments

Prompt. Which process activities currently have no role assigned to them, and what is the parent process for each of those activities? Return the list sorted alphabetically by activity name, with each entry containing the activity name and its parent process.

Configuration. Tags: `reports`, `components`, and `bpm`. The oracle fetches "Process Activities missing roles - BPM DQ", batch-fetches the activities to read `parent_id`, then batch-fetches the unique parents to resolve their names. The typed output contains activity-name and parent-process pairs. Collection metrics compare these pairs. The validation snapshot from 2026-05-30 contains ten activities.

Reasoning hazard. Parent resolution follows component containment via `parent_id`; it is not a typed reference traversal.

A.3.4 T04: Unrealised Capabilities and Owning Organisational Units

Prompt. Using our predefined report, which business capabilities are not realized by any application in our portfolio? For each such capability, which organizational units are responsible for it? Return a mapping from each capability name to the list of organizational unit names that own it (empty list if none owns it).

Configuration. Tags: `reports`, `graph-traversal`, `set-difference`, and `bcm`. The oracle fetches "Business Capabilities not realized by any applications - BCM DQ", traverses incoming "Owns" references, and builds a mapping from every reported capability to its organisational units. The typed output is a dictionary from capability name to a sorted list of organisational-unit names. Collection metrics compare capability-key completeness. The validation snapshot from 2026-05-30 contains eight capabilities, only one with an owner.

Reasoning hazard. Capabilities without owners must remain in the mapping with empty lists. The owning organisational unit is the reference source.

A.3.5 T05: Value-Stream Stage Capabilities With Maturity

Prompt. We have a report called "Business Value Stream Stages (Capabilities)" covering value stream stages and our business capabilities. For each value stream stage, find which business capabilities currently realize it, and what is the maturity level of each of those capabilities? Return one entry per stage-capability pair, with the stage name, capability name, and maturity level for each pair.

Configuration. Tags: `reports`, `components`, `graph-traversal`, `bvs`, and `bcm`. The oracle fetches "Business Value Stream Stages (Capabilities) - BVS", traverses outgoing "Is Realized By" references to business capabilities, and batch-fetches the capabilities to read `maturity`. The typed output contains stage-name, capability-name, and maturity triples. Collection metrics compare these triples. The validation snapshot from 2026-05-30 contains 77 stage-capability pairs.

Reasoning hazard. Maturity is a custom field absent from traversal stubs. Stages without capability links are correctly omitted.

A.3.6 T06: Most Expensive Physical Servers

Prompt. What are our 5 most expensive physical servers based on their total direct cost from our "all physical servers metric" report? For each server, find where it is physically located and which technology products are deployed to it. Return each server's name, its annual cost, its location, and the list of technology products deployed to it, ordered by cost from highest to lowest.

Configuration. Tags: `reports`, `graph-traversal`, `ranking`, and `itlm`. The oracle fetches "All physical servers metric - ITLM", sorts by `total_direct_cost`, keeps the top five identifiers, traverses outgoing "Is Located At" references and incoming "Deploys To" references, then merges the results by server. The typed output contains server name, cost, location, and a sorted technology-product list. Collection metrics compare normalised server tuples. The validation snapshot from 2026-05-31 contains five servers.

Reasoning hazard. The two traversals use opposite directions and must run only after the top-five filter is applied.

A.3.7 T07: Classified Data Stores and Accessing Applications

Prompt. Find all data assets classified as Restricted or Confidential, which are stored on at least one data store. For each such data component, identify which data store it is stored on and which applications (NOT application modules) access that data asset.

Group the results by data store, and for each data store report: the classified data assets it holds (with their classification level), and the applications that access any of those components.

Configuration. Tags: `components` and `graph-traversal`. The oracle queries all "Data Entity" components, batch-fetches their fields, filters to "Restricted" or "Confidential", traverses outgoing "Is Stored On" references and incoming "Accesses" references, then joins the two results through component identifiers. The typed output groups component-classification pairs and accessing applications by data store. Collection metrics compare normalised store entries. The validation snapshot from 2026-05-31 contains three data stores.

Reasoning hazard. This is an indirect store $\leftarrow$ entity $\rightarrow$ application join. Application modules access data entities through the same reference type as applications and must be excluded, and entities not stored on any data store fall out of scope.

A.3.8 T08: Application Cost and Integrations by Service Level

Prompt. From the applications and attributes report, what is the average total direct app cost for each service level category, and how many integrations does each category have in total? Return each category name, the average cost rounded to 2 decimal places, and the total integration count, sorted alphabetically by category name.

Configuration. Tags: `reports`, `aggregation`, `distribution`, and `apm`. The oracle fetches "All Applications and attributes - APM", groups rows by service level, averages non-null total direct cost values, sums integration counts while treating null as zero, and sorts the categories. The typed output contains category, average-cost, and integration-count triples. Collection metrics compare these triples. The validation snapshot from 2026-05-31 contains four service-level categories.

Reasoning hazard. Null handling differs by field: missing costs are excluded from the average, while missing integration counts contribute zero.

A.3.9 T09: Candidate Vulnerabilities by Product Review Date

Prompt. Among vulnerabilities marked as a candidate, identify which technology products each one impacts. Partition the results into three groups based on the technology product's review date: (1) products whose review date is before today, (2) products whose review date is after today, (3) products with no review date set. For each group, return a mapping from vulnerability name to the list of technology product names it impacts in that group.

Configuration. Tags: `components`, `graph-traversal`, `date-filter`, and `tpm`. The oracle queries and batch-fetches vulnerabilities, filters to candidates, traverses outgoing "Impacts" references, batch-fetches the impacted technology products, and partitions them by `review_date`. The typed output contains `past`, `future`, and `none` mappings. Collection metrics compare bucket, vulnerability-name, and product-name triples. The validation snapshot from 2026-05-31 contains 11 such triples.

Reasoning hazard. A vulnerability may appear in multiple buckets. Candidates without "Impacts" links are omitted rather than represented with empty lists. The answer is relative to the execution date.

A.3.10 T10: Organisational Headcount Overview

Prompt. Build an organizational headcount overview. For each organizational unit, compute two figures: how many people are directly assigned to it via "belongs to", and how many people belong to it indirectly, meaning the total number of people across all of its

child units recursively (0 if it has no child units). Return one entry per unit as `(unit_name, direct_count, indirect_count)`.

Configuration. Tags: `graph-traversal`, `hierarchy`, `aggregation`, and `om`. The oracle reconstructs the organisational-unit hierarchy, counts the people directly attached to each unit through "Belongs To", and then sums those direct counts over every descendant unit to produce the indirect figure. The typed output is a list of unit triples.

Reasoning hazard. The indirect count is a bottom-up recursive sum over the unit tree, not a single traversal hop. Leaf units contribute a zero indirect count, and people must not be counted twice across nested units.

A.3.11 T11: Critical Capabilities Without AI Support

Prompt. Which business capabilities at the highest criticality level (5) are not currently supported by any AI capability? Use the business capabilities supported by AI viewpoint, starting from the critical capabilities. Return the capability names that are not linked to an AI-supported system.

Configuration. Tags: `viewpoints`, `set-difference`, `eaim`, and `eaig`. The oracle selects business capabilities at criticality 5, executes the "business capabilities supported by AI" viewpoint from that seed set, and subtracts the capabilities reached through an AI-supported system from the critical set. The typed output is a list of capability names.

Reasoning hazard. The answer is the complement within the critical set. Capabilities with no AI link are the ones to return, so the result depends on set subtraction against the viewpoint output rather than on the viewpoint output itself.

A.3.12 T12: Applications Integrating With SAP-Supplied Systems

Prompt. SAP supplies several applications in our portfolio. Using the application integration map viewpoint, which applications in our portfolio integrate with at least one SAP-supplied application? For those integrating applications, which business capabilities do they realize? Return all the apps SAP supplies, the names of any integrating applications, and the realized business capability names.

Configuration. Tags: `viewpoints`, `components`, `graph-traversal`, `bcm`, and `apm`. The oracle resolves the SAP-supplied applications, executes the application integration map viewpoint to find the applications that integrate with them, and follows realisation references from those integrators to business capabilities. The typed output is three sorted lists: the SAP-supplied applications, the integrating applications, and the realised capabilities.

Reasoning hazard. Three distinct entity sets are drawn from one traversal. An application qualifies as an integrator only if it connects to a SAP-supplied application specifically, and the capabilities are a further hop beyond the integrators.

A.3.13 T13: Deliberately Delayed Debt Items

Prompt. Which debt items have been deliberately marked as action being delayed? For those delayed debt items, which applications do they impact, and which organizational units consume those applications? Return a list of `(organizational_unit_name, application_name)` pairs, representing every org unit consuming an application with deliberately delayed technical debt.

Configuration. Tags: `components`, `graph-traversal`, `tdm`, and `om`. The oracle filters Debt Item components whose recorded action is the delayed value, traverses "Impacts"

references to the affected applications, and then follows incoming “Consumes” references from organisational units. The typed output is a list of org-unit and application pairs.

Reasoning hazard. This is a two-hop chain from debt item to application to consuming unit. Only the deliberately delayed action value qualifies, and the resulting pairs are deduplicated.

A.3.14 T14: Mandated Products and Deployment Compliance

Prompt. Which technology products are mandated by approved architecture standards? Of those, which have at least one live deployment, and which have no deployments at all? Return the deployed product names, gap product names, and the compliance rate (fraction of mandated products that have live deployments, rounded to 2 decimal places).

Configuration. Tags: `components`, `graph-traversal`, `set-difference`, `tpm`, and `rec`. The oracle identifies the technology products mandated by approved architecture standards, checks each for a live deployment, partitions them into deployed and gap sets, and divides the deployed count by the full mandated count. The typed output reports both product lists and the compliance rate.

Reasoning hazard. Products without any deployment must enter the gap set rather than being dropped, and the compliance rate is taken over the complete mandated set, not over the deployed subset.

A.3.15 T15: Metamodel Path From Application to Risk

Prompt. Using the organization’s metamodel, is there any chain of references that connects an “Application” component type to a “Risk” component type (in that direction), potentially through intermediate component types? Return whether such a path exists. If yes, return one valid path as an alternating list starting with “Application” and ending with “Risk”, where even indices are component type names and odd indices are reference type names.

Configuration. Tags: `metamodel` and `graph-traversal`. The oracle runs a breadth-first search over the metamodel triple set from Application toward Risk and returns an existence flag together with one valid alternating path of component-type and reference-type names. The verifier checks that the path has odd length, starts and ends with the required types, and that every hop corresponds to a real metamodel triple.

Reasoning hazard. The search runs over the schema graph rather than the instance graph. The output structure is strict: any path that misorders types and reference types, or that includes a non-existent triple, fails verification even when a path does exist.

A.3.16 T16: Approved Versus Standardised Technology Products

Prompt. Most of our technology products carry an “approved” flag, indicating whether they have been reviewed as accurate and complete. Which deployed products still carry that flag but no longer appear in the active standardized-products report? And which products are on the standardized-products report without the approved flag set in the deployed-products report?

Configuration. Tags: `reports`, `cross-report`, `set-difference`, and `tpm`. The oracle joins the deployed-products report against the active standardized-products report on product identity and computes the two asymmetric differences: approved-but-not-standardised, and standardised-but-not-approved. The typed output is the two product lists.

Reasoning hazard. The task asks for two directional differences across two reports,

which are easily conflated. Products must be matched across reports by identity rather than by name alone.

A.3.17 T17: Doubly Exposed At-Risk Applications

Prompt. We have a report listing applications currently flagged as at risk, and a separate report of technology products whose primary support ends within the next year. Which of those at-risk applications also have one of those end-of-support products in their technology stack? Use the "Application Technology stack - TPM" viewpoint to trace each at-risk application's tech stack, then cross-reference with the ending-support products. Return the names of the double-exposed applications.

Configuration. Tags: `reports`, `viewpoints`, `components`, `set-intersection`, `tpm`, and `arm`. The oracle reads the at-risk applications report and the ending-support products report, traces each at-risk application's stack through the "Application Technology stack - TPM" viewpoint, and intersects that stack with the ending-support set. The typed output is a list of double-exposed application names.

Reasoning hazard. The answer is an intersection mediated by a per-application traversal. An application qualifies only when its traced stack contains at least one ending-support product, so the two reports cannot simply be intersected directly.

A.3.18 T18: AI Systems Without an Owner

Prompt. Our AI governance dashboard tracks ownership of AI systems in the portfolio. Using that dashboard, which AI systems currently have no owner assigned? Return their names and lifecycle phases.

Configuration. Tags: `dashboards`, `reports`, `drill-down`, and `eaig`. The oracle locates the ownership widget on the AI governance dashboard, drills through to its backing report, and selects the AI systems with no owner, reading each one's lifecycle phase. The typed output is a list of name and lifecycle-phase pairs.

Reasoning hazard. The answer requires drilling from a dashboard widget to its source report. Reading the widget's aggregate value alone does not yield the underlying systems.

A.3.19 T19: Unmapped EU DORA Requirements

Prompt. Which specific EU DORA requirements have not been mapped to any controls yet? Use the "EU DORA requirements total" report to get all requirements, and the "EU DORA mapped requirements" report to get the mapped ones.

Configuration. Tags: `reports`, `cross-report`, `set-difference`, and `reg`. The oracle subtracts the mapped-requirement set drawn from "EU DORA mapped requirements" from the full set drawn from "EU DORA requirements total". The typed output is a list of requirement names.

Reasoning hazard. This is the simplest task in the suite and serves as an overhead anchor. It still requires reading two reports and taking their difference rather than performing a single lookup.

A.3.20 T20: Applications Owned by a Person's Units

Prompt. Which applications do the organizational units Darryl Kerrigan is connected to own? Find all the organizational units he has a direct reference to, then identify the applications those units own, if any. Return the org unit names and the sorted list of application names.

Configuration. Tags: `graph-traversal`, `om`, and `apm`. The oracle resolves the person "Darryl Kerrigan", collects the organisational units directly referenced from him, and fol-

lows “Owns” references from those units to applications. The typed output reports the org units and the sorted applications.

Reasoning hazard. The traversal is anchored on a named person and is limited to directly referenced units. Units that own no applications still belong in the org-unit list.

A.3.21 T21: Data-Quality Dashboard Section Profiles

Prompt. The enterprise architecture data quality dashboard tracks several component types, named in its section headers. For each component type section, report a profile: how many entities of that exact type the dashboard shows, how many distinct reports the section draws from, and which single report is used by the most widgets in that section.

Configuration. Tags: `dashboards`, `reports`, `drill-down`, `aggregation`, and `ranking`. The oracle splits the data-quality dashboard into header-delimited sections, reads the entity count each section reports, counts the distinct backing reports per section, and identifies the report backing the most widgets. The typed output is one profile per section.

Reasoning hazard. The task depends on dashboard structure, the section headers and each widget’s report binding, rather than on widget values. The most-used report is an argmax over per-report widget counts within a section.

A.3.22 T22: Most Referenced Component Type

Prompt. According to our architecture metamodel, which component type is referenced by the greatest number of distinct other (not itself) component types? Return the type name and that count (in-degree).

Configuration. Tags: `metamodel`, `graph-traversal`, and `ranking`. The oracle treats the metamodel as a directed multigraph of component types and, for each type, counts the distinct other source types that reference it, then takes the maximum. The typed output is the type name and its in-degree.

Reasoning hazard. In-degree counts distinct source types, not reference instances, and self-references are excluded from the count.

A.3.23 T23: Adobe Retirement Impact

Prompt. We are planning to retire all our Adobe applications. Using the retiring application landscape impact viewpoint, which business capabilities do they collectively realize, and which directly connected organizational units consume them? Return the explicitly connected business capabilities and org units.

Configuration. Tags: `viewpoints`, `components`, `alm`, `bcm`, and `om`. The oracle resolves the Adobe applications, executes the retiring application landscape impact viewpoint from them, and collects the directly connected business capabilities and consuming organisational units. The typed output reports both lists.

Reasoning hazard. Only explicitly and directly connected capabilities and units count. Indirect reachability through the viewpoint must not inflate the answer.

A.3.24 T24: Differentiating Versus Commodity Capability Maturity

Prompt. We have differentiating and commodity business capabilities. Using the “all business capabilities” report, what is the average maturity score for the differentiating group (market differentiation ≥ 3) versus the commodity group (market differentiation < 3)? Ignore any capabilities where maturity is null. Are our differentiating capabilities actually better maintained? Return both averages, rounded to 2 decimal places, and whether the differentiating group is higher.

Configuration. Tags: `reports`, `aggregation`, `arithmetic`, and `bcm`. The oracle reads the "all business capabilities" report, partitions rows at a market-differentiation threshold of 3, averages maturity within each group while ignoring null values, and compares the two means. The typed output is the two averages and the comparison boolean.

Reasoning hazard. Rows with null maturity are excluded from both averages rather than treated as zero, and the boolean reflects a direct comparison of the two group means.

A.3.25 Reasoning Patterns

Table A.5: Retrieval and analysis patterns represented across the benchmark suite.

Pattern	Retrieval and analysis sequence	Tasks
1	Report fetch $\rightarrow$ in-memory aggregation	T08, T24
2	Report fetch $\rightarrow$ traversal $\rightarrow$ set or mapping operation	T02, T04, T05
3	Report fetch $\rightarrow$ parent or graph fetch	T03
4	Report fetch $\rightarrow$ two traversals $\rightarrow$ merge or rank	T06
5	Component query $\rightarrow$ traversal $\rightarrow$ set difference	T01, T13, T14
6	Component query $\rightarrow$ two traversals $\rightarrow$ join or partition	T07, T09
7	Recursive hierarchy aggregation	T10
8	Viewpoint execution $\rightarrow$ set or multi-hop projection	T11, T12, T17, T23
9	Dashboard widget $\rightarrow$ backing-report drill-down	T18, T21
10	Cross-report set difference	T16, T19
11	Metamodel search or degree analysis	T15, T22
12	Entity-anchored multi-hop traversal	T20

A.4 Supplementary Evaluation Tables

Table A.6: Shared harness configuration.

Parameter	H1	H2	H3
Model	`claude-sonnet-4-6` (all harnesses)		
Canonical variant	H1b (compact results)	H2b (dict-schema docs)	single configuration
Framework retries	2	2	2
Default run limit	45 turns	45 turns	45 turns
Default run timeout	90 seconds	90 seconds	90 seconds
Repetitions per task	7	7	7
Advertised interface	All 20 JSON tool schemas	`execute_code` with inline dictionary schemas	`execute_code` with domain module reference
State mechanism	Conversation history	Python variables and execution output	Python variables and normalised domain-operation outputs

Table A.7: Selected fields from the serialised trial-log schema.

Field	Type	Description
`task`	`TaskSpec`	Prompt, tags, and per-case limits.
`harness`	Enum	Harness variant: h1a, h1b, h2a, h2b, or h3.
`run_index`	Integer	Zero-based repetition index for the task–harness pair.
`success`	Boolean	Whether all verification conditions passed.
`latency_ms`	Float	Wall-clock duration from task start to terminal result.
`usage`	Object	Token counts, request count, tool calls, cache usage, and estimated cost.
`fail_category`	Enum	Terminal outcome such as pass, wrong answer, explicit failure, usage limit, timeout, or error.
`verify_result`	Object	Oracle verdict, expected and actual values, and collection statistics.
`turns`	List	Per-turn model, tool-call, and timing records.
`sandbox_errors`	Object	H2/H3 syntax, runtime, and API error counts.
`code_metrics`	Object or null	Generated-code line counts for H2/H3; null for H1.

A.5 Harness System Prompts

Each harness uses a fixed system prompt that is identical across all 24 tasks; the task-specific question is supplied separately as the user message, and the interface documentation, JSON tool schemas for H1 or the API and SDK reference embedded in the `execute_code` description for H2 and H3, is attached by the framework rather than written into the system prompt (Section 4.3.4). The three prompts below share a common skeleton, the same role framing, answer procedure, and grounding principles, and differ only in how they instruct the model to reach data and read results: H1 lists tool capabilities, while H2 and H3 describe a persistent Python execution environment and substitute the raw-API and domain-module idioms respectively. They are reproduced verbatim from the evaluated configuration.

```
You are an enterprise architecture assistant for Ardoq with access to the full set of data tools.

## Your Capabilities

- **Metamodel**: Explore component types, reference types, and field definitions
- **Components**: Search for specific components by name
- **Reports**: List, inspect, and query report data with filtering and aggregation
- **Viewpoints**: List and execute viewpoints to traverse the EA graph
- **Dashboards**: Access dashboard definitions and data
- **Graph**: Query the in-memory EA graph directly - list component types, filter by fields, traverse paths, aggregate values

## How to Answer

1. Use tools to gather data before answering. Never guess or fabricate data.
2. If a question requires data you cannot retrieve, return a FailResult with the most accurate fail_type and a concise reason.
3. Return <name>Result when you have a grounded, tool-verified answer.
4. Include the IDs or names of all Ardoq resources you used in the `sources` field.

## Core Principles

- Data-only answers: every claim must be backed by a tool result
- Exact accuracy: use exact names, IDs, and values as returned by tools
- Preserve source strings verbatim: when reporting entity names, capability names, or any other string values from Ardoq data, copy them exactly as they appear - do not paraphrase, reformat, or normalize them
- If something does not exist in the data, say so clearly - do not infer
- Use tool search and filter parameters whenever available - pass search terms, filters, and column selectors rather than fetching everything and filtering manually
- When querying report, dashboard, or viewpoint data: retrieve the definition first (columns, filter schema, available fields) so you can apply appropriate filters and column selectors in the data fetch
- When looking for reports, viewpoints, dashboards, or components: use regex name filters with alternation and partial patterns to match the intent of the user's query - e.g. `"capability|capabilities|bc"`, `"application.*portfolio|portfolio.*app"`. Prefer partial matches (no anchors) so substring hits are not missed. Try several keyword variants and synonyms before concluding a resource does not exist.
- **Batch exploratory tool calls.** When discovering what data exists, issue multiple tool calls in the same turn rather than waiting for each result before deciding to call the next. For example: if you want to find a capability report, an application portfolio report, and a technology
```

```
    dashboard all at once - search for all three in parallel, not one after
    the other. This is an efficiency move: fewer round-trips, fewer tokens
    consumed.
- For component filtering: call list_component_types() first to discover valid
     type names; call get_metamodel_fields(type_name) to verify field names
    before constructing filters for filter_components()
```

Listing A.1: System prompt for the H1 (JSON tool-calling) harness, identical across all tasks

```
You are an enterprise architecture assistant for Ardoq. You answer questions
    by writing and executing Python code against the Ardoq API.

## How to Answer

1. Call `execute_code` with Python to query Ardoq data. Call it multiple times
     as needed.
2. Start every code block with a `# comment` stating what this execution is
    trying to find out.
3. Use `print()` in your code - that is how you see results.
4. Produce a `<name>Result` or `FailResult` based on what the code returned.
5. Never guess or fabricate data. Every claim must come from printed code
    output.

## REPL Environment

Persistent Python REPL - variables and imports defined in one `execute_code`
    call persist into all subsequent calls for this task, exactly like Jupyter
     notebook cells. Fetch data once, store in a variable, process or filter
    in the next call. Never re-fetch data you already have.

## Core Principles

- Every claim must be backed by printed output - never guess or infer
- Use exact names, IDs, and values as returned; copy strings verbatim - do not
     paraphrase or normalize
- If something does not exist in the data, say so - do not infer
- **Never print full objects.** Extract only the fields you need. Bad: `print(
    row)`. Good: `print(row['name'], row['status'])`
- When looking for reports, viewpoints, dashboards, or components: use regex
    alternation and partial patterns - e.g. `"capability|capabilities|bc"`, `"
    application.*portfolio|portfolio.*app"`. Prefer partial matches (no
    anchors). Try several keyword variants before concluding a resource does
    not exist.
- Batch all exploration into one code block: discover multiple things in a
    single `execute_code` call, print only what you need to decide next.
- Fewer `execute_code` calls is better. Independent fetches and non-sequential
     steps belong in the same block. Split into multiple calls only when you
    need to inspect output before deciding what to do next.
```

Listing A.2: System prompt for the H2 (programmatic) harness, identical across all tasks

```
You are an enterprise architecture assistant for Ardoq. You answer questions
    by writing and executing Python code against the Ardoq domain module.

## How to Answer

1. Call `execute_code` with Python to query Ardoq data. Call it multiple times
     as needed.
2. Start every code block with a `# comment` stating what this execution is
    trying to find out.
```

```
3. Use `print()` in your code - that is how you see results.
4. Produce a `<name>Result` or `FailResult` based on what the code returned.
5. Never guess or fabricate data. Every claim must come from printed code output.

## REPL Environment

Persistent Python REPL - variables and imports defined in one `execute_code` call persist into all subsequent calls for this task, exactly like Jupyter notebook cells. Fetch data once, store in a variable, process or filter in the next call. Never re-fetch data you already have.

## Core Principles

- Every claim must be backed by printed output - never guess or infer
- Use exact names, IDs, and values as returned; copy strings verbatim - do not paraphrase or normalize
- If something does not exist in the data, say so - do not infer
- **Never print full objects.** Extract only the fields you need. Bad: `print(row)`. Good: `print(row.name, row.status)`
- When looking for reports, viewpoints, dashboards, or components: use regex alternation and partial patterns - e.g. `Q.name("capability|capabilities|bc")`, `Q.name("application.*portfolio|portfolio.*app")`. Prefer partial matches (no anchors). Try several keyword variants before concluding a resource does not exist.
- Batch all exploration into one code block: discover multiple things in a single `execute_code` call, print only what you need to decide next.
- Fewer `execute_code` calls is better. Independent fetches and non-sequential steps belong in the same block. Split into multiple calls only when you need to inspect output before deciding what to do next.
```

Listing A.3: System prompt for the H3 (agent-native) harness, identical across all tasks

A.6 H3 Tool-Description Patterns and SDK Examples

The H3 agent received more than the system prompt reproduced above. Its single `execute_code` tool description contained an executor preamble, the 18 pre-imported SDK names, six hand-written usage patterns, and a generated SDK reference. The patterns were part of the evaluated interface. They demonstrated how objects returned by one domain operation could become inputs to another, which was not apparent to the model from signatures alone during interface development.

The structural reference was produced by `generate_sdk_docs_v2()`. It parses the SDK source with Python's `ast` module, removes private implementation members, replaces public method bodies with ellipses, and preserves signatures, type annotations, and docstrings. The examples were maintained separately by hand because they document intended combinations of classes rather than individual signatures. Table A.8 lists the six pattern groups included in the frozen main-campaign tool description.

Table A.8: Curated H3 usage patterns supplied in the evaluated tool description.

Pattern	Interface behaviour demonstrated
Complex querying	Combine type, name, and field predicates with Boolean operators
Metamodel introspection	Discover valid types, fields, and relationship triples before retrieval
Mixed-direction traversal	Pass queried components into a multi-hop builder and inspect the resulting subgraph
Frontier filtering	Narrow a materialised seed set before a hop to remain within the traversal limit
Viewpoint traversal	Combine component and viewpoint queries, then analyse the returned subgraph
Report analysis	Fetch once, then filter, summarise, and aggregate rows locally

The following listings reproduce the evaluated idioms in expanded form. They are supplementary examples rather than additional experimental traces.

```
mm = await Metamodel.fetch_full()

# Inspect the fields and outgoing relationships of Application.
detail = mm.component_type_detail("Application")
print([(f.name, f.field_type, f.options)
       for f in detail.fields])
print([(r.reference_type, r.target_type)
       for r in detail.outgoing_relationships])

# The query expression describes intent; fetch() materialises it.
apps = await ComponentQuery(
    Q.type("Application")
    & Q.field("lifecycle_phase").eq("Live")
    & Q.field("vendor").is_not_null()
).fetch()

print(len(apps))
print([(a.name, a.get_field("vendor")) for a in apps[:10]])
```

Listing A.4: H3 metamodel discovery followed by a declarative component query.

```
servers = await ComponentQuery(Q.type("Server")).fetch()

graph = await (
    TraversalBuilder(servers)
    .hop("Is Located At", direction="outgoing",
         target_type="Location")
    .hop("Deploys To", direction="incoming",
         source_type="Application")
).build()

print(graph.summary())
print(graph.count_by("Application", "lifecycle_phase"))

for edge in graph.edges_of("Deploys To"):
    print(edge.source.name, edge.target.name)

# Derive the relationship structure already observed in the result.
for triple in graph.metamodel().triples:
```

```
    print(triple.source_type, triple.reference_type,
          triple.target_type)
```

Listing A.5: H3 composition from queried components through mixed-direction traversal and local graph analysis.

```
applications = await ComponentQuery(
    Q.type("Application")
).fetch()

graph = await (
    TraversalBuilder(applications)
    .filter(Q.field("lifecycle_phase").eq("Live"))
    .hop("Depends on", direction="outgoing")
    .hop("Deployed on", direction="outgoing",
         target_type="Infrastructure")
).build()

print(graph.summary())
print([c.name for c in graph.nodes_of("Infrastructure")])
```

Listing A.6: H3 frontier filtering before a bounded multi-hop traversal.

```
reports = await ReportQuery(Q.name("capabilit.*")).fetch()
report = reports[0]
print(report.total, [column.key for column in report.columns])

live = report.where(
    Q.field("lifecycle_phase").eq("Live")
)
print(live.field_summary("business_unit"))

cost_by_domain = live.group_sum("domain", "annual_cost")
print(sorted(cost_by_domain.items(),
             key=lambda item: item[1], reverse=True))
```

Listing A.7: H3 retrieval followed by local report analysis.

```
viewpoints = await ViewpointQuery(
    Q.name("application architecture")
).fetch()
seeds = await ComponentQuery(
    Q.type("Application") & Q.name("payments")
).fetch()

graph = await viewpoints[0].traverse(components=seeds)
print(graph.summary())

realised = graph.filter_triples(
    ref=Q.type("Realizes"),
    target=Q.type("Business Capability"),
)
print([c.name for c in
       realised.nodes_of("Business Capability")])
```

Listing A.8: H3 composition of viewpoint discovery, component seeds, and subgraph filtering.

A.7 Post-Run Reflection Schema and Prompt

When a trial terminates in a non-passing outcome (specifically, a wrong answer, turn-limit exhaustion, or timeout), the evaluation framework performs a single, separate reflection

call. This call uses the same model as the trial itself and is forced to return structured output matching the Pydantic `FailResult` schema. The schema is defined as:

```python
class FailResult(BaseModel):
    fail_type: FailCauseType  # e.g., 'data_not_found', 'wrong_approach'
    reason: str               # Brief explanation of the failure
```

Listing A.9: FailResult schema for agent self-report.

Where `FailCauseType` is a literal union of the following categories: `data_not_found`, `data_incomplete`, `field_not_found`, `ambiguous_question`, `conflicting_data`, `tool_errors`, `missing_tools`, `too_many_results`, `wrong_approach`, `data_shape_unexpected`, and `other`.

The post-run reflection prompt is constructed as follows:

> *You are analyzing a trial run that has terminated without a successful verification. Review the conversation history below, including the task prompt, all tool invocations, observations, and any execution errors.*
>
> *Determine why the task failed or could not be completed. Classify the root cause of the failure using one of the categories defined in the output schema, and provide a short, direct explanation of the failure based on the run history.*

In the case of a timeout, the history includes any pending tool requests that were in flight at the boundary. The resulting self-report is stored for analysis but is treated as secondary diagnosis rather than ground-truth evaluation.

Table A.9 lists representative reflections, three per harness, chosen to span the failure classes discussed in Section 6.6. The reflections are reproduced verbatim except for elisions marked `[…]` and the removal of long literal component identifiers. They illustrate that the self-reports are typically accurate root-cause analyses and that the perceived cause differs systematically by harness: clerical computation over in-context data for H1, data-acquisition and completion timing for H2, and semantic selection of the wrong set or relation for H3.

Table A.9: Representative agent self-reported failure reflections per harness, with the self-reported cause category and terminal outcome. Reflections are verbatim apart from elisions (`[…]`) and removed literal identifiers.

H	Task	Self-reported cause	Outcome	Reflection (abridged)
H1	T22	`wrong_approach`	Wrong answer	My manual count of distinct source component types referencing “Application” […] was incorrect — I counted 19 but the expected answer is 21. A more reliable approach would be to programmatically parse and deduplicate all source types […], but […] I made a counting error.
H1	T05	`too_many_results`	Timeout	The traversal returned 161 “Is Realized By” references across 21 value stream stages […]. Fetching maturity for all unique Business Capability targets required many sequential `get_component_fields` calls, and the task timed out before all maturity values could be collected […].

(continued on next page)

Table A.9: *(continued)*

H	Task	Self-reported cause	Outcome	Reflection (abridged)
H1	T19	`wrong_approach`	Wrong answer	When computing the set difference between total and mapped requirements, I incorrectly included raw component IDs […] in the unmapped list instead of resolving them to their requirement names […], resulting in 3 extra entries that should have been […] excluded.
H2	T12	`other`	Timeout	The analysis completed successfully — all data was retrieved and processed — but the wall-clock time limit of 90 seconds was exceeded before a final answer could be returned. All results were available: 11 SAP-supplied applications, 9 integrating non-SAP applications, and 10 realized business capabilities […].
H2	T10	`data_incomplete`	Wrong answer	The indirect counts for Operations (22 vs expected 25) and PMO (3 vs expected 5) are undercounted. The root cause is that my set-based deduplication across child OUs treated people belonging to multiple child units as a single person, when the expected answer counts total “belongs to” relationships (i.e., with multiplicity) rather than distinct people.
H2	T04	`wrong_approach`	Wrong answer	The verifier expects only “Owns” incoming references from Organizational Units to count as ownership, not “Is Realized By” outgoing references. My answer incorrectly included org units linked via “Is Realized By” […]. The correct mapping should only use the “Owns” reference type […].
H3	T12	`data_incomplete`	Wrong answer	Two SAP-supplied applications — “Ariba” and “Business Objects” — were missed because the query only searched for applications with names containing “SAP”. Consequently, Ariba was incorrectly treated as a non-SAP integrating app […]. The realized business capabilities were also affected as a result.
H3	T16	`wrong_approach`	Wrong answer	The “approved but not in standardized report” list was too broad. The question asks specifically about *deployed* products with the approved flag — but the query fetched all 152 Technology Products with `approved=True`, rather than filtering to only those that are actually deployed. […] the result included 142 items instead of 5.
H3	T11	`wrong_approach`	Wrong answer	The traversal was seeded only with the 64 critical (criticality=5) business capabilities, but the viewpoint returned a subgraph of only 17 of them — missing 47 that have no connections at all in the graph. […] BCs with zero connections were silently dropped, and those disconnected ones are precisely the ones NOT supported by AI.

A.8 Representative Trial Trace

A.8.1 T13: Delayed Technical Debt and Consuming Organizational Units

This trace uses the first available H1 failure and the first passing H2 and H3 runs for T13. The task asks for organizational-unit/application pairs where the organizational unit consumes an application directly impacted by a debt item whose action is marked as delayed.

```
Which debt items have been deliberately marked as action being delayed?
For those delayed debt items, which applications do they impact, and which
organizational units consume those applications? Return a list of
(organizational_unit_name, application_name) pairs, representing every org
unit consuming an application with deliberately delayed technical debt.
```

Listing A.10: T13 task prompt.

```
Harness: H1b
Run: 1
Passed: false
Fail category: wrong_answer
Turns: 13
Tool calls issued: 17
Total tokens: 249444

Tool calls issued in the run (abridged):
  1. list_reports("tech.*debt|debt|technical debt")
  2. list_viewpoints("tech.*debt|debt")
  3. get_report_definition("Business impact of live Tech Debt - TD")
  4. filter_components(type="Debt Item", debt_action="Delay")
  5. get_report_data("Business impact of live Tech Debt - TD",
                     columns=["Debt Item", "Applications",
                              "Organizational Units"])
  6. get_viewpoint_data("Business impact of live Debt - TD",
                        start=[two delayed Debt Item ids])

Verifier outcome:
  expected pairs: 5
  actual pairs: 9
  precision=0.5556, recall=1.0, f1=0.7143
  missing: none
  extra:
    (IT Operations, SQL Server Data Warehouse (Marketing))
    (IT Operations, SmartChat)
    (Operations, Product D Web Portal)
    (Project Team Alpha, AppDynamics)
```

Listing A.11: T13 H1b abridged trace and verifier outcome, run 1.

```
delayed_debt = await get_report_data(
    "4c6012d689dc70215be1ae84",
    filters={"rules": [{"column_name": "debt_action",
                        "operator": "equal",
                        "value": "Delay"}]},
    include_columns=["_id", "name", "debt_action", "Impacts--outgoing"]
)

debt_ids = ["7a67f3d3702f7381072aec10",
            "108b20ff4fe82a6609b0fa34"]

direct_apps = await traverse_one_hop(
    debt_ids, direction="outgoing", component_type="Application"
)

app_ids = [
    "913c4fb33e6a9afa2b47e283",
    "4f0d805bba213b19749f7c63",
    "57285f7c1a177d410dea21d9",
    "4d1388375b859070448ebc88",
]

```

```
consumers = await traverse_one_hop(
    app_ids, direction="incoming",
    reference_type="Consumes",
    component_type="Organizational Unit"
)
```

Listing A.12: T13 H2b generated-code excerpt, run 1.

```
Harness: H2b
Run: 1
Passed: true
Fail category: pass
Turns: 8
execute_code calls issued: 7
Wrapped sandbox API calls: 7
Total tokens: 93729
Verifier: precision=1.0, recall=1.0, f1=1.0
```

Listing A.13: T13 H2b run summary, run 1.

```
delayed_debt = await ComponentQuery(
    Q.type("Debt Item") & Q.field("debt_action").eq("Delay")
).fetch()

g = await (
    TraversalBuilder(delayed_debt)
    .hop("Impacts", direction="outgoing", target_type="Application")
).build()

g2 = await (
    TraversalBuilder(g.nodes_of("Application"))
    .hop("Consumes", direction="incoming",
         source_type="Organizational Unit")
).build()

pairs = sorted(
    (ref.source.name, ref.target.name)
    for ref in g2.edges_of("Consumes")
)
```

Listing A.14: T13 H3 generated-code excerpt, run 1.

```
Harness: H3
Run: 1
Passed: true
Fail category: pass
Turns: 8
execute_code calls issued: 7
Total tokens: 82637
Verifier: precision=1.0, recall=1.0, f1=1.0
```

Listing A.15: T13 H3 run summary, run 1.

```
(Customer Support, SmartChat)
(Operations, Product Pricing Database)
(Ordering Services Dev, Product Pricing Database)
(Product Services Dev, Product D Sales Operational Data Store)
(Product Services Dev, Product D Web Portal)
```

Listing A.16: T13 oracle result shared by the passing H2b and H3 runs.

A.9 Task-suite Validation Survey Instrument

The text below is the title and description shown to respondents of the task-suite validation survey (Section 4.2.3). Respondents then rated each of the 24 benchmark tasks on the three dimensions on a 1–5 scale. The pooled and per-task results are reported in Tables 6.1 and A.4.

AI Agent Task Benchmark Evaluation

Part of Alexander Nygård's thesis research, titled: Agent–System Interface Designs for Enterprise Platforms.

Thesis goal: Compare and evaluate 3 unique agent harness designs on a data-retrieval benchmark of EA tasks in Ardoq.

The below 24 tasks constitute the developed benchmark, covering a wide range of questions and focus areas (evaluated on the seed data in our dev environments).

To evaluate the usefulness and realism of the created tasks, you're asked to act as an EA, and evaluate each question across 3 dimensions:

- *Difficulty — if you were given this task, how difficult would it be for you?*
- *Usefulness — how useful/relevant is the answer to the question in your workflow?*
- *Frequency of use — is the question type something you would ask rarely, or often?*

Survey takes ~10 mins, 24 questions.

Even if you don't personally have experience with EA tasks/workflows, please answer what you think, based on your understanding of the platform and use-cases. No wrong answers here!

Note: no names or emails will be included in the thesis. Only a summary of respondents' roles, and mean/median scores for each task will be included.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
DK-2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk